

Вопросы к вступительному экзамену в аспирантуру

8 сентября 2026 г.

Содержание

1	Вероятность, условная вероятность	12
1.1	Случайные события и вероятностное пространство	12
1.2	Аксиомы вероятности и основные свойства	12
1.3	Классическое и геометрическое определения вероятности	12
1.4	Условная вероятность	12
1.5	Формулы умножения, полной вероятности и Байеса	12
1.6	Независимость событий	12
1.7	Схема Бернулли	12
2	Выпуклые задачи поиска минимума функционала	13
2.1	Выпуклые множества и выпуклые функции	13
2.2	Выпуклые функционалы	13
2.3	Существование и единственность минимума	13
2.4	Необходимые и достаточные условия оптимальности	13
2.5	Субдифференциал и условия субградиентного типа	13
2.6	Основные численные методы	13
3	Линейные непрерывные функционалы	14
3.1	Линейные функционалы	14
3.2	Непрерывность и ограниченность	14
3.3	Норма линейного функционала	14
3.4	Сопряжённое пространство	14
3.5	Теорема Хана–Банаха	14
3.6	Теорема Рисса о представлении функционала	14
4	Математическое программирование	15
4.1	Общая постановка задачи	15
4.2	Классификация задач математического программирования	15
4.3	Линейное программирование	15
4.4	Двойственность в линейном программировании	15
4.5	Нелинейное программирование	15
4.6	Условия Каруша–Куна–Таккера	15
4.7	Целочисленное и дискретное программирование	15
5	Метрические и нормированные пространства	16
5.1	Метрические пространства	16
5.2	Сходимость и фундаментальные последовательности	16
5.3	Полнота и пополнение метрического пространства	16
5.4	Открытые, замкнутые и компактные множества	16
5.5	Нормированные пространства	16
5.6	Банаховы пространства	16
5.7	Эквивалентность норм	16

6	Оптимальное управление	17
6.1	Постановка задачи оптимального управления	17
6.2	Управляемая система и допустимые управления	17
6.3	Функционал качества и ограничения	17
6.4	Принцип максимума Понтрягина	17
6.5	Метод динамического программирования	17
6.6	Уравнение Гамильтона–Якоби–Беллмана	17
6.7	Линейно-квадратичная задача	17
6.8	Численные методы оптимального управления	17
7	Основы вариационного исчисления	18
7.1	Функционалы и вариации	18
7.2	Первая вариация функционала	18
7.3	Уравнение Эйлера–Лагранжа	18
7.4	Естественные граничные условия	18
7.5	Задачи с ограничениями	18
7.6	Изопериметрические задачи и множители Лагранжа	18
7.7	Вторая вариация и достаточные условия экстремума	18
8	Понятие меры и интеграла Лебега	19
8.1	Сигма-алгебра и измеримое пространство	19
8.2	Мера и её основные свойства	19
8.3	Измеримые функции	19
8.4	Интеграл простых и неотрицательных функций	19
8.5	Интеграл Лебега	19
8.6	Основные свойства интеграла Лебега	19
8.7	Теоремы о предельном переходе	19
8.8	Сравнение интегралов Римана и Лебега	19
9	Пространства интегрируемых функций	20
9.1	Пространства L^p	20
9.2	Норма и классы эквивалентных функций	20
9.3	Неравенства Гёльдера и Минковского	20
9.4	Полнота пространств L^p	20
9.5	Гильбертово пространство L^2	20
9.6	Сопряжённые пространства	20
9.7	Вложения пространств L^p	20
10	Пространства Соболева. Обобщённые производные	21
10.1	Обобщённые функции и слабые производные	21
10.2	Определение пространств Соболева	21
10.3	Норма и полнота пространств Соболева	21
10.4	Плотность гладких функций	21
10.5	Теоремы вложения	21
10.6	Теоремы о следах	21
10.7	Слабая постановка краевых задач	21
11	Случайные величины и векторы	22
11.1	Случайная величина и функция распределения	22
11.2	Дискретные и непрерывные распределения	22
11.3	Математическое ожидание и моменты	22
11.4	Преобразования случайных величин	22
11.5	Случайные векторы и совместное распределение	22
11.6	Маргинальные и условные распределения	22
11.7	Условное математическое ожидание	22
11.8	Основные предельные теоремы	22

12	Элементы корреляционной теории случайных векторов	23
12.1	Ковариация и коэффициент корреляции	23
12.2	Ковариационная и корреляционная матрицы	23
12.3	Свойства ковариационной матрицы	23
12.4	Линейные преобразования случайных векторов	23
12.5	Некоррелированность и независимость	23
12.6	Линейная регрессия и наилучший линейный прогноз	23
12.7	Метод главных компонент	23
13	Элементы теории случайных процессов	24
13.1	Определение случайного процесса	24
13.2	Конечномерные распределения	24
13.3	Математическое ожидание и корреляционная функция	24
13.4	Стационарные случайные процессы	24
13.5	Непрерывность и дифференцируемость в среднем квадратичном	24
13.6	Гауссовские процессы	24
13.7	Марковские процессы	24
13.8	Спектральное представление стационарных процессов	24
14	Булевы функции	25
14.1	Основные определения	25
14.2	Разложение по переменной и нормальные формы	25
14.3	Полином Жегалкина	25
14.4	Двойственность, симметрия и монотонность	25
14.5	Функциональная полнота. Теорема Поста	26
14.6	Минимизация булевых формул	26
14.7	Что важно сказать на экзамене	26
15	Графы и их представление в ЭВМ	27
15.1	Основные определения	27
15.2	Маршруты, цепи и циклы	27
15.3	Матрица смежности	27
15.4	Матрица инцидентности	27
15.5	Списки смежности	28
15.6	Список рёбер	28
15.7	Выбор представления	28
15.8	Изоморфизм графов	28
15.9	Что важно сказать на экзамене	28
16	Деревья и их основные свойства	29
16.1	Определение дерева	29
16.2	Листья и степени вершин	29
16.3	Корневые деревья	29
16.4	Упорядоченные и бинарные деревья	29
16.5	Центр дерева	30
16.6	Остовные деревья	30
16.7	Код Прюфера	30
16.8	Что важно сказать на экзамене	30
17	Комбинаторика	31
17.1	Основные правила подсчёта	31
17.2	Перестановки, размещения и сочетания	31
17.3	Биномиальные и мультиномиальные коэффициенты	31
17.4	Формула включений и исключений	32
17.5	Разбиения множеств	32
17.6	Производящие функции	32
17.7	Рекуррентные соотношения	32
17.8	Что важно сказать на экзамене	33

18 Множества и отношения	34
18.1 Множества и операции над ними	34
18.2 Мощност и декартово произведение	34
18.3 Бинарные отношения	34
18.4 Свойства отношений на одном множестве	34
18.5 Отношение эквивалентности	35
18.6 Отношения порядка	35
18.7 Функции как специальные отношения	35
18.8 Что важно сказать на экзамене	35
19 Потоки в сетях. Теорема Форда и Фалкерсона. Алгоритмы нахождения максимального потока	36
19.1 Транспортная сеть и допустимый поток	36
19.2 Остаточная сеть	36
19.3 Разрез сети	36
19.4 Теорема Форда–Фалкерсона	37
19.5 Алгоритм Форда–Фалкерсона	37
19.6 Алгоритм Эдмондса–Карпа	37
19.7 Алгоритм Диница	37
19.8 Целочисленность и применения	37
19.9 Что важно сказать на экзамене	37
20 Представление деревьев в ЭВМ	38
20.1 Общие подходы	38
20.2 Массив родителей	38
20.3 Списки детей	38
20.4 Связное представление бинарного дерева	38
20.5 Последовательное представление бинарного дерева	39
20.6 Обходы как способ кодирования	39
20.7 Код Прюфера	39
20.8 Польская запись деревьев выражений	39
20.9 Что важно сказать на экзамене	39
21 Связность графов. Алгоритмы обхода графов	40
21.1 Компоненты связности	40
21.2 Теорема Менгера	40
21.3 Поиск в ширину	40
21.4 Поиск в глубину	40
21.5 Мосты и точки сочленения	41
21.6 Связность ориентированных графов	41
21.7 Топологическая сортировка	41
21.8 Что важно сказать на экзамене	41
22 Аппроксимация функциональных зависимостей	42
22.1 Постановка задачи	42
22.2 Нормы и критерии качества	42
22.3 Линейная аппроксимационная модель	42
22.4 Основные классы аппроксимирующих функций	42
22.5 Теорема Вейерштрасса	43
22.6 Равномерное наилучшее приближение	43
22.7 Выбор сложности модели	43
22.8 Что важно сказать на экзамене	43
23 Вычислительные методы линейной алгебры	44
23.1 Задачи вычислительной линейной алгебры	44
23.2 Метод Гаусса	44
23.3 LU -разложение	44
23.4 Разложение Холецкого	44
23.5 Метод прогонки	45
23.6 QR -разложение	45
23.7 Стационарные итерационные методы	45
23.8 Методы подпространств Крылова	45

23.9 Собственные значения	46
23.10 Обусловленность и невязка	46
23.11 Что важно сказать на экзамене	46
24 Метод конечных разностей	47
24.1 Основная идея	47
24.2 Уравнение теплопроводности	47
24.3 Волновое уравнение	47
24.4 Эллиптическое уравнение	48
24.5 Граничные условия	48
24.6 Исследование устойчивости	48
24.7 Преимущества и ограничения	48
24.8 Что важно сказать на экзамене	48
25 Метод конечных элементов. Лагранжевы элементы	49
25.1 Идея метода конечных элементов	49
25.2 Галёркинская аппроксимация	49
25.3 Лагранжевы конечные элементы	49
25.4 Линейный треугольный элемент	50
25.5 Локальные матрицы и сборка	50
25.6 Отображение с эталонного элемента	50
25.7 Граничные условия	50
25.8 Сходимость и оценка погрешности	50
25.9 Что важно сказать на экзамене	50
26 Метод конечных элементов. Эрмитовы элементы	51
26.1 Назначение эрмитовых элементов	51
26.2 Слабая постановка задачи изгиба балки	51
26.3 Кубический эрмитов элемент в одном измерении	51
26.4 Локальная матрица жёсткости балки	52
26.5 Существенные и естественные условия	52
26.6 Эрмитова интерполяция и порядок точности	52
26.7 Многомерные C^1 -элементы	52
26.8 Сравнение с лагранжевыми элементами	53
26.9 Что важно сказать на экзамене	53
27 Метод наименьших квадратов	54
27.1 Линейная задача наименьших квадратов	54
27.2 Геометрический смысл	54
27.3 Численно устойчивое решение	54
27.4 Аппроксимация табличных данных	55
27.5 Непрерывный метод наименьших квадратов	55
27.6 Взвешенный и регуляризованный МНК	55
27.7 Нелинейный метод наименьших квадратов	56
27.8 Статистическая интерпретация	56
27.9 Что важно сказать на экзамене	56
28 Полиномиальная интерполяция	57
28.1 Постановка задачи и единственность	57
28.2 Форма Лагранжа	57
28.3 Барическая формула	57
28.4 Форма Ньютона и разделённые разности	57
28.5 Погрешность интерполяции	58
28.6 Феномен Рунге и узлы Чебышёва	58
28.7 Интерполяция Эрмита	58
28.8 Что важно сказать на экзамене	58

29 Сплайн-аппроксимация	59
29.1 Определение сплайна	59
29.2 Интерполирующий кубический сплайн	59
29.3 Граничные условия	59
29.4 Вариационное свойство	59
29.5 Оценки погрешности	60
29.6 Сглаживающий сплайн	60
29.7 B -сплайны	60
29.8 Что важно сказать на экзамене	60
30 Численное дифференцирование	61
30.1 Постановка задачи	61
30.2 Формулы для первой производной	61
30.3 Вторая и высшие производные	61
30.4 Вывод методом неопределённых коэффициентов	61
30.5 Неравномерная сетка	62
30.6 Погрешность и выбор шага	62
30.7 Экстраполяция Ричардсона	62
30.8 Дифференцирование зашумлённых данных	62
30.9 Комплексный шаг	63
30.10 Что важно сказать на экзамене	63
31 Численное интегрирование	64
31.1 Квадратурные формулы	64
31.2 Формулы Ньютона–Котеса	64
31.3 Составные формулы	64
31.4 Правило Рунге и адаптивное интегрирование	65
31.5 Квадратуры Гаусса	65
31.6 Специальные квадратуры	65
31.7 Несобственные интегралы и особенности	65
31.8 Ошибки вычислений	66
31.9 Что важно сказать на экзамене	66
32 Численные методы поиска экстремума функций	67
32.1 Постановка задачи	67
32.2 Одномерная минимизация	67
32.3 Градиентный спуск	67
32.4 Метод Ньютона	67
32.5 Квазиньютоновские методы	68
32.6 Метод сопряжённых градиентов	68
32.7 Условная оптимизация	68
32.8 Критерии остановки	68
32.9 Что важно сказать на экзамене	68
33 Численные методы решения систем обыкновенных дифференциальных уравнений	69
33.1 Задача Коши	69
33.2 Погрешность, порядок и сходимость	69
33.3 Методы Эйлера и трапеций	69
33.4 Методы Рунге–Кутты	69
33.5 Вложенные методы и автоматический выбор шага	70
33.6 Линейные многошаговые методы	70
33.7 Методы дифференцирования назад	70
33.8 Устойчивость и жёсткие системы	71
33.9 Решение неявного шага	71
33.10 Краевые задачи	71
33.11 Что важно сказать на экзамене	71

34 Вариационные принципы построения математических моделей	72
34.1 Понятие вариационного принципа	72
34.2 Функционал действия и энергетические функционалы	72
34.3 Принцип стационарного действия	72
34.4 Принцип возможных перемещений	72
34.5 Уравнения Эйлера–Лагранжа	72
34.6 Учёт связей и ограничений	72
34.7 Диссипативные системы	72
34.8 Примеры вариационных моделей	72
35 Вычислительный эксперимент в математическом моделировании	73
35.1 Понятие и цели вычислительного эксперимента	73
35.2 Этапы вычислительного эксперимента	73
35.3 Постановка серии расчётов	73
35.4 Планирование вычислительного эксперимента	73
35.5 Верификация и валидация	73
35.6 Анализ чувствительности	73
35.7 Обработка и интерпретация результатов	73
35.8 Воспроизводимость вычислений	73
36 Источники погрешности в математическом моделировании	74
36.1 Погрешность математической модели	74
36.2 Погрешность исходных данных и параметров	74
36.3 Погрешность дискретизации	74
36.4 Итерационная погрешность	74
36.5 Погрешность округления	74
36.6 Накопление и распространение ошибок	74
36.7 Устойчивость вычислительного алгоритма	74
36.8 Оценка полной погрешности	74
37 Краевые задачи для дифференциальных уравнений в частных производных	75
37.1 Классификация дифференциальных уравнений в частных производных	75
37.2 Начальные и граничные условия	75
37.3 Эллиптические краевые задачи	75
37.4 Параболические начально-краевые задачи	75
37.5 Гиперболические начально-краевые задачи	75
37.6 Корректность постановки задачи	75
37.7 Классическая и слабая постановки	75
37.8 Основные численные методы решения	75
38 Математические модели в биологии и биоинформатике (скип)	76
38.1 Особенности биологических систем как объектов моделирования	76
38.2 Популяционные модели	76
38.3 Эпидемиологические модели	76
38.4 Модели типа реакция–диффузия	76
38.5 Модели биохимических сетей	76
38.6 Модели последовательностей в биоинформатике	76
38.7 Филогенетические модели	76
38.8 Идентификация и проверка биологических моделей	76
39 Математические модели в механике деформируемого твёрдого тела	77
39.1 Кинематика деформирования	77
39.2 Тензоры деформаций и напряжений	77
39.3 Уравнения баланса	77
39.4 Определяющие соотношения	77
39.5 Начальные и граничные условия	77
39.6 Линейная теория упругости	77
39.7 Пластичность и вязкоупругость	77
39.8 Численные методы механики твёрдого тела	77

40 Математические модели в экономике	78
40.1 Особенности экономических моделей	78
40.2 Балансовые модели	78
40.3 Оптимизационные модели	78
40.4 Динамические модели экономического роста	78
40.5 Модели межотраслевого баланса	78
40.6 Игровые модели	78
40.7 Стохастические модели	78
40.8 Идентификация и проверка экономических моделей	78
41 Математические модели механики жидкости и газа	79
41.1 Гипотеза сплошной среды	79
41.2 Кинематические характеристики потока	79
41.3 Законы сохранения массы, импульса и энергии	80
41.4 Определяющие соотношения	81
41.5 Уравнения Эйлера и Навье–Стокса	81
41.6 Начальные и граничные условия	82
41.7 Критерии подобия и безразмерные параметры	83
41.8 Модели турбулентности	84
41.9 Основные численные методы	84
42 Проверка адекватности математических моделей	86
42.1 Понятие адекватности модели	86
42.2 Верификация и валидация	86
42.3 Сопоставление с экспериментальными данными	87
42.4 Критерии качества модели	88
42.5 Калибровка параметров	89
42.6 Идентифицируемость модели	90
42.7 Анализ чувствительности	90
42.8 Учёт неопределённости	91
43 Универсальность математических моделей	93
43.1 Абстрагирование при построении моделей	93
43.2 Принцип аналогий	93
43.3 Подobie и безразмеризация	94
43.4 Масштабные законы	95
43.5 Универсальные математические структуры	96
43.6 Перенос моделей между предметными областями	97
43.7 Границы применимости универсальных моделей	97
44 Базовые концепции объектно-ориентированного программирования	99
44.1 Объекты и классы	99
44.2 Инкапсуляция	99
44.3 Наследование	100
44.4 Полиморфизм	101
44.5 Абстракция и интерфейсы	101
44.6 Композиция и агрегация	102
44.7 Принципы проектирования объектно-ориентированных систем	103
44.8 Преимущества и ограничения объектно-ориентированного подхода	104
45 Компьютерная графика. Работа с графическими библиотеками	105
45.1 Основные задачи компьютерной графики	105
45.2 Растровая и векторная графика	105
45.3 Геометрические преобразования и системы координат	107
45.4 Графический конвейер	108
45.5 Растеризация и визуализация	109
45.6 Графические программные интерфейсы	110
45.7 Шейдеры и программируемый конвейер	111
45.8 Научная визуализация и интерактивная графика	111

46 Пакеты прикладных программ для математического моделирования	114
46.1 Понятие пакета прикладных программ	114
46.2 Классификация пакетов математического моделирования	114
46.3 Архитектура и основные компоненты	115
46.4 Численные решатели	116
46.5 Подготовка исходных данных и постобработка	117
46.6 Форматы данных и интеграция	118
46.7 Верификация программного комплекса	119
46.8 Критерии выбора пакета	120
47 Представление о языках программирования высокого уровня	121
47.1 Уровни языков программирования	121
47.2 Синтаксис и семантика	121
47.3 Типы данных и системы типов	122
47.4 Основные парадигмы программирования	123
47.5 Компиляция и интерпретация	124
47.6 Организация памяти и выполнение программы	124
47.7 Библиотеки и экосистема языка	125
47.8 Критерии выбора языка программирования	126
48 Суперкомпьютерные технологии	127
48.1 Архитектура высокопроизводительных вычислительных систем	127
48.2 Параллелизм и уровни параллельной обработки	128
48.3 Иерархия памяти	128
48.4 Распределённая память и технология MPI	129
48.5 Общая память и технология OpenMP	130
48.6 Вычисления на графических процессорах	130
48.7 Декомпозиция вычислительной задачи	131
48.8 Производительность, ускорение и масштабируемость	132
48.9 Отказоустойчивость и воспроизводимость	133
49 Основные понятия и принципы математического моделирования. Прямые и обратные задачи математического моделирования. Универсальность математических моделей. Принцип аналогий. Иерархия моделей	135
49.1 Основные понятия математического моделирования	135
49.2 Объект, модель и цель моделирования	136
49.3 Этапы построения математической модели	136
49.4 Прямые задачи	137
49.5 Обратные задачи	138
49.6 Универсальность моделей	139
49.7 Принцип аналогий	139
49.8 Иерархия моделей	140
49.9 Проверка и уточнение модели	141
50 Интерполирование функций, полиномы Чебышева, интерполяция с кратными узлами, кубические сплайны	142
50.1 Постановка задачи интерполяции	142
50.2 Интерполяционные многочлены Лагранжа и Ньютона	142
50.3 Погрешность полиномиальной интерполяции	142
50.4 Полиномы и узлы Чебышева	142
50.5 Интерполяция с кратными узлами	142
50.6 Интерполяционный многочлен Эрмита	142
50.7 Кубические интерполяционные сплайны	142
50.8 Граничные условия для кубического сплайна	142
50.9 Сравнение глобальной и сплайн-интерполяции	142

51 Исследование операций и задачи искусственного интеллекта. Экспертизы и неформальные процедуры. Автоматизация проектирования	143
51.1 Предмет и задачи исследования операций	143
51.2 Модели принятия решений	144
51.3 Оптимизационные задачи исследования операций	144
51.4 Основные задачи искусственного интеллекта	145
51.5 Экспертные системы	146
51.6 Экспертизы и неформальные процедуры	147
51.7 Системы автоматизированного проектирования	147
51.8 Интеграция оптимизации, искусственного интеллекта и проектирования	148
52 Вариационные методы решения краевых задач и определения собственных значений	150
52.1 Слабая постановка краевой задачи	150
52.2 Энергетический функционал	150
52.3 Метод Рунге	150
52.4 Метод Галёркина	150
52.5 Вариационная постановка задачи на собственные значения	150
52.6 Отношение Рэлея	150
52.7 Минимаксные принципы	150
52.8 Сходимость и оценка погрешности	150
53 Вычислительный эксперимент. Принципы проведения вычислительного эксперимента. Модель, алгоритм, программа	151
53.1 Понятие вычислительного эксперимента	151
53.2 Цели и задачи вычислительного эксперимента	151
53.3 Триада «модель–алгоритм–программа»	152
53.4 Этапы проведения вычислительного эксперимента	153
53.5 Планирование серии расчётов	154
53.6 Верификация алгоритма и программы	155
53.7 Валидация математической модели	156
53.8 Анализ и представление результатов	156
53.9 Воспроизводимость вычислительного эксперимента	157
54 Моделирование методом динамики частиц. Молекулярная динамика	159
54.1 Частичная модель вещества	159
54.2 Уравнения движения частиц	159
54.3 Потенциалы межчастичного взаимодействия	159
54.4 Начальные и граничные условия	159
54.5 Численное интегрирование уравнений движения	159
54.6 Списки соседей и вычислительная эффективность	159
54.7 Термостаты и статистические ансамбли	159
54.8 Вычисление макроскопических характеристик	159
54.9 Преимущества и ограничения молекулярной динамики	159
55 Алгоритмические языки. Представление о языках программирования высокого уровня. Пакеты прикладных программ	160
55.1 Алгоритм и алгоритмический язык	160
55.2 Синтаксис и семантика языка	161
55.3 Языки низкого и высокого уровня	162
55.4 Парадигмы программирования	162
55.5 Трансляторы: компиляторы и интерпретаторы	163
55.6 Среда выполнения и библиотеки	164
55.7 Пакеты прикладных программ	165
55.8 Критерии выбора языка и программного пакета	167
56 Численные методы вычисления определителей и обратных матриц	168
56.1 Вычисление определителя методом Гаусса	168
56.2 Вычисление определителя по LU -разложению	168
56.3 Вычисление обратной матрицы через решение систем	168
56.4 Метод Гаусса–Жордана	168
56.5 Обусловленность задачи обращения матрицы	168

56.6	Численная устойчивость и выбор главного элемента	168
56.7	Оценка вычислительной сложности	168
56.8	Особенности разреженных и структурированных матриц	168
57	Метод граничных элементов	169
57.1	Основная идея метода граничных элементов	169
57.2	Фундаментальное решение и функция Грина	169
57.3	Переход к граничному интегральному уравнению	169
57.4	Дискретизация границы	169
57.5	Граничные элементы и аппроксимация неизвестных	169
57.6	Формирование системы линейных уравнений	169
57.7	Сингулярные интегралы	169
57.8	Преимущества и ограничения метода	169
57.9	Сравнение с методом конечных элементов	169
58	Алгоритмы сортировки	170
58.1	Постановка задачи сортировки	170
58.2	Устойчивость и другие свойства алгоритмов	170
58.3	Сортировки вставками, выбором и обментами	171
58.4	Сортировка слиянием	172
58.5	Быстрая сортировка	173
58.6	Пирамидальная сортировка	174
58.7	Сортировки без сравнений	174
58.8	Оценка временной и пространственной сложности	175
58.9	Нижняя оценка для сортировок сравнениями	175
58.10	Внешняя сортировка и выбор алгоритма	176

1 Вероятность, условная вероятность

1.1 Случайные события и вероятностное пространство

1.2 Аксиомы вероятности и основные свойства

1.3 Классическое и геометрическое определения вероятности

1.4 Условная вероятность

1.5 Формулы умножения, полной вероятности и Байеса

1.6 Независимость событий

1.7 Схема Бернулли

2 Выпуклые задачи поиска минимума функционала

2.1 Выпуклые множества и выпуклые функции

2.2 Выпуклые функционалы

2.3 Существование и единственность минимума

2.4 Необходимые и достаточные условия оптимальности

2.5 Субдифференциал и условия субградиентного типа

2.6 Основные численные методы

3 Линейные непрерывные функционалы

3.1 Линейные функционалы

3.2 Непрерывность и ограниченность

3.3 Норма линейного функционала

3.4 Сопряжённое пространство

3.5 Теорема Хана–Банаха

3.6 Теорема Рисса о представлении функционала

4 Математическое программирование

4.1 Общая постановка задачи

4.2 Классификация задач математического программирования

4.3 Линейное программирование

4.4 Двойственность в линейном программировании

4.5 Нелинейное программирование

4.6 Условия Каруша–Куна–Таккера

4.7 Целочисленное и дискретное программирование

5 Метрические и нормированные пространства

5.1 Метрические пространства

5.2 Сходимость и фундаментальные последовательности

5.3 Полнота и пополнение метрического пространства

5.4 Открытые, замкнутые и компактные множества

5.5 Нормированные пространства

5.6 Банаховы пространства

5.7 Эквивалентность норм

6 Оптимальное управление

6.1 Постановка задачи оптимального управления

6.2 Управляемая система и допустимые управления

6.3 Функционал качества и ограничения

6.4 Принцип максимума Понтрягина

6.5 Метод динамического программирования

6.6 Уравнение Гамильтона–Якоби–Беллмана

6.7 Линейно-квадратичная задача

6.8 Численные методы оптимального управления

7 Основы вариационного исчисления

7.1 Функционалы и вариации

7.2 Первая вариация функционала

7.3 Уравнение Эйлера–Лагранжа

7.4 Естественные граничные условия

7.5 Задачи с ограничениями

7.6 Изопериметрические задачи и множители Лагранжа

7.7 Вторая вариация и достаточные условия экстремума

8 Понятие меры и интеграла Лебега

8.1 Сигма-алгебра и измеримое пространство

8.2 Мера и её основные свойства

8.3 Измеримые функции

8.4 Интеграл простых и неотрицательных функций

8.5 Интеграл Лебега

8.6 Основные свойства интеграла Лебега

8.7 Теоремы о предельном переходе

8.8 Сравнение интегралов Римана и Лебега

9 Пространства интегрируемых функций

9.1 Пространства L^p

9.2 Норма и классы эквивалентных функций

9.3 Неравенства Гёльдера и Минковского

9.4 Полнота пространств L^p

9.5 Гильбертово пространство L^2

9.6 Сопряжённые пространства

9.7 Вложения пространств L^p

10 Пространства Соболева. Обобщённые производные

10.1 Обобщённые функции и слабые производные

10.2 Определение пространств Соболева

10.3 Норма и полнота пространств Соболева

10.4 Плотность гладких функций

10.5 Теоремы вложения

10.6 Теоремы о следах

10.7 Слабая постановка краевых задач

- 11 Случайные величины и векторы
 - 11.1 Случайная величина и функция распределения
 - 11.2 Дискретные и непрерывные распределения
 - 11.3 Математическое ожидание и моменты
 - 11.4 Преобразования случайных величин
 - 11.5 Случайные векторы и совместное распределение
 - 11.6 Маргинальные и условные распределения
 - 11.7 Условное математическое ожидание
 - 11.8 Основные предельные теоремы

12 Элементы корреляционной теории случайных векторов

12.1 Ковариация и коэффициент корреляции

12.2 Ковариационная и корреляционная матрицы

12.3 Свойства ковариационной матрицы

12.4 Линейные преобразования случайных векторов

12.5 Некоррелированность и независимость

12.6 Линейная регрессия и наилучший линейный прогноз

12.7 Метод главных компонент

13 Элементы теории случайных процессов

13.1 Определение случайного процесса

13.2 Конечномерные распределения

13.3 Математическое ожидание и корреляционная функция

13.4 Стационарные случайные процессы

13.5 Непрерывность и дифференцируемость в среднем квадратичном

13.6 Гауссовские процессы

13.7 Марковские процессы

13.8 Спектральное представление стационарных процессов

14 Булевы функции

14.1 Основные определения

Булевой функцией от n переменных называется отображение

$$f: \{0, 1\}^n \rightarrow \{0, 1\}.$$

Множество всех булевых функций от n переменных обозначают через P_n . Функция может быть задана формулой, таблицей истинности, вектором значений или одной из нормальных форм.

Переменная x_i называется *существенной* для функции f , если существуют два набора, отличающиеся только значением x_i , на которых значения f различны. Если такого набора нет, переменная фиктивна.

К основным булевым операциям относятся

$$\neg x, \quad x \wedge y, \quad x \vee y, \quad x \oplus y, \quad x \rightarrow y = \neg x \vee y,$$

а также штрих Шеффера $x | y = \neg(x \wedge y)$ и стрелка Пирса $x \downarrow y = \neg(x \vee y)$.

Булева алгебра подчиняется законам коммутативности, ассоциативности, дистрибутивности, поглощения и идемпотентности. Особенно важны законы де Моргана:

$$\neg(x \wedge y) = \neg x \vee \neg y, \quad \neg(x \vee y) = \neg x \wedge \neg y.$$

14.2 Разложение по переменной и нормальные формы

Для любой переменной x_i справедливо разложение Шеннона:

$$f(x_1, \dots, x_n) = (\neg x_i \wedge f|_{x_i=0}) \vee (x_i \wedge f|_{x_i=1}).$$

Последовательное применение этой формулы даёт представление функции через элементарные конъюнкции или дизъюнкции.

Минтермом называется конъюнкция, содержащая каждую переменную ровно один раз — с отрицанием или без него. Минтерм равен единице ровно на одном наборе. Дизъюнкция минтермов, соответствующих единичным строкам таблицы истинности, называется совершенной дизъюнктивной нормальной формой (СДНФ):

$$f(x_1, \dots, x_n) = \bigvee_{\alpha: f(\alpha)=1} x_1^{\alpha_1} \wedge \dots \wedge x_n^{\alpha_n},$$

где $x^1 = x$, $x^0 = \neg x$.

Двойственным образом строится совершенная конъюнктивная нормальная форма (СКНФ) как конъюнкция макстермов, соответствующих нулевым строкам таблицы истинности. Для функции, не равной тождественно нулю или единице, СДНФ и СКНФ определены однозначно с точностью до перестановки членов.

14.3 Полином Жегалкина

Любая булева функция единственным образом представляется полиномом над полем $\mathbb{F}_2$:

$$f(x_1, \dots, x_n) = a_0 \oplus \bigoplus_i a_i x_i \oplus \bigoplus_{i < j} a_{ij} x_i x_j \oplus \dots \oplus a_{12\dots n} x_1 x_2 \dots x_n,$$

где коэффициенты принимают значения 0 или 1. Это представление называется полиномом Жегалкина, или алгебраической нормальной формой.

Функция называется *линейной*, если её полином Жегалкина имеет степень не выше одной:

$$f = a_0 \oplus a_1 x_1 \oplus \dots \oplus a_n x_n.$$

14.4 Двойственность, симметрия и монотонность

Двойственной к f называется функция

$$f^*(x_1, \dots, x_n) = \neg f(\neg x_1, \dots, \neg x_n).$$

Замена $\wedge \leftrightarrow \vee$ и $0 \leftrightarrow 1$ в формуле приводит к формуле для двойственной функции. Функция самодвойственна, если $f = f^*$.

Функция симметрична, если её значение не меняется при любой перестановке переменных. Тогда значение зависит только от числа единиц в наборе.

На множестве $\{0, 1\}^n$ вводится покоординатный порядок: $\alpha \leq \beta$, если $\alpha_i \leq \beta_i$ для всех i . Функция называется монотонной, если

$$\alpha \leq \beta \implies f(\alpha) \leq f(\beta).$$

14.5 Функциональная полнота. Теорема Поста

Система булевых функций F называется *полной*, если любую булеву функцию можно выразить формулой, использующей функции из F и переменные. Классические полные системы:

$$\{\neg, \wedge\}, \quad \{\neg, \vee\}, \quad \{\mid\}, \quad \{\downarrow\}.$$

В теореме Поста используются пять замкнутых классов:

1. T_0 — функции, сохраняющие ноль: $f(0, \dots, 0) = 0$;
2. T_1 — функции, сохраняющие единицу: $f(1, \dots, 1) = 1$;
3. S — самодвойственные функции;
4. M — монотонные функции;
5. L — линейные функции.

Теорема Поста. Система F функционально полна тогда и только тогда, когда она не содержится целиком ни в одном из классов T_0, T_1, S, M, L . Иными словами, в системе должны существовать функции, нарушающие каждое из пяти указанных свойств.

14.6 Минимизация булевых формул

Задача минимизации состоит в уменьшении числа литералов, конъюнкций или логических операций при сохранении реализуемой функции. Основные методы:

- карты Карно — наглядный метод для малого числа переменных;
- метод Квайна–Мак-Класки — табличное построение простых импликант;
- использование законов булевой алгебры и поглощения.

Сокращённая ДНФ строится из простых импликант; минимальная ДНФ выбирается среди покрытий всех единичных наборов минимальной стоимости.

14.7 Что важно сказать на экзамене

Булева функция задаётся таблицей истинности и имеет канонические представления: СДНФ, СКНФ и полином Жегалкина. Полнота системы функций проверяется по теореме Поста. Для практической реализации логических схем существенны минимизация формул и выбор базиса.

15 Графы и их представление в ЭВМ

15.1 Основные определения

Графом называется пара

$$G = (V, E),$$

где V — множество вершин, а E — множество рёбер. В неориентированном графе ребро является неупорядоченной парой $\{u, v\}$, в ориентированном графе дуга является упорядоченной парой (u, v) .

Граф может быть простым, мультиграфом, псевдографом, ориентированным, взвешенным или помеченным. В простом графе нет петель и кратных рёбер. Число вершин обозначим $n = |V|$, число рёбер — $m = |E|$.

Вершины u и v называются смежными, если они соединены ребром. Степень вершины v — число инцидентных ей рёбер:

$$d(v) = |\{e \in E : v \in e\}|.$$

Для орграфа различают полустепень захода $d^-(v)$ и полустепень исхода $d^+(v)$.

Для неориентированного графа справедлива лемма о рукопожатиях:

$$\sum_{v \in V} d(v) = 2m.$$

Для орграфа

$$\sum_{v \in V} d^+(v) = \sum_{v \in V} d^-(v) = m.$$

15.2 Маршруты, цепи и циклы

Маршрутом называется чередующаяся последовательность вершин и рёбер, в которой каждое ребро соединяет соседние вершины последовательности. Если рёбра не повторяются, маршрут называется цепью; если не повторяются вершины, кроме возможного совпадения первой и последней, — простой цепью.

Замкнутая простая цепь называется циклом. Граф без циклов называется ациклическим. Две вершины связаны, если существует соединяющая их цепь. Связный неориентированный граф имеет одну компоненту связности.

Расстояние $d(u, v)$ — длина кратчайшей цепи между вершинами. Эксцентриситет, радиус и диаметр графа определяются формулами

$$e(v) = \max_{u \in V} d(u, v), \quad R(G) = \min_{v \in V} e(v), \quad D(G) = \max_{v \in V} e(v).$$

15.3 Матрица смежности

Для графа с вершинами $v_1, \dots, v_n$ матрица смежности $A = (a_{ij})$ задаётся как

$$a_{ij} = \begin{cases} 1, & \text{если существует ребро или дуга } (v_i, v_j), \\ 0, & \text{иначе.} \end{cases}$$

Для простого неориентированного графа A симметрична, а диагональные элементы равны нулю. Для взвешенного графа вместо единицы обычно хранят вес ребра.

Память: $O(n^2)$. Проверка наличия ребра выполняется за $O(1)$, но перебор соседей вершины требует $O(n)$. Представление выгодно для плотных графов. Степени матрицы имеют комбинаторный смысл: элемент $(A^k)_{ij}$ равен числу маршрутов длины k из v_i в v_j .

15.4 Матрица инцидентности

Матрица инцидентности B имеет размер $n \times m$. Для неориентированного графа

$$b_{ij} = \begin{cases} 1, & \text{если вершина } v_i \text{ инцидентна ребру } e_j, \\ 0, & \text{иначе.} \end{cases}$$

Каждый столбец простого графа содержит две единицы. Для орграфа часто используют знаковую матрицу: -1 в начале дуги и $+1$ в конце. Память в плотном виде составляет $O(nm)$, поэтому на практике матрица инцидентности чаще хранится разреженно.

15.5 Списки смежности

Для каждой вершины хранится список её соседей. Память составляет

$$O(n + m)$$

для орграфа и $O(n + 2m) = O(n + m)$ для неориентированного графа. Перебор всех соседей вершины выполняется за $O(d(v))$, а полный обход графа — за $O(n + m)$. Проверка конкретного ребра в обычном списке занимает $O(d(v))$; при использовании хеш-таблиц ожидаемое время можно уменьшить до $O(1)$.

Списки смежности являются стандартным представлением разреженных графов и используются в BFS, DFS, алгоритмах Дейкстры, Краскала, Прима и других.

15.6 Список рёбер

Граф представляется массивом пар или троек

$$(u_i, v_i) \quad \text{или} \quad (u_i, v_i, w_i), \quad i = 1, \dots, m.$$

Память — $O(m)$. Представление удобно, когда алгоритм обрабатывает рёбра последовательно, например в алгоритме Краскала или Беллмана–Форда. Поиск всех соседей одной вершины без дополнительного индекса требует просмотра всего массива.

15.7 Выбор представления

Представление	Память	Проверка ребра	Перебор соседей
Матрица смежности	$O(n^2)$	$O(1)$	$O(n)$
Списки смежности	$O(n + m)$	$O(d(v))$	$O(d(v))$
Список рёбер	$O(m)$	$O(m)$	$O(m)$

Для плотного графа, когда m порядка n^2 , удобна матрица смежности. Для разреженного графа предпочтительны списки смежности. В реальных программах часто применяют комбинированные структуры: массив вершин, массив рёбер и индексы начала списков смежности.

15.8 Изоморфизм графов

Графы $G_1 = (V_1, E_1)$ и $G_2 = (V_2, E_2)$ изоморфны, если существует биекция $\varphi: V_1 \rightarrow V_2$, сохраняющая смежность:

$$\{u, v\} \in E_1 \iff \{\varphi(u), \varphi(v)\} \in E_2.$$

Изоморфные графы имеют одинаковые инварианты: числа вершин и рёбер, набор степеней, число компонент, циклов и так далее. Обратное в общем случае неверно.

15.9 Что важно сказать на экзамене

Следует дать определения графа, степени, пути и цикла, затем сравнить четыре основных представления. Главный практический выбор: матрица смежности для плотных графов и быстрых запросов о наличии ребра; списки смежности — для разреженных графов и линейных обходов.

16 Деревья и их основные свойства

16.1 Определение дерева

Неориентированный граф называется лесом, если он не содержит циклов. Связный лес называется деревом. Таким образом, дерево — связный ациклический граф. Компоненты связности леса являются деревьями.

Для графа $G = (V, E)$ с $n = |V|$ вершинами следующие утверждения эквивалентны:

1. G является деревом;
2. G связен и имеет $n - 1$ ребро;
3. G ациклический и имеет $n - 1$ ребро;
4. между любыми двумя вершинами существует единственная простая цепь;
5. G связен, но удаление любого ребра нарушает связность;
6. G ациклический, но добавление любого нового ребра создаёт ровно один цикл.

Из этих свойств следует, что каждое ребро дерева является мостом.

16.2 Листья и степени вершин

Вершина степени один называется листом, или висячей вершиной. Всякое дерево с $n \geq 2$ содержит не менее двух листьев. Это можно доказать, рассмотрев концы максимальной простой цепи: у них не может быть дополнительных соседей, иначе цепь можно было бы продолжить или возник бы цикл.

По лемме о рукопожатиях для дерева

$$\sum_{v \in V} d(v) = 2(n - 1).$$

Если n_k — число вершин степени k , то

$$\sum_{k \geq 1} n_k = n, \quad \sum_{k \geq 1} k n_k = 2(n - 1).$$

Отсюда, например,

$$n_1 = 2 + \sum_{k \geq 3} (k - 2) n_k.$$

16.3 Корневые деревья

Если в дереве выделена вершина r , она называется корнем, а дерево — корневым. Для каждой вершины $v \neq r$ единственная цепь из r в v определяет родителя $p(v)$. Соседи, удалённые от корня на один уровень дальше, называются детьми.

Уровень вершины — расстояние от корня. Максимальный уровень называется высотой дерева. Поддерево вершины v состоит из v и всех её потомков.

Ориентированное корневое дерево, или арборесценция, можно получить, направив все рёбра от корня. Тогда

$$d^-(r) = 0, \quad d^-(v) = 1 \quad (v \neq r),$$

и каждая вершина достижима из корня.

16.4 Упорядоченные и бинарные деревья

Корневое дерево называется упорядоченным, если для детей каждой вершины задан порядок. Бинарное дерево — корневое дерево, в котором у каждой вершины не более двух детей, различаемых как левый и правый.

Для полного бинарного дерева, где каждая внутренняя вершина имеет ровно двух детей, число листьев L и число внутренних вершин I связаны равенством

$$L = I + 1.$$

Если высота бинарного дерева равна h и корень имеет уровень 0, то число вершин не превосходит

$$1 + 2 + \dots + 2^h = 2^{h+1} - 1.$$

16.5 Центр дерева

Эксцентриситет вершины v определяется как

$$e(v) = \max_{u \in V} d(u, v).$$

Вершины с минимальным эксцентриситетом образуют центр дерева. Центр любого дерева состоит либо из одной вершины, либо из двух смежных вершин. Его можно найти, последовательно удаляя все листья: после нескольких слоёв останется центр.

Диаметр дерева — длина максимальной простой цепи. Практический алгоритм его нахождения:

1. выполнить BFS или DFS из произвольной вершины и найти наиболее далёкую вершину u ;
2. выполнить обход из u ; наиболее далёкая вершина v является другим концом диаметра, а $d(u, v)$ — длина диаметра.

16.6 Остовные деревья

Остовным деревом связного графа G называется подграф, содержащий все вершины G и являющийся деревом. Любой связный граф имеет остовное дерево: достаточно выполнить DFS или BFS и взять рёбра, по которым впервые достигаются вершины.

Если рёбра имеют веса, задача минимального остовного дерева решается алгоритмами Краскала или Прима. Для полного помеченного графа на n вершинах число различных остовных деревьев равно n^{n-2} — формула Кэли.

16.7 Код Прюфера

Помеченному дереву на вершинах $\{1, \dots, n\}$ соответствует последовательность Прюфера длины $n-2$. На каждом шаге удаляют лист с наименьшей меткой и записывают метку его соседа. Код задаёт дерево однозначно и даёт простое доказательство формулы Кэли, поскольку существует ровно n^{n-2} последовательностей длины $n-2$ над алфавитом из n меток.

16.8 Что важно сказать на экзамене

Главная характеристика дерева — эквивалентность связности с отсутствием циклов при $n-1$ рёбрах. Нужно уметь объяснить единственность пути, наличие листьев, понятия корня, уровня и поддеревья, а также связь деревьев с остовами графа.

17 Комбинаторика

17.1 Основные правила подсчёта

Правило суммы. Если объект можно выбрать одним из k взаимно исключающих способов, причём число вариантов равно $N_1, \dots, N_k$, то общее число вариантов равно

$$N_1 + \dots + N_k.$$

Правило произведения. Если построение объекта состоит из k последовательных независимых выборов с числами вариантов $N_1, \dots, N_k$, то число объектов равно

$$N_1 N_2 \dots N_k.$$

Принцип Дирихле. Если N объектов размещены по m ящикам, то хотя бы в одном ящике находится не менее $\lceil N/m \rceil$ объектов.

17.2 Перестановки, размещения и сочетания

Число перестановок n различных элементов:

$$P_n = n!.$$

Если среди n элементов имеются группы одинаковых элементов размеров $n_1, \dots, n_s$, где $n_1 + \dots + n_s = n$, то число различных перестановок равно

$$\frac{n!}{n_1! \dots n_s!}.$$

Число размещений без повторений из n элементов по k :

$$A_n^k = \frac{n!}{(n-k)!}.$$

Число размещений с повторениями:

$$\overline{A}_n^k = n^k.$$

Число сочетаний без повторений:

$$C_n^k = \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Число сочетаний с повторениями:

$$\overline{C}_n^k = \binom{n+k-1}{k}.$$

Последняя формула следует из метода “звёзд и перегородок”.

17.3 Биномиальные и мультиномиальные коэффициенты

Формула бинома Ньютона:

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}.$$

Основные тождества:

$$\binom{n}{k} = \binom{n}{n-k}, \quad \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k},$$

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0 \quad (n \geq 1).$$

Мультиномиальная формула:

$$(x_1 + \dots + x_s)^n = \sum_{n_1 + \dots + n_s = n} \frac{n!}{n_1! \dots n_s!} x_1^{n_1} \dots x_s^{n_s}.$$

17.4 Формула включений и исключений

Для конечных множеств $A_1, \dots, A_m$

$$\left| \bigcup_{i=1}^m A_i \right| = \sum_i |A_i| - \sum_{i < j} |A_i \cap A_j| + \sum_{i < j < k} |A_i \cap A_j \cap A_k| - \dots + (-1)^{m+1} |A_1 \cap \dots \cap A_m|.$$

Формула применяется при подсчёте объектов, нарушающих хотя бы одно условие, например при нахождении числа беспорядков:

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

17.5 Разбиения множеств

Число разбиений n -элементного множества на k непустых неупорядоченных блоков называется числом Стирлинга второго рода и обозначается $S(n, k)$. Оно удовлетворяет рекуррентному соотношению

$$S(n, k) = S(n-1, k-1) + kS(n-1, k).$$

Первое слагаемое соответствует случаю, когда новый элемент образует отдельный блок, второе — включению его в один из k существующих блоков.

Число всех разбиений множества называется числом Белла:

$$B_n = \sum_{k=0}^n S(n, k).$$

Числа Стирлинга первого рода подсчитывают перестановки по числу циклов. В зависимости от соглашения используют знаковую и беззнаковую версии.

17.6 Производящие функции

Для последовательности $a_0, a_1, \dots$ обычной производящей функцией называется формальный ряд

$$A(z) = \sum_{n=0}^{\infty} a_n z^n.$$

Производящие функции переводят рекуррентные соотношения в алгебраические. Например, для чисел Фибоначчи $F_0 = 0, F_1 = 1, F_{n+2} = F_{n+1} + F_n$:

$$F(z) = \sum_{n \geq 0} F_n z^n = \frac{z}{1 - z - z^2}.$$

Экспоненциальная производящая функция имеет вид

$$\hat{A}(z) = \sum_{n=0}^{\infty} a_n \frac{z^n}{n!}$$

и особенно удобна для подсчёта помеченных структур.

17.7 Рекуррентные соотношения

Линейное однородное рекуррентное соотношение с постоянными коэффициентами

$$a_n = c_1 a_{n-1} + \dots + c_p a_{n-p}$$

решают с помощью характеристического многочлена

$$\lambda^p - c_1 \lambda^{p-1} - \dots - c_p = 0.$$

Для простых корней λ_i решение имеет вид

$$a_n = \sum_i C_i \lambda_i^n.$$

Кратному корню кратности r соответствуют слагаемые $n^j \lambda^n, j = 0, \dots, r-1$.

17.8 Что важно сказать на экзамене

Нужно различать упорядоченный и неупорядоченный выбор, наличие повторений и ограничений. Базовый набор инструментов: правила суммы и произведения, перестановки, размещения, сочетания, включения–исключения, числа Стирлинга и производящие функции.

18 Множества и отношения

18.1 Множества и операции над ними

Множество — совокупность различных объектов. Запись $x \in A$ означает, что x является элементом A . Множество A является подмножеством B , если

$$A \subseteq B \iff \forall x (x \in A \Rightarrow x \in B).$$

Основные операции:

$$A \cup B = \{x : x \in A \text{ или } x \in B\},$$

$$A \cap B = \{x : x \in A \text{ и } x \in B\},$$

$$A \setminus B = \{x : x \in A, x \notin B\},$$

$$A \Delta B = (A \setminus B) \cup (B \setminus A).$$

Дополнение $\bar{A}$ рассматривается относительно фиксированного универсального множества U .

Законы де Моргана:

$$\overline{A \cup B} = \bar{A} \cap \bar{B}, \quad \overline{A \cap B} = \bar{A} \cup \bar{B}.$$

Множество всех подмножеств A обозначается 2^A или $\mathcal{P}(A)$. Для конечного множества $|A| = n$ справедливо

$$|\mathcal{P}(A)| = 2^n.$$

18.2 Мощность и декартово произведение

Множества A и B равномощны, если существует биекция $A \rightarrow B$. Множество счётно, если оно конечно или равномощно $\mathbb{N}$. Множества $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ счётны, а $\mathbb{R}$ несчётно.

Декартово произведение определяется как

$$A \times B = \{(a, b) : a \in A, b \in B\}.$$

Для конечных множеств

$$|A \times B| = |A| |B|.$$

18.3 Бинарные отношения

Бинарным отношением между A и B называется подмножество

$$R \subseteq A \times B.$$

Если $(a, b) \in R$, пишут aRb . Область определения и множество значений:

$$\text{Dom } R = \{a \in A : \exists b \in B \ aRb\}, \quad \text{Im } R = \{b \in B : \exists a \in A \ aRb\}.$$

Обратное отношение:

$$R^{-1} = \{(b, a) : (a, b) \in R\}.$$

Композиция отношений $R \subseteq A \times B$ и $S \subseteq B \times C$:

$$S \circ R = \{(a, c) : \exists b \in B (aRb \wedge bSc)\}.$$

Композиция ассоциативна, но в общем случае некоммутативна.

18.4 Свойства отношений на одном множестве

Пусть $R \subseteq A \times A$. Отношение называется:

- рефлексивным, если aRa для всех $a \in A$;
- антирефлексивным, если aRa не выполняется ни для одного a ;
- симметричным, если $aRb \Rightarrow bRa$;

- антисимметричным, если aRb и bRa влекут $a = b$;
- транзитивным, если aRb и bRc влекут aRc ;
- связным, если для любых различных a, b выполнено aRb или bRa .

Транзитивное замыкание R^+ — наименьшее транзитивное отношение, содержащее R :

$$R^+ = R \cup R^2 \cup R^3 \cup \dots$$

Рефлексивно-транзитивное замыкание обозначают R^* .

18.5 Отношение эквивалентности

Отношение эквивалентности рефлексивно, симметрично и транзитивно. Класс эквивалентности элемента a :

$$[a] = \{x \in A : xRa\}.$$

Классы эквивалентности либо совпадают, либо не пересекаются и образуют разбиение множества A . Обратно, любое разбиение множества порождает отношение эквивалентности: два элемента эквивалентны, если принадлежат одному блоку.

Множество всех классов обозначается A/R и называется фактор-множеством.

18.6 Отношения порядка

Рефлексивное, антисимметричное и транзитивное отношение называется частичным порядком. Пара $(A, \preceq)$ — частично упорядоченное множество. Если любые два элемента сравнимы, порядок линейный.

Следует различать:

- минимальный элемент — элемент, ниже которого нет другого;
- наименьший элемент — элемент, не превосходящий всех остальных;
- максимальный и наибольший элементы;
- нижнюю и верхнюю границы подмножества;
- инфимум и супремум — наибольшую нижнюю и наименьшую верхнюю границы.

Наименьший элемент, если существует, единственен; минимальных элементов может быть несколько.

Конечный частичный порядок удобно изображать диаграммой Хассе, в которой транзитивные рёбра и петли не показываются.

18.7 Функции как специальные отношения

Функция $f: A \rightarrow B$ — отношение, в котором каждому $a \in A$ соответствует ровно один $b \in B$. Функция называется инъективной, если разные аргументы имеют разные значения; сюръективной, если каждый элемент B является значением; биективной, если выполняются оба свойства.

Обратная функция существует тогда и только тогда, когда исходная функция биективна.

18.8 Что важно сказать на экзамене

Нужно показать связь трёх конструкций: операции над множествами, отношения как подмножества декартова произведения и функции как частный случай отношений. Особенно важны эквивалентность и разбиения, а также различие частичного и линейного порядка.

19 Потоки в сетях. Теорема Форда и Фалкерсона. Алгоритмы нахождения максимального потока

19.1 Транспортная сеть и допустимый поток

Транспортной сетью называется ориентированный граф

$$G = (V, E)$$

с выделенными вершинами s — источником и t — стоком, а также функцией пропускных способностей

$$c: E \rightarrow \mathbb{R}_{\geq 0}.$$

Поток — функция $f: E \rightarrow \mathbb{R}$, удовлетворяющая условиям:

1. ограничение пропускной способности:

$$0 \leq f(u, v) \leq c(u, v);$$

2. сохранение потока во внутренних вершинах:

$$\sum_{(u,v) \in E} f(u, v) = \sum_{(v,w) \in E} f(v, w), \quad v \in V \setminus \{s, t\}.$$

Величина потока определяется как чистый выход из источника:

$$|f| = \sum_{(s,v) \in E} f(s, v) - \sum_{(v,s) \in E} f(v, s).$$

По закону сохранения она равна чистому входу в сток.

19.2 Остаточная сеть

Для текущего потока f остаточная пропускная способность прямой дуги равна

$$c_f(u, v) = c(u, v) - f(u, v),$$

а обратной дуги —

$$c_f(v, u) = f(u, v).$$

Остаточная сеть G_f содержит все дуги с положительной остаточной пропускной способностью.

Путь из s в t в остаточной сети называется увеличивающим. Его остаточная пропускная способность

$$\Delta = \min_{(u,v) \in P} c_f(u, v).$$

Поток увеличивают на Δ вдоль прямых дуг пути и уменьшают на Δ вдоль обратных. Обратные дуги принципиальны: они позволяют отменять ранее принятые локально неудачные решения.

19.3 Разрез сети

Разрезом называется разбиение вершин

$$V = S \cup T, \quad S \cap T = \emptyset, \quad s \in S, \quad t \in T.$$

Его пропускная способность равна

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

Для любого допустимого потока и любого разреза

$$|f| \leq c(S, T).$$

Следовательно, пропускная способность любого разреза является верхней оценкой максимального потока.

19.4 Теорема Форда–Фалкерсона

Теорема. Максимальная величина потока из s в t равна минимальной пропускной способности разреза:

$$\max_f |f| = \min_{(S,T)} c(S,T).$$

Схема доказательства. Если в остаточной сети нет пути из s в t , обозначим через S множество вершин, достижимых из s , и положим $T = V \setminus S$. Тогда все прямые дуги из S в T насыщены, а поток по всем дугам из T в S равен нулю. Поэтому

$$|f| = c(S,T).$$

С одной стороны, поток не превосходит ёмкости любого разреза; с другой стороны, для найденного разреза достигается равенство. Следовательно, поток максимален, а разрез минимален.

19.5 Алгоритм Форда–Фалкерсона

1. Инициализировать нулевой поток $f = 0$.
2. Пока в остаточной сети существует увеличивающий путь P :
 - (a) найти $\Delta = \min_{e \in P} c_f(e)$;
 - (b) увеличить поток вдоль P на Δ .
3. Если увеличивающего пути нет, вернуть поток f .

При целых пропускных способностях каждое увеличение повышает величину потока не менее чем на единицу, поэтому алгоритм завершается. Оценка времени при произвольном поиске пути:

$$O(mF),$$

где F — величина максимального целочисленного потока. Эта оценка псевдополиномиальна. Для иррациональных ёмкостей неудачный выбор путей может даже привести к незавершению.

19.6 Алгоритм Эдмондса–Карпа

В алгоритме Эдмондса–Карпа увеличивающий путь ищется с помощью BFS. Поэтому выбирается путь с минимальным числом дуг. Его сложность

$$O(nm^2).$$

Эта оценка полиномиальна и не зависит от величин пропускных способностей.

19.7 Алгоритм Диница

Алгоритм Диница строит слоистую сеть BFS и затем находит в ней блокирующий поток с помощью DFS. После блокирующего потока расстояние от s до t в остаточной сети увеличивается. Для общего графа сложность составляет

$$O(n^2m).$$

На практике алгоритм Диница часто значительно быстрее Эдмондса–Карпа.

19.8 Целочисленность и применения

Если все пропускные способности целые, существует максимальный поток с целыми значениями на дугах. Это позволяет сводить к потоку задачи о паросочетаниях, назначениях, распределении ресурсов и непересекающихся путях.

Для двудольного графа максимальное паросочетание строится сетью с единичными пропускными способностями: источник соединяется с левой долей, рёбра графа идут слева направо, правая доля соединяется со стоком.

19.9 Что важно сказать на экзамене

Ключевые понятия: допустимый поток, остаточная сеть, увеличивающий путь и разрез. Теорема максимального потока и минимального разреза одновременно даёт критерий оптимальности и обоснование алгоритма Форда–Фалкерсона.

20 Представление деревьев в ЭВМ

20.1 Общие подходы

Дерево можно хранить теми же способами, что и произвольный граф: матрицей или списками смежности. Однако наличие корня, единственного родителя и иерархической структуры позволяет использовать более компактные представления.

Выбор структуры зависит от требуемых операций:

- переход к родителю;
- перебор детей;
- вставка и удаление поддерева;
- поиск по ключу;
- последовательное хранение и сериализация.

20.2 Массив родителей

Для корневого дерева с вершинами $1, \dots, n$ хранится массив

$$p[1 \dots n],$$

где $p[v]$ — номер родителя вершины v , а для корня используется специальное значение, например 0 или -1 .

Память составляет $O(n)$. Переход к родителю выполняется за $O(1)$, но поиск всех детей вершины без дополнительного индекса требует $O(n)$. Поэтому массив родителей удобен, когда движение происходит преимущественно снизу вверх.

Глубину вершины можно находить последовательным переходом по родителям. Для многократных запросов применяют двоичный подъём: дополнительно хранят $up[v][k]$ — предка вершины v на расстоянии 2^k . Тогда предок на заданной высоте и наименьший общий предок вычисляются за $O(\log n)$.

20.3 Списки детей

Для каждой вершины хранится список её детей. Общий объём памяти равен $O(n)$, поскольку в дереве $n - 1$ ребро. Перебор детей вершины занимает время, пропорциональное их числу.

На практике используют:

- динамические списки или векторы детей;
- один массив рёбер и массив начал диапазонов, если дерево неизменно;
- связанное представление “первый ребёнок — следующий брат”.

В последнем случае каждый узел содержит два указателя:

$$firstChild(v), \quad nextSibling(v).$$

Так любое дерево произвольной степени представляется структурой, похожей на бинарное дерево, с постоянным числом ссылок на узел.

20.4 Связное представление бинарного дерева

Узел бинарного дерева обычно содержит данные и указатели на левое и правое поддерева:

```
struct Node {
    Data value;
    Node* left;
    Node* right;
    Node* parent; // необязательно
};
```

Память — $O(n)$. Переход к ребёнку выполняется за $O(1)$. Наличие указателя на родителя упрощает подъём вверх, но увеличивает память. В языках с автоматическим управлением памятью вместо указателей используются ссылки.

20.5 Последовательное представление бинарного дерева

Полное или почти полное бинарное дерево удобно хранить в массиве. При нумерации с единицы для вершины i :

$$\text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1, \quad \text{parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor.$$

При нумерации с нуля используются индексы $2i + 1$, $2i + 2$ и $\lfloor (i - 1)/2 \rfloor$.

Так хранят бинарные кучи. Представление не требует указателей и обладает хорошей локальностью памяти. Для сильно разреженного дерева оно неэффективно, поскольку возникает много пустых позиций.

20.6 Обходы как способ кодирования

Для бинарного дерева определены три основных обхода:

- прямой: корень, левое поддерево, правое поддерево;
- симметричный: левое поддерево, корень, правое поддерево;
- обратный: левое поддерево, правое поддерево, корень.

Один обход в общем случае не определяет дерево однозначно. Для восстановления бинарного дерева с различными ключами достаточно пары обходов “прямой + симметричный” или “обратный + симметричный”. При сериализации можно хранить прямой обход вместе с маркерами пустых поддеревьев; тогда представление однозначно.

Для упорядоченного дерева используется скобочная запись. Например, вершина и её дети записываются рекурсивно как

$$v(T_1, T_2, \dots, T_k).$$

20.7 Код Прюфера

Свободное помеченное дерево на n вершинах кодируется последовательностью длины $n - 2$. Алгоритм построения:

1. выбрать лист с наименьшей меткой;
2. записать метку его единственного соседа;
3. удалить лист и повторить действия до двух оставшихся вершин.

Код Прюфера занимает $O(n)$ памяти и однозначно определяет дерево. Число вхождений метки v в код равно $d(v) - 1$. Построение и декодирование можно выполнить за $O(n \log n)$ с очередью с приоритетом или за $O(n)$ при специальной организации данных.

20.8 Польская запись деревьев выражений

Дерево арифметического выражения можно кодировать префиксной или постфиксной записью. Например, выражению

$$(a + b)c$$

соответствуют:

$$\text{префиксная запись } * + a b c, \quad \text{постфиксная запись } a b + c *.$$

Постфиксная запись вычисляется стеком без скобок и удобна для компиляторов.

20.9 Что важно сказать на экзамене

Для корневого дерева основные структуры — массив родителей и списки детей; для бинарного — узлы с двумя ссылками или массивное хранение. Нужно сравнить память и стоимость переходов, а также упомянуть сериализацию обходами и код Прюфера.

21 Связность графов. Алгоритмы обхода графов

21.1 Компоненты связности

Две вершины неориентированного графа связаны, если существует соединяющая их цепь. Это отношение эквивалентности, поэтому множество вершин разбивается на компоненты связности. Граф связан, если компонента одна.

Ребро называется мостом, если его удаление увеличивает число компонент связности. Вершина называется точкой сочленения, если её удаление вместе с инцидентными рёбрами увеличивает число компонент.

Вершинная связность $\kappa(G)$ — минимальное число вершин, удаление которых делает граф несвязным или тривиальным. Рёберная связность $\lambda(G)$ — минимальное число рёбер, удаление которых нарушает связность. Для нетривиального графа

$$\kappa(G) \leq \lambda(G) \leq \delta(G),$$

где $\delta(G)$ — минимальная степень вершины.

21.2 Теорема Менгера

Пути между u и v называются внутренне вершинно непересекающимися, если у них нет общих внутренних вершин. Они рёберно непересекаются, если не имеют общих рёбер.

Теорема Менгера, вершинная форма. Для двух несмежных вершин u и v минимальное число вершин, удаление которых разделяет u и v , равно максимальному числу попарно внутренне вершинно непересекающихся путей между u и v .

Рёберная форма. Минимальное число рёбер в u - v -разрезе равно максимальному числу попарно рёберно непересекающихся u - v -путей.

Теорема связывает структурную устойчивость графа с числом независимых маршрутов и может быть выведена из теоремы о максимальном потоке и минимальном разрезе.

21.3 Поиск в ширину

BFS обходит граф слоями по расстоянию от начальной вершины. Используется очередь.

1. Отметить стартовую вершину s и поместить её в очередь.
2. Пока очередь не пуста, извлечь вершину v .
3. Для каждого неотмеченного соседа u отметить его, записать $parent[u] = v$, положить $dist[u] = dist[v] + 1$ и добавить u в очередь.

При представлении списками смежности сложность

$$O(n + m),$$

память — $O(n)$. BFS находит кратчайшие пути по числу рёбер в невзвешенном графе и строит дерево поиска.

21.4 Поиск в глубину

DFS проходит по одному пути настолько глубоко, насколько возможно, после чего возвращается назад. Реализуется рекурсией или явным стеком.

1. Отметить вершину v .
2. Для каждого неотмеченного соседа u вызвать $DFS(u)$.

Сложность при списках смежности также равна $O(n + m)$. DFS используется для поиска компонент, топологической сортировки, циклов, мостов, точек сочленения и компонент сильной связности.

Для несвязного графа внешний цикл запускает обход из каждой ещё не посещённой вершины. Полученный лес поиска содержит по одному дереву на компоненту.

21.5 Мосты и точки сочленения

В DFS вводят время входа $tin[v]$ и значение

$$low[v] = \min\{tin[v], tin[p] \text{ по обратным рёбрам}, low[u] \text{ по рёбрам дерева}\}.$$

Для ребра дерева (v, u) :

$$(v, u) \text{ — мост} \iff low[u] > tin[v].$$

Вершина v , не являющаяся корнем DFS, является точкой сочленения, если существует ребёнок u такой, что

$$low[u] \geq tin[v].$$

Корень является точкой сочленения, если у него более одного ребёнка в дереве DFS. Алгоритм работает за $O(n + m)$.

21.6 Связность ориентированных графов

Орграф слабо связан, если после игнорирования направлений получается связный граф. Он сильно связан, если для любых u, v существуют ориентированные пути $u \rightsquigarrow v$ и $v \rightsquigarrow u$.

Компонента сильной связности — максимальное по включению множество попарно взаимно достижимых вершин. После сжатия каждой компоненты в одну вершину получается ациклический орграф — конденсация.

Основные алгоритмы:

- алгоритм Косарайю: два DFS, второй — по транспонированному графу в порядке убывания времени выхода; $O(n + m)$;
- алгоритм Тарьяна: один DFS, стек и значения low ; $O(n + m)$.

21.7 Топологическая сортировка

Топологический порядок орграфа — линейный порядок вершин, в котором каждая дуга ведёт слева направо. Он существует тогда и только тогда, когда орграф ацикличесен.

Его можно получить:

- DFS: записывать вершины при выходе и затем обратить список;
- алгоритмом Кана: многократно удалять вершины с нулевой полустепенью захода.

Оба алгоритма имеют сложность $O(n + m)$.

21.8 Что важно сказать на экзамене

BFS и DFS — базовые линейные обходы. BFS даёт кратчайшие пути в невзвешенном графе, DFS раскрывает структурные свойства: циклы, мосты, точки сочленения, сильные компоненты и топологический порядок. Теорема Менгера связывает связность с числом независимых путей.

22 Аппроксимация функциональных зависимостей

22.1 Постановка задачи

Аппроксимацией называется замена сложной функции f более простой функцией φ из заранее выбранного класса V :

$$f(x) \approx \varphi(x), \quad \varphi \in V.$$

Исходная зависимость может быть задана аналитически, таблицей значений (x_i, y_i) или результатами эксперимента. Класс V выбирают с учётом требуемой точности, гладкости, вычислительной стоимости и физического смысла модели.

Следует различать:

- интерполяцию: $\varphi(x_i) = y_i$ во всех узлах;
- аппроксимацию в среднем: минимизируется интегральная или дискретная ошибка;
- равномерное приближение: минимизируется максимальная ошибка;
- регрессию: данные считаются зашумлёнными, точное прохождение через них не требуется.

22.2 Нормы и критерии качества

Качество приближения оценивается нормой ошибки $e = f - \varphi$. На отрезке $[a, b]$ часто используют

$$\|e\|_\infty = \max_{x \in [a, b]} |e(x)|, \quad \|e\|_2 = \left(\int_a^b |e(x)|^2 dx \right)^{1/2}.$$

Для табличных данных:

$$\|e\|_{2,h} = \left(\sum_{i=1}^N w_i |y_i - \varphi(x_i)|^2 \right)^{1/2},$$

где $w_i > 0$ — веса наблюдений.

Абсолютная ошибка измеряется в единицах величины, относительная — отношением к масштабу функции. Для данных с различной дисперсией разумно использовать веса, обратно пропорциональные дисперсии ошибок.

22.3 Линейная аппроксимационная модель

Пусть

$$\varphi(x; c) = \sum_{j=0}^m c_j \phi_j(x),$$

где базисные функции ϕ_j заданы, а коэффициенты c_j неизвестны. Такая модель линейна по параметрам, даже если функции ϕ_j нелинейны по x .

В дискретном случае вводят матрицу дизайна

$$A_{ij} = \phi_j(x_i), \quad Ac \approx y.$$

При квадратичном критерии задача сводится к методу наименьших квадратов:

$$\min_c \|Ac - y\|_2^2.$$

Геометрически решение является ортогональной проекцией вектора данных на столбцовое пространство A .

22.4 Основные классы аппроксимирующих функций

1. Алгебраические многочлены

$$\varphi(x) = c_0 + c_1 x + \dots + c_m x^m.$$

Они просты, но степенной базис может быть плохо обусловлен при больших m .

2. Ортогональные разложения

$$\varphi_m(x) = \sum_{j=0}^m c_j \phi_j(x),$$

где ϕ_j ортогональны в выбранном скалярном произведении. Примеры: многочлены Лежандра и Чебышева, тригонометрическая система.

3. **Сплайны** — кусочно-полиномиальные функции с заданной гладкостью. Они устойчивее глобальных многочленов и локально реагируют на изменение данных.

4. Рациональные функции

$$\varphi(x) = \frac{P_m(x)}{Q_n(x)}.$$

Они хорошо описывают функции с полюсами и насыщением, но задача подбора коэффициентов обычно нелинейна.

5. Экспоненциальные и степенные модели:

$$y = ae^{bx}, \quad y = ax^b.$$

Иногда их линеаризуют логарифмированием, однако это изменяет структуру ошибки.

22.5 Теорема Вейерштрасса

Если f непрерывна на $[a, b]$, то для любого $\varepsilon > 0$ существует многочлен P такой, что

$$\max_{x \in [a, b]} |f(x) - P(x)| < \varepsilon.$$

Теорема гарантирует принципиальную возможность равномерного полиномиального приближения, но не утверждает, что многочлен высокой степени численно удобен. На практике глобальная аппроксимация высокой степени может быть неустойчивой; часто лучше применять сплайны или ортогональные базисы.

22.6 Равномерное наилучшее приближение

Задача Чебышёва имеет вид

$$\min_{\varphi \in V} \|f - \varphi\|_{\infty}.$$

Для многочленов степени не выше m наилучшее равномерное приближение характеризуется чередованием ошибки: при стандартных условиях ошибка достигает максимального по модулю значения с чередующимися знаками не менее чем в $m + 2$ точках. На этом основан алгоритм Ремеза.

22.7 Выбор сложности модели

Увеличение числа параметров уменьшает ошибку на обучающих данных, но может привести к переобучению и сильной чувствительности к шуму. Поэтому учитывают:

- ошибку на независимой проверочной выборке;
- обусловленность матрицы задачи;
- физическую интерпретируемость коэффициентов;
- устойчивость к изменению данных;
- регуляризацию, например

$$\min_c (\|Ac - y\|_2^2 + \lambda \|Lc\|_2^2).$$

22.8 Что важно сказать на экзамене

Аппроксимация отличается от интерполяции отсутствием требования точного совпадения во всех узлах. Нужно назвать критерий ошибки, выбрать класс функций и объяснить получение коэффициентов. Хорошая модель должна быть не только точной, но и устойчивой, умеренно сложной и адекватной природе данных.

23 Вычислительные методы линейной алгебры

23.1 Задачи вычислительной линейной алгебры

Основные задачи:

- решение систем линейных алгебраических уравнений $Ax = b$;
- вычисление собственных значений и собственных векторов;
- задачи наименьших квадратов;
- матричные разложения и оценка обусловленности.

Численный метод должен учитывать округление, накопление ошибок, структуру и разреженность матрицы. Ошибка решения определяется не только алгоритмом, но и обусловленностью самой задачи.

23.2 Метод Гаусса

Прямой ход метода Гаусса последовательными элементарными преобразованиями строк приводит систему к верхнетреугольному виду

$$Ux = \tilde{b}.$$

Затем выполняется обратная подстановка:

$$x_i = \frac{1}{u_{ii}} \left(\tilde{b}_i - \sum_{j=i+1}^n u_{ij} x_j \right).$$

Число арифметических операций имеет порядок $O(n^3)$, память для плотной матрицы — $O(n^2)$. Без выбора главного элемента метод может быть численно неустойчив. Обычно применяют частичный выбор: на k -м шаге выбирают максимальный по модулю элемент в столбце среди строк $k, \dots, n$ и меняют строки местами.

23.3 LU -разложение

Метод Гаусса эквивалентен факторизации

$$PA = LU,$$

где P — матрица перестановки, L — нижнетреугольная матрица с единичной диагональю, U — верхнетреугольная. Решение выполняется в два этапа:

$$Ly = Pb, \quad Ux = y.$$

Факторизация стоит $O(n^3)$, а решение для каждого нового правого столбца — $O(n^2)$. Поэтому LU особенно выгодно, когда матрица одна, а правых частей несколько.

Определитель можно вычислить как

$$\det A = \det(P) \prod_{i=1}^n u_{ii}.$$

23.4 Разложение Холецкого

Если A симметрична и положительно определена, существует единственное разложение

$$A = LL^T$$

с положительной диагональю L . Оно примерно вдвое дешевле общего LU и не требует выбора главного элемента. Формулы:

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2},$$
$$l_{ji} = \frac{1}{l_{ii}} \left(a_{ji} - \sum_{k=1}^{i-1} l_{jk} l_{ik} \right), \quad j > i.$$

Разложение Холецкого широко применяется для нормальных уравнений, ковариационных матриц и конечно-элементных систем.

23.5 Метод прогонки

Для трёхдиагональной системы

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$$

метод прогонки, или алгоритм Томаса, выполняет исключение за $O(n)$ операций и использует $O(n)$ памяти. Представим

$$x_i = \alpha_i x_{i+1} + \beta_i.$$

Коэффициенты вычисляются прямым ходом, после чего выполняется обратная подстановка. Устойчивость гарантируется, например, при строгом диагональном преобладании.

23.6 QR -разложение

Матрица представляется как

$$A = QR,$$

где $Q^T Q = I$, а R верхнетреугольна. Разложение строят преобразованиями Хаусхолдера или вращениями Гивенса. Оно устойчивее нормальных уравнений в задаче наименьших квадратов:

$$\min_x \|Ax - b\|_2 \implies Rx = Q^T b.$$

Преобразования Гивенса удобны для разреженных матриц и последовательного обнуления отдельных элементов.

23.7 Стационарные итерационные методы

Пусть $A = D + L + U$, где D — диагональная часть, L и U — нижняя и верхняя строго треугольные части.

Метод Якоби:

$$x^{(k+1)} = -D^{-1}(L + U)x^{(k)} + D^{-1}b.$$

Метод Гаусса–Зейделя:

$$(D + L)x^{(k+1)} = b - Ux^{(k)}.$$

Метод верхней релаксации:

$$(D + \omega L)x^{(k+1)} = \omega b - (\omega U + (\omega - 1)D)x^{(k)}.$$

Для итерации $x^{(k+1)} = Bx^{(k)} + c$ необходимым и достаточным условием сходимости при любом начальном приближении является

$$\rho(B) < 1,$$

где $\rho(B)$ — спектральный радиус. Достаточное условие для Якоби и Зейделя — строгое диагональное преобладание; для симметричной положительно определённой матрицы метод Зейделя сходится.

23.8 Методы подпространств Крылова

Для больших разреженных систем используют пространства

$$\mathcal{K}_k(A, r_0) = \text{span}\{r_0, Ar_0, \dots, A^{k-1}r_0\}.$$

Метод сопряжённых градиентов применяется к симметричным положительно определённым матрицам. В точной арифметике он даёт решение не более чем за n шагов, а на практике скорость зависит от распределения собственных значений. Один шаг требует умножения матрицы на вектор.

Для несимметричных матриц применяют GMRES, BiCGStab и другие методы. Ключевой элемент — предобуславливание:

$$M^{-1}Ax = M^{-1}b,$$

где M приближает A , но легко обращается.

23.9 Собственные значения

Степенной метод:

$$y^{(k+1)} = Ax^{(k)}, \quad x^{(k+1)} = \frac{y^{(k+1)}}{\|y^{(k+1)}\|}$$

сходится к собственному вектору, соответствующему доминирующему по модулю собственному значению, если оно единственно и начальный вектор имеет ненулевую проекцию на его собственный вектор.

Обратная итерация

$$(A - \mu I)y^{(k+1)} = x^{(k)}$$

находит собственное значение, ближайшее к сдвигу μ . Отношение Рэлея

$$\rho(x) = \frac{x^T Ax}{x^T x}$$

даёт приближение собственного значения.

Для симметричных матриц применяют метод вращений Якоби или QR -алгоритм.

23.10 Обусловленность и невязка

Число обусловленности в согласованной норме:

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

При малом возмущении правой части

$$\frac{\|\delta x\|}{\|x\|} \lesssim \kappa(A) \frac{\|\delta b\|}{\|b\|}.$$

Малая невязка

$$r = b - A\tilde{x}$$

не всегда означает малую ошибку решения: при большой $\kappa(A)$ задача сама чувствительна.

23.11 Что важно сказать на экзамене

Следует разделить прямые и итерационные методы. Для плотных систем стандартны Гаусс и LU , для симметричных положительно определённых — Холецкий, для трёхдиагональных — прогонка, для больших разреженных — методы Крылова с предобуславливанием. Обязательно упомянуть устойчивость и обусловленность.

24 Метод конечных разностей

24.1 Основная идея

Метод конечных разностей заменяет дифференциальную задачу системой алгебраических уравнений на сетке. На равномерной одномерной сетке

$$x_i = a + ih, \quad i = 0, \dots, N, \quad h = \frac{b-a}{N}$$

производные аппроксимируют разностями, полученными из формулы Тейлора:

$$\begin{aligned} u'(x_i) &= \frac{u_{i+1} - u_i}{h} + O(h), \\ u'(x_i) &= \frac{u_{i+1} - u_{i-1}}{2h} + O(h^2), \\ u''(x_i) &= \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + O(h^2). \end{aligned}$$

Разностная схема должна обладать аппроксимацией, устойчивостью и сходимостью. Локальная погрешность аппроксимации получается подстановкой точного решения в схему. Устойчивость означает ограниченность роста возмущений. Для линейной корректно поставленной начальной задачи теорема Лакса утверждает: при наличии аппроксимации устойчивость эквивалентна сходимости.

24.2 Уравнение теплопроводности

Рассмотрим

$$u_t = a^2 u_{xx} + f(x, t), \quad 0 < x < L, \quad 0 < t \leq T.$$

Введём сетку $x_i = ih$, $t^n = n\tau$ и обозначим $u_i^n \approx u(x_i, t^n)$.

Явная схема:

$$\frac{u_i^{n+1} - u_i^n}{\tau} = a^2 \frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{h^2} + f_i^n.$$

Она имеет порядок $O(\tau + h^2)$ и переписывается как

$$u_i^{n+1} = r u_{i-1}^n + (1 - 2r) u_i^n + r u_{i+1}^n + \tau f_i^n, \quad r = \frac{a^2 \tau}{h^2}.$$

Условие устойчивости:

$$r \leq \frac{1}{2}.$$

Неявная схема:

$$\frac{u_i^{n+1} - u_i^n}{\tau} = a^2 \frac{u_{i-1}^{n+1} - 2u_i^{n+1} + u_{i+1}^{n+1}}{h^2} + f_i^{n+1}.$$

Она безусловно устойчива, но на каждом шаге требует решения трёхдиагональной системы. Порядок также $O(\tau + h^2)$.

Схема Кранка–Николсон усредняет пространственный оператор на слоях n и $n+1$:

$$\frac{u_i^{n+1} - u_i^n}{\tau} = \frac{a^2}{2h^2} [(u_{i-1}^n - 2u_i^n + u_{i+1}^n) + (u_{i-1}^{n+1} - 2u_i^{n+1} + u_{i+1}^{n+1})] + f_i^{n+1/2}.$$

Она имеет порядок $O(\tau^2 + h^2)$ и безусловно устойчива в энергетической норме.

24.3 Волновое уравнение

Для задачи

$$u_{tt} = a^2 u_{xx} + f$$

стандартная трёхслойная схема:

$$\frac{u_i^{n+1} - 2u_i^n + u_i^{n-1}}{\tau^2} = a^2 \frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{h^2} + f_i^n.$$

Порядок аппроксимации — $O(\tau^2 + h^2)$. Условие Куранта:

$$\frac{a\tau}{h} \leq 1.$$

Для запуска схемы, кроме u_i^0 , нужен первый временной слой u_i^1 , который строят по начальному условию для скорости и разложению Тейлора.

24.4 Эллиптическое уравнение

Для уравнения Пуассона

$$-\Delta u = f$$

на прямоугольной сетке с шагами h_x, h_y используется пятиточечный шаблон:

$$-\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} - \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} = f_{i,j}.$$

При $h_x = h_y = h$ схема имеет порядок $O(h^2)$. Получается большая разреженная система. Для её решения применяют методы Зейделя, релаксации, сопряжённых градиентов, многосеточные методы.

24.5 Граничные условия

Условие Дирихле задаётся непосредственным присваиванием узловых значений. Условие Неймана

$$u_x(a) = q$$

можно аппроксимировать односторонней разностью

$$\frac{u_1 - u_0}{h} = q + O(h)$$

или схемой второго порядка. Фиктивные узлы позволяют сохранить симметричный шаблон. Для условия Робена $\alpha u + \beta u_n = g$ комбинируют значения функции и разностную производную.

Важно, чтобы порядок аппроксимации граничных условий не снижал общий порядок схемы.

24.6 Исследование устойчивости

Для линейных задач с постоянными коэффициентами применяется метод фон Неймана. Погрешность представляют гармоникой

$$\varepsilon_i^n = G^n e^{ikx_i},$$

где G — коэффициент усиления. Условие устойчивости требует

$$|G| \leq 1$$

для всех допустимых волновых чисел.

Для нелинейных задач и переменных коэффициентов используют энергетические оценки, принцип максимума и исследование спектра матрицы перехода.

24.7 Преимущества и ограничения

Преимущества: простота построения на регулярных сетках, прозрачность шаблонов, эффективность для прямоугольных областей. Ограничения: трудности на сложной геометрии, при локальном сгущении сетки и при необходимости точного соблюдения законов сохранения. Для сложных областей часто предпочтительны методы конечных объёмов или конечных элементов.

24.8 Что важно сказать на экзамене

Нужно показать переход от производных к сеточным разностям, определить аппроксимацию, устойчивость и сходимость. В качестве примеров разумно привести явную и неявную схемы теплопроводности, условие Куранта для волнового уравнения и пятиточечный шаблон для уравнения Пуассона.

25 Метод конечных элементов. Лагранжевы элементы

25.1 Идея метода конечных элементов

Метод конечных элементов (МКЭ) применяется к краевым задачам для дифференциальных уравнений. Область разбивается на простые подобласти — конечные элементы, а искомая функция приближается кусочно-полиномиальной функцией. Коэффициенты определяются из слабой, или вариационной, постановки.

Рассмотрим модельную задачу Пуассона:

$$-\nabla \cdot (k \nabla u) = f \quad \text{в } \Omega, \quad u = 0 \quad \text{на } \Gamma_D,$$

с возможным условием Неймана

$$k \frac{\partial u}{\partial n} = g \quad \text{на } \Gamma_N.$$

Умножим уравнение на тестовую функцию v , проинтегрируем по области и применим формулу Грина:

$$\int_{\Omega} k \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx + \int_{\Gamma_N} g v \, ds.$$

Слабая постановка: найти $u \in V$, где

$$V = \{v \in H^1(\Omega) : v|_{\Gamma_D} = 0\},$$

такое, что

$$a(u, v) = \ell(v) \quad \forall v \in V.$$

25.2 Галёркинская аппроксимация

Выбирается конечномерное пространство $V_h \subset V$ с базисом $\{\varphi_1, \dots, \varphi_N\}$. Приближённое решение имеет вид

$$u_h = \sum_{j=1}^N U_j \varphi_j.$$

Требование

$$a(u_h, v_h) = \ell(v_h) \quad \forall v_h \in V_h$$

приводит к системе

$$KU = F,$$

где

$$K_{ij} = a(\varphi_j, \varphi_i), \quad F_i = \ell(\varphi_i).$$

Матрица K называется матрицей жёсткости.

25.3 Лагранжевы конечные элементы

Лагранжев элемент задаётся тройкой (K, P, Σ) :

- K — геометрический элемент;
- P — пространство полиномов;
- Σ — набор степеней свободы, обычно значений функции в узлах.

Базисные функции Лагранжа удовлетворяют условию

$$\varphi_i(x_j) = \delta_{ij}.$$

Поэтому коэффициент U_i равен узловому значению приближённого решения.

Для одномерного линейного элемента $K = [x_1, x_2]$:

$$\varphi_1(x) = \frac{x_2 - x}{h}, \quad \varphi_2(x) = \frac{x - x_1}{h}, \quad h = x_2 - x_1.$$

На элементе

$$u_h(x) = U_1 \varphi_1(x) + U_2 \varphi_2(x).$$

25.4 Линейный треугольный элемент

На треугольнике используются полиномы первой степени

$$P_1(K) = \text{span}\{1, x, y\}.$$

Степени свободы — значения в трёх вершинах. Через барицентрические координаты $\lambda_1, \lambda_2, \lambda_3$ базис записывается как

$$\varphi_i = \lambda_i, \quad \lambda_1 + \lambda_2 + \lambda_3 = 1.$$

Градиенты базисных функций постоянны внутри элемента, поэтому градиент линейного конечно-элементного решения кусочно постоянен.

Квадратичный треугольный элемент имеет шесть узлов: три вершины и три середины рёбер. Его базис:

$$\varphi_i = \lambda_i(2\lambda_i - 1), \quad i = 1, 2, 3,$$

$$\varphi_{ij} = 4\lambda_i\lambda_j.$$

25.5 Локальные матрицы и сборка

На каждом элементе K_e вычисляются локальные матрица и вектор:

$$K_{ij}^{(e)} = \int_{K_e} k \nabla \varphi_j^{(e)} \cdot \nabla \varphi_i^{(e)} dx,$$

$$F_i^{(e)} = \int_{K_e} f \varphi_i^{(e)} dx + \int_{\partial K_e \cap \Gamma_N} g \varphi_i^{(e)} ds.$$

Затем локальные вклады суммируются в общую разреженную систему по глобальным номерам степеней свободы. Этот процесс называется сборкой.

Для одномерного элемента при постоянном k локальная матрица жёсткости равна

$$K^{(e)} = \frac{k}{h} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

25.6 Отображение с эталонного элемента

Вычисления удобно выполнять на эталонном элементе $\hat{K}$. Геометрическое отображение

$$x = F_e(\hat{x})$$

переносит базисные функции и интегралы на физический элемент. Для изопараметрических элементов геометрия и решение аппроксимируются одними и теми же функциями формы.

При замене переменных учитываются якобиан и преобразование градиента:

$$\nabla_x \varphi = J^{-T} \nabla_{\hat{x}} \hat{\varphi}, \quad dx = |\det J| d\hat{x}.$$

25.7 Граничные условия

Условия Неймана входят в правую часть слабой постановки естественным образом. Условия Дирихле являются существенными: соответствующие степени свободы задаются явно, после чего система модифицируется исключением известных значений или другим эквивалентным способом.

25.8 Сходимость и оценка погрешности

При эллиптичности билинейной формы и корректной постановке лемма Сеа даёт

$$\|u - u_h\|_V \leq C \inf_{v_h \in V_h} \|u - v_h\|_V.$$

То есть ошибка МКЭ сравнима с наилучшей ошибкой аппроксимации выбранным пространством.

Для Лагранжевых элементов степени p на регулярной сетке при достаточной гладкости решения типичны оценки

$$\|u - u_h\|_{H^1(\Omega)} \leq Ch^p |u|_{H^{p+1}(\Omega)},$$

$$\|u - u_h\|_{L^2(\Omega)} \leq Ch^{p+1} |u|_{H^{p+1}(\Omega)}.$$

25.9 Что важно сказать на экзамене

Необходимо показать цепочку: сильная постановка — интегрирование по частям — слабая постановка — конечномерное пространство — система $KU = F$. Лагранжевы элементы используют значения функции в узлах и обеспечивают C^0 -непрерывность между элементами.

26 Метод конечных элементов. Эрмитовы элементы

26.1 Назначение эрмитовых элементов

Лагранжевы элементы обеспечивают непрерывность функции, но производные между элементами обычно разрывны. Для задач, слабая постановка которых требует пространства H^2 , нужна C^1 -согласованная аппроксимация. Типичный пример — уравнение изгиба балки Эйлера–Бернулли четвёртого порядка:

$$(EIu'')'' = q(x).$$

Эрмитовы элементы используют в качестве степеней свободы не только значения функции, но и значения её производных в узлах. Это позволяет обеспечить непрерывность первой производной.

26.2 Слабая постановка задачи изгиба балки

Для простоты рассмотрим

$$EIu^{(4)} = q(x), \quad x \in (0, L).$$

После двукратного интегрирования по частям получаем билинейную форму

$$a(u, v) = \int_0^L EIu''v'' dx.$$

Слабая постановка требует

$$u, v \in H^2(0, L) \quad \text{с учётом существенных граничных условий.}$$

Следовательно, конечно-элементное пространство должно быть глобально C^1 -непрерывным.

26.3 Кубический эрмитов элемент в одном измерении

На элементе $K = [x_1, x_2]$ длины h используются четыре степени свободы:

$$u(x_1), \quad u'(x_1), \quad u(x_2), \quad u'(x_2).$$

В локальной координате

$$\xi = \frac{x - x_1}{h} \in [0, 1]$$

приближение записывается как

$$u_h(x) = N_1(\xi)u_1 + N_2(\xi)\theta_1 + N_3(\xi)u_2 + N_4(\xi)\theta_2,$$

где $\theta_i = u'(x_i)$, а функции формы равны

$$N_1(\xi) = 1 - 3\xi^2 + 2\xi^3,$$

$$N_2(\xi) = h(\xi - 2\xi^2 + \xi^3),$$

$$N_3(\xi) = 3\xi^2 - 2\xi^3,$$

$$N_4(\xi) = h(-\xi^2 + \xi^3).$$

Они удовлетворяют интерполяционным условиям:

	N_1	N_2	N_3	N_4
$x = x_1$	1	0	0	0
$\frac{d}{dx} _{x_1}$	0	1	0	0
$x = x_2$	0	0	1	0
$\frac{d}{dx} _{x_2}$	0	0	0	1

26.4 Локальная матрица жёсткости балки

При постоянном EI локальная матрица для вектора степеней свободы

$$U_e = (u_1, \theta_1, u_2, \theta_2)^T$$

имеет вид

$$K^{(e)} = \frac{EI}{h^3} \begin{pmatrix} 12 & 6h & -12 & 6h \\ 6h & 4h^2 & -6h & 2h^2 \\ -12 & -6h & 12 & -6h \\ 6h & 2h^2 & -6h & 4h^2 \end{pmatrix}.$$

Она получается из формулы

$$K_{ij}^{(e)} = \int_{x_1}^{x_2} E I N_i'' N_j'' dx.$$

Локальный вектор нагрузки:

$$F_i^{(e)} = \int_{x_1}^{x_2} q(x) N_i(x) dx.$$

После сборки значения перемещения и угла поворота в общем узле совпадают для соседних элементов, поэтому глобальная функция принадлежит C^1 .

26.5 Существенные и естественные условия

Для балки геометрические условия

$$u = \bar{u}, \quad u' = \bar{\theta}$$

являются существенными и задаются на степенях свободы. Момент и поперечная сила, связанные с

$$M = EI u'', \quad Q = -(EI u'')',$$

возникают как естественные граничные условия после интегрирования по частям.

26.6 Эрмитова интерполяция и порядок точности

Кубический эрмитов интерполянт точно воспроизводит полиномы степени не выше трёх. Для достаточно гладкой функции локальная ошибка имеет порядок

$$\|u - I_h u\|_{L^\infty(K)} = O(h^4),$$

а ошибка производной — $O(h^3)$. В энергетической норме задачи балки при регулярной сетке получают типичную оценку порядка $O(h^2)$.

26.7 Многомерные C^1 -элементы

В двух измерениях построение C^1 -согласованных элементов значительно сложнее. Примеры:

- прямоугольный элемент Богнера–Фокса–Шмита;
- треугольный элемент Аргириса пятой степени;
- элемент Клафа–Точера с разбиением треугольника.

Они используются для пластин, оболочек и уравнений четвёртого порядка. На практике часто применяют смешанные постановки, позволяющие снизить порядок уравнения и использовать обычные C^0 -элементы.

26.8 Сравнение с лагранжевыми элементами

Лагранжевы элементы:

- степени свободы — значения функции;
- глобальная непрерывность — обычно C^0 ;
- подходят для уравнений второго порядка.

Эрмитовы элементы:

- степени свободы — значения функции и производных;
- обеспечивают C^1 -непрерывность;
- естественны для задач четвёртого порядка, но сложнее в многомерном случае.

26.9 Что важно сказать на экзамене

Главное отличие эрмитовых элементов — наличие производных среди степеней свободы. На примере кубического элемента балки нужно привести четыре функции формы, объяснить C^1 -согласованность и показать, почему она требуется слабой постановкой уравнения четвёртого порядка.

27 Метод наименьших квадратов

27.1 Линейная задача наименьших квадратов

Пусть дана переопределённая система

$$Ax \approx b, \quad A \in \mathbb{R}^{m \times n}, \quad m \geq n.$$

В общем случае точного решения нет. Метод наименьших квадратов выбирает

$$x_* = \arg \min_x \|Ax - b\|_2^2.$$

Вектор

$$r = b - Ax_*$$

называется невязкой.

Дифференцируя функционал

$$J(x) = (Ax - b)^T (Ax - b),$$

получаем нормальные уравнения

$$A^T A x = A^T b.$$

Если столбцы A линейно независимы, матрица $A^T A$ положительно определена и решение единственно:

$$x_* = (A^T A)^{-1} A^T b.$$

На практике обратную матрицу явно не вычисляют.

27.2 Геометрический смысл

Вектор Ax_* является ортогональной проекцией b на столбцовое пространство $\mathcal{R}(A)$. Поэтому

$$A^T r = 0.$$

То есть невязка ортогональна каждому столбцу матрицы A . Если A имеет неполный столбцовый ранг, решений может быть несколько; среди них часто выбирают решение минимальной нормы, задаваемое псевдообратной матрицей:

$$x_* = A^+ b.$$

27.3 Численно устойчивое решение

Нормальные уравнения ухудшают обусловленность:

$$\kappa_2(A^T A) = \kappa_2(A)^2.$$

Поэтому предпочтительны:

1. QR -разложение $A = QR$:

$$\min_x \|Ax - b\|_2 \implies Rx = Q^T b;$$

2. сингулярное разложение

$$A = U \Sigma V^T, \quad A^+ = V \Sigma^+ U^T.$$

SVD наиболее надёжно при вырожденности и позволяет определить численный ранг, но дороже QR .

27.4 Аппроксимация табличных данных

Пусть заданы точки (t_i, y_i) и модель

$$\varphi(t) = \sum_{j=0}^{n-1} c_j \phi_j(t).$$

Минимизируется

$$\sum_{i=1}^m \left(y_i - \sum_{j=0}^{n-1} c_j \phi_j(t_i) \right)^2.$$

Матрица задачи имеет элементы

$$A_{ij} = \phi_j(t_i).$$

Для линейной регрессии $\varphi(t) = c_0 + c_1 t$ нормальные уравнения принимают вид

$$\begin{pmatrix} m & \sum t_i \\ \sum t_i & \sum t_i^2 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum t_i y_i \end{pmatrix}.$$

27.5 Непрерывный метод наименьших квадратов

В пространстве со скалярным произведением

$$(f, g) = \int_a^b w(x) f(x) g(x) dx, \quad w(x) > 0,$$

ищется

$$\varphi_n = \sum_{j=0}^n c_j \phi_j$$

из условия

$$\|f - \varphi_n\|_2^2 \rightarrow \min.$$

Нормальные уравнения:

$$\sum_{j=0}^n (\phi_j, \phi_i) c_j = (f, \phi_i), \quad i = 0, \dots, n.$$

Если базис ортогонален, система распадается:

$$c_i = \frac{(f, \phi_i)}{(\phi_i, \phi_i)}.$$

При ортонормированном базисе $c_i = (f, \phi_i)$.

27.6 Взвешенный и регуляризованный МНК

Если наблюдения имеют различную точность, используют веса:

$$\min_x (Ax - b)^T W (Ax - b), \quad W = W^T > 0.$$

Нормальные уравнения:

$$A^T W A x = A^T W b.$$

При независимых ошибках с дисперсиями σ_i^2 обычно выбирают $W_{ii} = 1/\sigma_i^2$.

Для неустойчивых задач применяется регуляризация Тихонова:

$$\min_x (\|Ax - b\|_2^2 + \lambda \|Lx\|_2^2),$$

что приводит к системе

$$(A^T A + \lambda L^T L) x = A^T b.$$

27.7 Нелинейный метод наименьших квадратов

Для модели $r_i(x)$ минимизируется

$$J(x) = \frac{1}{2} \sum_{i=1}^m r_i(x)^2.$$

Метод Гаусса–Ньютона линеаризует невязку:

$$r(x + \Delta x) \approx r(x) + J_r(x)\Delta x$$

и решает

$$J_r^T J_r \Delta x = -J_r^T r.$$

Метод Левенберга–Марквардта добавляет стабилизирующий член:

$$(J_r^T J_r + \lambda I) \Delta x = -J_r^T r.$$

27.8 Статистическая интерпретация

Если

$$b = Ax + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I),$$

то оценка МНК совпадает с оценкой максимального правдоподобия. При выполнении условий линейной модели она является несмещённой, а её ковариационная матрица равна

$$\text{Cov}(x_*) = \sigma^2 (A^T A)^{-1}.$$

27.9 Что важно сказать на экзамене

Нужно вывести нормальные уравнения, объяснить ортогональность невязки и отметить, что вычислять решение через $(A^T A)^{-1}$ нежелательно. Численно надёжные методы — QR и SVD . Важные расширения — веса, регуляризация и нелинейный МНК.

28 Полиномиальная интерполяция

28.1 Постановка задачи и единственность

Пусть заданы попарно различные узлы

$$x_0, x_1, \dots, x_n$$

и значения $y_i = f(x_i)$. Требуется построить многочлен P_n степени не выше n такой, что

$$P_n(x_i) = y_i, \quad i = 0, \dots, n.$$

Такой многочлен существует и единственен. Действительно, если два многочлена степени не выше n совпадают в $n + 1$ различных точках, их разность имеет $n + 1$ корень и потому тождественно равна нулю.

28.2 Форма Лагранжа

Интерполяционный многочлен Лагранжа:

$$P_n(x) = \sum_{i=0}^n y_i \ell_i(x),$$

где

$$\ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

Базисные многочлены удовлетворяют

$$\ell_i(x_j) = \delta_{ij}.$$

Форма Лагранжа удобна теоретически, но при добавлении нового узла требуется перестроение всего выражения.

28.3 Барицентрическая формула

Для устойчивого вычисления используют барицентрическую форму. Введём веса

$$w_i = \frac{1}{\prod_{j \neq i} (x_i - x_j)}.$$

Тогда при $x \neq x_i$

$$P_n(x) = \frac{\sum_{i=0}^n \frac{w_i y_i}{x - x_i}}{\sum_{i=0}^n \frac{w_i}{x - x_i}}.$$

Если $x = x_i$, полагают $P_n(x) = y_i$. После предварительного вычисления весов стоимость одного значения — $O(n)$.

28.4 Форма Ньютона и разделённые разности

Многочлен Ньютона:

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + \dots + f[x_0, \dots, x_n] \prod_{j=0}^{n-1} (x - x_j).$$

Разделённые разности определяются рекурсивно:

$$f[x_i] = f(x_i),$$

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

При добавлении нового узла требуется вычислить только один новый коэффициент. Многочлен удобно вычислять вложенной схемой, аналогичной схеме Горнера.

Для равноотстоящих узлов разделённые разности выражаются через конечные разности, что приводит к формулам Ньютона вперёд и назад.

28.5 Погрешность интерполяции

Если $f \in C^{n+1}[a, b]$, то для каждого $x \in [a, b]$ существует $\xi = \xi(x)$ такое, что

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x),$$

где

$$\omega_{n+1}(x) = \prod_{i=0}^n (x - x_i).$$

Следовательно,

$$|f(x) - P_n(x)| \leq \frac{\max |f^{(n+1)}|}{(n+1)!} |\omega_{n+1}(x)|.$$

Погрешность зависит не только от гладкости функции, но и от расположения узлов.

28.6 Феномен Рунге и узлы Чебышёва

При интерполяции на равномерной сетке увеличение степени не гарантирует сходимости. Для функции

$$f(x) = \frac{1}{1 + 25x^2}$$

на $[-1, 1]$ возникают сильные колебания у концов отрезка — феномен Рунге.

Чтобы уменьшить максимум $|\omega_{n+1}(x)|$, выбирают узлы Чебышёва:

$$x_k = \cos \frac{(2k+1)\pi}{2(n+1)}, \quad k = 0, \dots, n,$$

с последующим линейным переносом на $[a, b]$. Они сгущаются у концов и дают почти оптимальную устойчивость полиномиальной интерполяции.

Чувствительность интерполятора к ошибкам данных характеризуется константой Лебега

$$\Lambda_n = \max_x \sum_{i=0}^n |\ell_i(x)|.$$

Для равноотстоящих узлов она растёт быстро, для узлов Чебышёва — логарифмически.

28.7 Интерполяция Эрмита

Если в узлах заданы значения функции и нескольких производных, строится интерполяционный многочлен Эрмита. Например, при заданных $f(x_i)$ и $f'(x_i)$ многочлен степени не выше $2n+1$ удовлетворяет

$$H(x_i) = f(x_i), \quad H'(x_i) = f'(x_i).$$

Его можно строить формой Ньютона с кратными узлами; разделённая разность на совпадающих узлах определяется производной:

$$f[\underbrace{x_i, \dots, x_i}_{k+1}] = \frac{f^{(k)}(x_i)}{k!}.$$

28.8 Что важно сказать на экзамене

Следует привести теорему существования и единственности, формы Лагранжа и Ньютона и остаточный член. Принципиальный практический момент: высокая степень на равномерных узлах может ухудшать результат; узлы Чебышёва и барицентрическая формула существенно повышают устойчивость.

29 Сплайн-аппроксимация

29.1 Определение сплайна

Пусть задано разбиение

$$a = x_0 < x_1 < \dots < x_n = b.$$

Сплайном степени p называется функция, которая на каждом отрезке $[x_{i-1}, x_i]$ является многочленом степени не выше p и обладает заданной гладкостью в узлах. Наиболее распространены кубические сплайны класса C^2 .

Главное преимущество сплайна перед глобальным многочленом — локальность: изменение одного узла влияет в основном на соседние участки, а степень остаётся малой независимо от числа данных.

29.2 Интерполирующий кубический сплайн

Кубический сплайн S должен удовлетворять:

1. S_i — кубический многочлен на $[x_{i-1}, x_i]$;
2. $S(x_i) = y_i$;
3. S , S' и S'' непрерывны во внутренних узлах;
4. заданы два дополнительных граничных условия.

Обозначим

$$h_i = x_i - x_{i-1}, \quad M_i = S''(x_i).$$

На отрезке $[x_{i-1}, x_i]$ сплайн можно записать как

$$S_i(x) = \frac{M_{i-1}(x_i - x)^3}{6h_i} + \frac{M_i(x - x_{i-1})^3}{6h_i} + \left(y_{i-1} - \frac{M_{i-1}h_i^2}{6}\right) \frac{x_i - x}{h_i} + \left(y_i - \frac{M_ih_i^2}{6}\right) \frac{x - x_{i-1}}{h_i}.$$

Непрерывность первой производной приводит к трёхдиагональной системе

$$h_i M_{i-1} + 2(h_i + h_{i+1})M_i + h_{i+1}M_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i} \right),$$

$$i = 1, \dots, n-1.$$

29.3 Граничные условия

Для замыкания системы используют два условия:

- естественный сплайн:

$$S''(a) = S''(b) = 0;$$

- закреплённый сплайн:

$$S'(a) = f'_a, \quad S'(b) = f'_b;$$

- периодический сплайн: совпадают значения производных на концах;
- условие not-a-knot: первые два и последние два участка образуют по одному кубическому многочлену.

При естественных условиях система для M_i имеет строгую диагональную преобладимость и решается методом прогонки за $O(n)$.

29.4 Вариационное свойство

Естественный интерполирующий кубический сплайн минимизирует функционал изгиба

$$J[v] = \int_a^b (v''(x))^2 dx$$

среди функций $v \in H^2(a, b)$, проходящих через все заданные точки. Это объясняет его геометрическую гладкость: среди интерполянтов он имеет минимальную “кривизну” в интегральном смысле.

29.5 Оценки погрешности

Если $f \in C^4[a, b]$, сетка квазиравномерна, а $h = \max h_i$, то для кубического интерполирующего сплайна типичны оценки

$$\|f - S\|_\infty = O(h^4), \quad \|f' - S'\|_\infty = O(h^3), \quad \|f'' - S''\|_\infty = O(h^2).$$

Точные константы зависят от граничных условий и регулярности сетки.

29.6 Сглаживающий сплайн

Для зашумлённых данных точная интерполяция нежелательна. Сглаживающий кубический сплайн минимизирует

$$J[S] = \sum_{i=0}^n w_i (y_i - S(x_i))^2 + \lambda \int_a^b (S''(x))^2 dx, \quad \lambda \geq 0.$$

Первое слагаемое отвечает за близость к данным, второе — за гладкость. При $\lambda \rightarrow 0$ получается интерполирующий сплайн, при $\lambda \rightarrow \infty$ решение приближается к линейной регрессии.

Параметр λ выбирают по оценке шума, перекрёстной проверке или обобщённой кросс-валидации.

29.7 B-сплайны

B-сплайны образуют локальный базис пространства сплайнов. Сплайн записывается как

$$S(x) = \sum_j c_j B_{j,p}(x).$$

Каждая базисная функция $B_{j,p}$ отлична от нуля лишь на небольшом числе соседних интервалов. Это даёт разреженные матрицы, локальное управление формой и численную устойчивость. Базис строится рекурсией Кокса–де Бура.

29.8 Что важно сказать на экзамене

Нужно определить кусочно-полиномиальную структуру и гладкость, вывести или хотя бы записать трёхдиагональную систему для вторых производных кубического сплайна, перечислить граничные условия и объяснить отличие интерполирующего сплайна от сглаживающего.

30 Численное дифференцирование

30.1 Постановка задачи

Численное дифференцирование применяется, когда функция задана таблицей значений или её аналитическая производная неудобна для вычисления. Производная заменяется линейной комбинацией значений функции в соседних узлах:

$$f^{(m)}(x_0) \approx \sum_j a_j f(x_j).$$

Коэффициенты выбираются так, чтобы формула была точна на полиномах максимально возможной степени. Обычно они выводятся из разложения Тейлора или дифференцирования интерполяционного многочлена.

30.2 Формулы для первой производной

На равномерной сетке с шагом h :

$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h)$$

— правая разность;

$$f'(x) = \frac{f(x) - f(x-h)}{h} + O(h)$$

— левая разность;

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2)$$

— центральная разность.

Формула второго порядка на границе:

$$f'(x) = \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} + O(h^2).$$

Формула четвёртого порядка во внутренней точке:

$$f'(x) = \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h} + O(h^4).$$

30.3 Вторая и высшие производные

Стандартная центральная формула для второй производной:

$$f''(x) = \frac{f(x-h) - 2f(x) + f(x+h)}{h^2} + O(h^2).$$

Формула четвёртого порядка:

$$f''(x) = \frac{-f(x+2h) + 16f(x+h) - 30f(x) + 16f(x-h) - f(x-2h)}{12h^2} + O(h^4).$$

Для смешанной производной функции двух переменных:

$$\frac{\partial^2 f}{\partial x \partial y}(x, y) \approx \frac{f(x+h, y+k) - f(x+h, y-k) - f(x-h, y+k) + f(x-h, y-k)}{4hk}.$$

30.4 Вывод методом неопределённых коэффициентов

Пусть требуется формула

$$f'(x_0) \approx \frac{1}{h} \sum_{j=-r}^s a_j f(x_0 + jh).$$

Подставим ряды Тейлора:

$$f(x_0 + jh) = \sum_{k=0}^{\infty} \frac{(jh)^k}{k!} f^{(k)}(x_0).$$

Для точности порядка p коэффициенты должны удовлетворять моментным условиям

$$\begin{aligned}\sum_j a_j &= 0, & \sum_j j a_j &= 1, \\ \sum_j j^k a_j &= 0, & k &= 2, \dots, p.\end{aligned}$$

Решение этой линейной системы даёт разностную формулу требуемого порядка.

30.5 Неравномерная сетка

При неравных шагах используют интерполяционный многочлен Лагранжа или Ньютона и дифференцируют его. Для трёх точек $x_{i-1} < x_i < x_{i+1}$ коэффициенты зависят от обоих шагов

$$h_- = x_i - x_{i-1}, \quad h_+ = x_{i+1} - x_i.$$

В частности,

$$\begin{aligned}f'(x_i) \approx & -\frac{h_+}{h_-(h_- + h_+)}f_{i-1} + \frac{h_+ - h_-}{h_-h_+}f_i \\ & + \frac{h_-}{h_+(h_- + h_+)}f_{i+1}.\end{aligned}$$

Формула имеет второй порядок при регулярном сгущении сетки.

30.6 Погрешность и выбор шага

Полная ошибка состоит из погрешности усечения и округления. Для центральной разности первой производной:

$$E(h) \approx C_1 h^2 + C_2 \frac{\varepsilon_{\text{mach}}}{h}.$$

При слишком большом h доминирует ошибка аппроксимации, при слишком малом — катастрофическая потеря значащих цифр при вычитании близких чисел. Оптимальный шаг по порядку величины:

$$h_{\text{opt}} \sim \varepsilon_{\text{mach}}^{1/3} \quad \text{для центральной разности первой производной.}$$

Для второй производной баланс ещё жёстче, поскольку ошибка округления делится на h^2 .

Численное дифференцирование является некорректной по Адамару операцией: малый высокочастотный шум в функции может давать большую ошибку производной.

30.7 Экстраполяция Ричардсона

Пусть приближение имеет разложение

$$D(h) = f'(x) + Ch^p + O(h^{p+1}).$$

Тогда комбинация

$$D_{\text{ext}} = \frac{2^p D(h/2) - D(h)}{2^p - 1}$$

устраняет главный член ошибки и повышает порядок точности. Метод можно применять последовательно, строя таблицу экстраполяции.

30.8 Дифференцирование зашумлённых данных

Прямые разности усиливают шум. Поэтому применяют:

- предварительное сглаживание;
- локальную полиномиальную аппроксимацию, например фильтр Савицкого–Голея;
- сглаживающие сплайны с последующим аналитическим дифференцированием;
- регуляризованные методы.

Порядок формулы сам по себе не гарантирует хорошего результата: широкие шаблоны могут сильнее реагировать на шум и границы.

30.9 Комплексный шаг

Для аналитической функции, вычисляемой в комплексной арифметике,

$$f'(x) = \frac{\operatorname{Im} f(x + ih)}{h} + O(h^2).$$

Метод не содержит вычитания близких чисел, поэтому допускает очень малые h . Он неприменим, если вычислительный код не поддерживает комплексные аргументы или содержит неаналитические операции.

30.10 Что важно сказать на экзамене

Нужно вывести формулы из рядов Тейлора, назвать их порядок и объяснить конфликт между ошибкой усечения и округления. Численное дифференцирование существенно чувствительнее к шуму, чем интегрирование, поэтому выбор шага и сглаживание имеют принципиальное значение.

31 Численное интегрирование

31.1 Квадратурные формулы

Численное интегрирование заменяет интеграл конечной суммой:

$$I(f) = \int_a^b w(x)f(x) dx \approx Q_n(f) = \sum_{i=1}^n A_i f(x_i),$$

где x_i — узлы, A_i — веса, $w(x)$ — весовая функция. Погрешность равна

$$R_n(f) = I(f) - Q_n(f).$$

Алгебраический порядок точности — наибольшая степень p , для которой формула точна для всех многочленов степени не выше p .

31.2 Формулы Ньютона–Котеса

Они получаются интегрированием интерполяционного многочлена на равноотстоящих узлах. Простейшие формулы на одном отрезке:

Левые прямоугольники:

$$\int_a^b f(x) dx \approx (b-a)f(a), \quad R = O((b-a)^2).$$

Средние прямоугольники:

$$\int_a^b f(x) dx \approx (b-a)f\left(\frac{a+b}{2}\right),$$

$$R = -\frac{(b-a)^3}{24}f''(\xi).$$

Трапеции:

$$\int_a^b f(x) dx \approx \frac{b-a}{2}(f(a) + f(b)),$$

$$R = -\frac{(b-a)^3}{12}f''(\xi).$$

Формула Симпсона:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right],$$

$$R = -\frac{(b-a)^5}{2880}f^{(4)}(\xi).$$

Симпсон точен для многочленов степени до трёх включительно.

31.3 Составные формулы

Разобьём $[a, b]$ на N равных частей, $h = (b-a)/N$. Составная формула трапеций:

$$T_h = h \left[\frac{f_0 + f_N}{2} + \sum_{i=1}^{N-1} f_i \right],$$

$$I - T_h = -\frac{b-a}{12}h^2f''(\xi).$$

Для чётного N составная формула Симпсона:

$$S_h = \frac{h}{3} \left[f_0 + f_N + 4 \sum_{\substack{i=1 \\ i \text{ нечётное}}}^{N-1} f_i + 2 \sum_{\substack{i=2 \\ i \text{ чётное}}}^{N-2} f_i \right],$$

$$I - S_h = -\frac{b-a}{180}h^4f^{(4)}(\xi).$$

31.4 Правило Рунге и адаптивное интегрирование

Если погрешность метода имеет вид Ch^p , то по двум сеткам:

$$I - Q_{h/2} \approx \frac{Q_{h/2} - Q_h}{2^p - 1}.$$

Это правило Рунге используется для оценки ошибки. В адаптивном алгоритме отрезок делят только там, где локальная оценка превышает допуск. Адаптивный Симпсон сравнивает формулу на всём отрезке с суммой формул на двух половинах.

31.5 Квадратуры Гаусса

В формулах Ньютона–Котеса узлы фиксированы. Если выбирать и узлы, и веса, то n -узловая квадратура может быть точна для многочленов степени до $2n - 1$:

$$\int_a^b w(x)f(x) dx \approx \sum_{i=1}^n A_i f(x_i).$$

Теорема Гаусса–Кристоффеля. Узлы квадратуры наивысшего алгебраического порядка являются корнями ортогонального многочлена степени n относительно веса $w(x)$ на $[a, b]$. Веса положительны.

Для $w(x) = 1$ на $[-1, 1]$ узлами являются корни многочлена Лежандра P_n . Переход на произвольный отрезок:

$$x = \frac{a+b}{2} + \frac{b-a}{2}t.$$

Пример двухузловой формулы Гаусса–Лежандра:

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

Она точна для многочленов степени до трёх.

31.6 Специальные квадратуры

Различным весам и областям соответствуют семейства ортогональных многочленов:

- Гаусс–Чебышёв:

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx;$$

- Гаусс–Лагерр:

$$\int_0^\infty e^{-x} f(x) dx;$$

- Гаусс–Эрмит:

$$\int_{-\infty}^\infty e^{-x^2} f(x) dx.$$

Выбор квадратуры, согласованной с весом и особенностями подынтегральной функции, резко повышает эффективность.

31.7 Несобственные интегралы и особенности

Для бесконечного интервала применяют замену переменной или специальные квадратуры Лагерра и Эрмита. При интегрируемой особенности полезно выделить её аналитически или сделать замену, сглаживающую функцию. Простое равномерное сгущение сетки часто неэффективно.

Для периодических гладких функций составная формула трапеций может сходиться очень быстро. Для осциллирующих интегралов применяют методы Филона и квадратуры, учитывающие фазу.

31.8 Ошибки вычислений

Численное интегрирование обычно сглаживает шум, однако при большом числе узлов накапливаются ошибки округления. Формулы Ньютона–Котеса высокого порядка могут иметь большие веса разных знаков и быть неустойчивыми. Поэтому на практике чаще используют составные низкопорядковые формулы, адаптивные методы или квадратуры Гаусса.

31.9 Что важно сказать на экзамене

Нужно привести общий вид квадратурной формулы, формулы трапеций и Симпсона с порядками ошибок, правило Рунге и принцип квадратур Гаусса. Главное отличие: Ньютон–Котес использует равноотстоящие узлы, Гаусс выбирает оптимальные узлы и достигает точности $2n - 1$.

32 Численные методы поиска экстремума функций

32.1 Постановка задачи

Задача безусловной оптимизации:

$$f(x) \rightarrow \min, \quad x \in \mathbb{R}^n.$$

Точка x_* является локальным минимумом, если существует окрестность, в которой $f(x_*) \leq f(x)$. Для дифференцируемой функции необходимое условие внутреннего экстремума:

$$\nabla f(x_*) = 0.$$

Если $f \in C^2$ и гессиан $\nabla^2 f(x_*)$ положительно определён, то x_* — строгий локальный минимум. Для выпуклой функции любой локальный минимум является глобальным.

Численный метод строит последовательность

$$x_{k+1} = x_k + \alpha_k p_k,$$

где p_k — направление поиска, α_k — длина шага.

32.2 Одномерная минимизация

Для унимодальной функции на $[a, b]$ применяются методы, не использующие производные.

Метод дихотомии. Интервал сужается сравнением значений в двух близких точках около середины. Длина интервала уменьшается примерно вдвое за итерацию.

Метод золотого сечения. Внутренние точки выбираются так, чтобы одна из них переиспользовалась на следующей итерации. Коэффициент сжатия:

$$\rho = \frac{\sqrt{5} - 1}{2} \approx 0,618.$$

За итерацию требуется одно новое вычисление функции.

Метод парабол. По трём точкам строится квадратичный интерполянт, а его вершина используется как новое приближение. Метод быстрее при гладкой функции, но требует защитных условий. Практический алгоритм Брента сочетает параболическую интерполяцию с золотым сечением.

Если доступна производная, минимум можно искать как корень $f'(x) = 0$ методами бисекции, секущих или Ньютона.

32.3 Градиентный спуск

Направление наискорейшего локального убывания:

$$p_k = -\nabla f(x_k).$$

Итерация:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k).$$

Шаг может быть постоянным, найденным одномерным поиском или выбранным по условиям Армихо и Вольфа.

Если градиент липшицев с константой L и функция выпукла, то при $0 < \alpha < 2/L$ метод сходится. Для сильно выпуклой функции с параметром μ скорость линейная и зависит от отношения L/μ , то есть от обусловленности. На вытянутых линиях уровня градиентный спуск медленно “зигзагирует”.

32.4 Метод Ньютона

Разложение второго порядка приводит к системе

$$\nabla^2 f(x_k) p_k = -\nabla f(x_k), \quad x_{k+1} = x_k + p_k.$$

Вблизи невырожденного минимума метод имеет квадратичную сходимость. Недостатки: вычисление и факторизация гессиана, отсутствие глобальной сходимости и возможная неположительная определённость вдали от минимума.

Глобализацию обеспечивают демпфирование

$$x_{k+1} = x_k + \alpha_k p_k$$

или доверительная область, где квадратичная модель минимизируется при $\|p\| \leq \Delta_k$.

32.5 Квазиныютоновские методы

Они строят приближение гессиана или обратного гессиана по изменениям

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

Наиболее распространён метод BFGS. Обновление обратного гессиана:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \rho_k = \frac{1}{y_k^T s_k}.$$

При $y_k^T s_k > 0$ положительная определённости сохраняется. L-BFGS хранит только несколько последних пар (s_k, y_k) и подходит для задач большой размерности.

32.6 Метод сопряжённых градиентов

Для квадратичной функции

$$f(x) = \frac{1}{2} x^T A x - b^T x, \quad A = A^T > 0,$$

метод строит A -сопряжённые направления и в точной арифметике достигает решения не более чем за n итераций. Для общего нелинейного функционала существуют нелинейные варианты Флетчера–Ривса и Полака–Рибьера.

32.7 Условная оптимизация

Пусть

$$g_i(x) \leq 0, \quad h_j(x) = 0.$$

Функция Лагранжа:

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \mu_j h_j(x).$$

При выполнении условий регулярности необходимы условия Каруша–Куна–Таккера:

$$\begin{aligned} \nabla_x \mathcal{L}(x_*, \lambda_*, \mu_*) &= 0, \\ g_i(x_*) &\leq 0, \quad h_j(x_*) = 0, \\ \lambda_i &\geq 0, \\ \lambda_i g_i(x_*) &= 0. \end{aligned}$$

Численные подходы:

- проекция на допустимое множество;
- штрафные и барьерные функции;
- метод множителей Лагранжа;
- последовательное квадратичное программирование;
- методы внутренней точки.

32.8 Критерии остановки

Обычно контролируют несколько величин:

$$\begin{aligned} \|\nabla f(x_k)\| &\leq \varepsilon_g, \quad \|x_{k+1} - x_k\| \leq \varepsilon_x(1 + \|x_k\|), \\ |f(x_{k+1}) - f(x_k)| &\leq \varepsilon_f(1 + |f(x_k)|). \end{aligned}$$

Для условной задачи дополнительно проверяются невязки ограничений и условия ККТ. Остановка только по малому изменению функции может быть ложной на плоском участке.

32.9 Что важно сказать на экзамене

Нужно разделить одномерные методы, методы первого и второго порядка и условную оптимизацию. Градиентный спуск дешёв, но чувствителен к обусловленности; Ньютон быстр вблизи решения, но дорог; BFGS даёт практический компромисс. Для ограничений основным теоретическим инструментом являются условия ККТ.

33 Численные методы решения систем обыкновенных дифференциальных уравнений

33.1 Задача Коши

Рассматривается система

$$y'(t) = f(t, y(t)), \quad y(t_0) = y_0, \quad y \in \mathbb{R}^m.$$

На сетке

$$t_0 < t_1 < \dots < t_N, \quad h_n = t_{n+1} - t_n$$

строятся приближения $y_n \approx y(t_n)$.

Методы классифицируют как:

- одношаговые и многошаговые;
- явные и неявные;
- одно- и многостадийные;
- методы постоянного и переменного шага.

33.2 Погрешность, порядок и сходимость

Локальная погрешность — ошибка одного шага при точных предыдущих значениях. Если она имеет порядок $O(h^{p+1})$, то при устойчивости глобальная ошибка обычно имеет порядок $O(h^p)$; p называется порядком метода.

Согласованность означает стремление локальной погрешности к нулю. Нулевая устойчивость многошагового метода предотвращает рост возмущений. Для линейных многошаговых методов согласованность и нулевая устойчивость эквивалентны сходимости.

33.3 Методы Эйлера и трапеций

Явный Эйлер:

$$y_{n+1} = y_n + h_n f(t_n, y_n).$$

Порядок — первый. Метод прост, но имеет малую область устойчивости.

Неявный Эйлер:

$$y_{n+1} = y_n + h_n f(t_{n+1}, y_{n+1}).$$

На каждом шаге решается нелинейная система. Метод первого порядка, но A -устойчив и хорошо подавляет жёсткие моды.

Метод трапеций:

$$y_{n+1} = y_n + \frac{h_n}{2} [f(t_n, y_n) + f(t_{n+1}, y_{n+1})].$$

Он имеет второй порядок и A -устойчив, но не L -устойчив: очень жёсткие моды могут затухать с паразитными осцилляциями.

33.4 Методы Рунге–Кутты

Общий s -стадийный метод:

$$K_i = f \left(t_n + c_i h, y_n + h \sum_{j=1}^s a_{ij} K_j \right), \quad i = 1, \dots, s,$$

$$y_{n+1} = y_n + h \sum_{i=1}^s b_i K_i.$$

Коэффициенты задаются таблицей Бутчера

$$\begin{array}{c|c} c & A \\ \hline & b^T \end{array}.$$

Если A строго нижнетреугольна, метод явный. Если на диагонали или выше неё есть ненулевые элементы, стадии связаны неявно.

Классический метод Рунге–Кутты четвёртого порядка:

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \\ k_4 &= f(t_n + h, y_n + hk_3), \\ y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

33.5 Вложенные методы и автоматический выбор шага

Вложенная пара Рунге–Кутты вычисляет два приближения порядков p и $p-1$ на одних стадиях. Их разность оценивает локальную ошибку:

$$err_n = \|y_{n+1}^{(p)} - y_{n+1}^{(p-1)}\|.$$

Новый шаг выбирают по формуле

$$h_{new} = \eta h \left(\frac{tol}{err_n} \right)^{1/(p+1)},$$

где $\eta < 1$ — коэффициент безопасности. Для векторной задачи ошибку нормируют с учётом абсолютного и относительного допусков каждой компоненты.

33.6 Линейные многошаговые методы

Общий метод имеет вид

$$\sum_{j=0}^k \alpha_j y_{n+j} = h \sum_{j=0}^k \beta_j f_{n+j}.$$

Если коэффициент при новом значении правой части равен нулю, метод явный; иначе — неявный.

Методы Адамса–Башфорта строятся интегрированием интерполяционного многочлена по предыдущим значениям f . Например, метод второго порядка:

$$y_{n+1} = y_n + \frac{h}{2}(3f_n - f_{n-1}).$$

Методы Адамса–Моултона неявны. Метод второго порядка совпадает с методом трапеций:

$$y_{n+1} = y_n + \frac{h}{2}(f_{n+1} + f_n).$$

Часто применяется схема предиктор–корректор: явный метод предсказывает y_{n+1} , неявный уточняет его.

33.7 Методы дифференцирования назад

BDF аппроксимируют производную интерполяционным многочленом по значениям решения и используют правую часть в новом времени. BDF1 — неявный Эйлер:

$$y_{n+1} - y_n = hf_{n+1}.$$

BDF2:

$$\frac{3y_{n+1} - 4y_n + y_{n-1}}{2h} = f_{n+1}.$$

BDF особенно эффективны для жёстких систем. На практике применяют порядки до пяти или шести; с ростом порядка ухудшаются свойства устойчивости.

33.8 Устойчивость и жёсткие системы

Для тестового уравнения

$$y' = \lambda y, \quad \operatorname{Re} \lambda < 0$$

численный метод даёт

$$y_{n+1} = R(z)y_n, \quad z = h\lambda.$$

Область абсолютной устойчивости:

$$\mathcal{S} = \{z \in \mathbb{C} : |R(z)| \leq 1\}.$$

Метод A -устойчив, если вся левая полуплоскость входит в $\mathcal{S}$. Метод L -устойчив, если он A -устойчив и $R(z) \rightarrow 0$ при $|z| \rightarrow \infty$ в левой полуплоскости.

Система жёсткая, если в ней присутствуют сильно различающиеся временные масштабы, и шаг явного метода определяется устойчивостью, а не требуемой точностью. Для таких задач используют неявный Эйлера, BDF, неявные и диагонально-неявные методы Рунге–Кутты.

33.9 Решение неявного шага

Неявный метод приводит к нелинейной системе

$$F(y_{n+1}) = 0.$$

Её решают методом Ньютона:

$$[I - h\gamma J_f] \Delta y = -F(y^{(k)}), \quad y^{(k+1)} = y^{(k)} + \Delta y.$$

Основная стоимость приходится на построение и факторизацию якобиана. В больших разреженных системах применяют приближённый якобиан, повторное использование факторизации и ньютон–крыловские методы.

33.10 Краевые задачи

Для краевых задач ОДУ применяют:

- метод стрельбы — сведение к задаче Коши и подбор недостающих начальных условий;
- конечно-разностные методы;
- коллокацию и конечные элементы.

Метод стрельбы прост, но может быть неустойчив для чувствительных или жёстких задач.

33.11 Что важно сказать на экзамене

Следует объяснить порядок, локальную и глобальную ошибки, явность и неявность. Основные семейства: Эйлера, Рунге–Кутты, Адамса и BDF. Центральный практический вопрос — устойчивость: явные методы хороши для нежёстких задач, неявные и BDF — для жёстких систем.

34 Вариационные принципы построения математических моделей

34.1 Понятие вариационного принципа

34.2 Функционал действия и энергетические функционалы

34.3 Принцип стационарного действия

34.4 Принцип возможных перемещений

34.5 Уравнения Эйлера–Лагранжа

34.6 Учёт связей и ограничений

34.7 Диссипативные системы

34.8 Примеры вариационных моделей

35 Вычислительный эксперимент в математическом моделировании

35.1 Понятие и цели вычислительного эксперимента

35.2 Этапы вычислительного эксперимента

35.3 Постановка серии расчётов

35.4 Планирование вычислительного эксперимента

35.5 Верификация и валидация

35.6 Анализ чувствительности

35.7 Обработка и интерпретация результатов

35.8 Воспроизводимость вычислений

36 Источники погрешности в математическом моделировании

36.1 Погрешность математической модели

36.2 Погрешность исходных данных и параметров

36.3 Погрешность дискретизации

36.4 Итерационная погрешность

36.5 Погрешность округления

36.6 Накопление и распространение ошибок

36.7 Устойчивость вычислительного алгоритма

36.8 Оценка полной погрешности

37 Краевые задачи для дифференциальных уравнений в частных производных

37.1 Классификация дифференциальных уравнений в частных производных

37.2 Начальные и граничные условия

37.3 Эллиптические краевые задачи

37.4 Параболические начально-краевые задачи

37.5 Гиперболические начально-краевые задачи

37.6 Корректность постановки задачи

37.7 Классическая и слабая постановки

37.8 Основные численные методы решения

38 Математические модели в биологии и биоинформатике (скип)

38.1 Особенности биологических систем как объектов моделирования

38.2 Популяционные модели

38.3 Эпидемиологические модели

38.4 Модели типа реакция–диффузия

38.5 Модели биохимических сетей

38.6 Модели последовательностей в биоинформатике

38.7 Филогенетические модели

38.8 Идентификация и проверка биологических моделей

- 39 Математические модели в механике деформируемого твёрдого тела**
 - 39.1 Кинематика деформирования**
 - 39.2 Тензоры деформаций и напряжений**
 - 39.3 Уравнения баланса**
 - 39.4 Определяющие соотношения**
 - 39.5 Начальные и граничные условия**
 - 39.6 Линейная теория упругости**
 - 39.7 Пластичность и вязкоупругость**
 - 39.8 Численные методы механики твёрдого тела**

40 Математические модели в экономике

40.1 Особенности экономических моделей

40.2 Балансовые модели

40.3 Оптимизационные модели

40.4 Динамические модели экономического роста

40.5 Модели межотраслевого баланса

40.6 Игровые модели

40.7 Стохастические модели

40.8 Идентификация и проверка экономических моделей

41 Математические модели механики жидкости и газа

Механика жидкости и газа изучает движение и равновесие сплошных текучих сред. Математическая модель строится на основе законов сохранения массы, импульса и энергии, дополненных определяющими соотношениями для конкретной среды и начальными и граничными условиями.

Основными искомыми полями обычно являются

$$\rho(\mathbf{x}, t), \quad \mathbf{v}(\mathbf{x}, t), \quad p(\mathbf{x}, t), \quad T(\mathbf{x}, t),$$

где ρ — плотность, $\mathbf{v}$ — скорость, p — давление, T — температура.

41.1 Гипотеза сплошной среды

Реальная жидкость или газ состоят из отдельных молекул, однако при описании макроскопических процессов обычно используется **гипотеза сплошной среды**. Согласно ей, вещество считается непрерывно распределённым в пространстве, а его характеристики задаются непрерывными полями.

Физические величины определяются как результат осреднения по малому объёму ΔV , который, с одной стороны, достаточно мал по сравнению с характерным масштабом задачи, а с другой — содержит достаточно большое число молекул. Например,

$$\rho(\mathbf{x}, t) = \lim_{\Delta V \rightarrow 0} \frac{\Delta m}{\Delta V},$$

где предел понимается в макроскопическом смысле, то есть до масштабов, на которых становится существенной молекулярная структура вещества.

Применимость модели сплошной среды определяется отношением длины свободного пробега молекул λ к характерному размеру области L :

$$\text{Kn} = \frac{\lambda}{L},$$

где Kn — **число Кнудсена**. При $\text{Kn} \ll 1$ континуальная модель обычно применима. При увеличении Kn необходимы кинетические модели, например уравнение Больцмана.

В рамках гипотезы сплошной среды вводятся поля плотности, скорости, давления, температуры, внутренней энергии, напряжений и других характеристик.

41.2 Кинематические характеристики потока

Положение частицы среды описывается радиус-вектором

$$\mathbf{x} = \mathbf{x}(\mathbf{a}, t),$$

где $\mathbf{a}$ задаёт положение частицы в начальный момент времени. Такое описание называется **лагранжевым**.

В **эйлеровом** описании рассматриваются поля физических величин в фиксированных точках пространства. Основной кинематической характеристикой является поле скорости

$$\mathbf{v}(\mathbf{x}, t) = \begin{pmatrix} u(\mathbf{x}, t) \\ v(\mathbf{x}, t) \\ w(\mathbf{x}, t) \end{pmatrix}.$$

Ускорение материальной частицы равно полной, или материальной, производной скорости:

$$\frac{D\mathbf{v}}{Dt} = \frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla)\mathbf{v}.$$

Первое слагаемое описывает локальное изменение скорости во времени, второе — конвективное изменение, связанное с перемещением частицы в неоднородном поле скоростей.

Материальная производная произвольной скалярной функции $\phi(\mathbf{x}, t)$ имеет вид

$$\frac{D\phi}{Dt} = \frac{\partial \phi}{\partial t} + \mathbf{v} \cdot \nabla \phi.$$

Важной характеристикой является дивергенция скорости:

$$\nabla \cdot \mathbf{v}.$$

Она характеризует локальную скорость объёмного расширения среды. Для несжимаемой среды

$$\nabla \cdot \mathbf{v} = 0.$$

Градиент скорости можно разложить на симметричную и антисимметричную части:

$$\nabla \mathbf{v} = \mathbf{D} + \mathbf{W},$$

где

$$\mathbf{D} = \frac{1}{2} (\nabla \mathbf{v} + (\nabla \mathbf{v})^T)$$

— тензор скоростей деформации, а

$$\mathbf{W} = \frac{1}{2} (\nabla \mathbf{v} - (\nabla \mathbf{v})^T)$$

характеризует локальное вращение среды.

Вектор вихря определяется как

$$\boldsymbol{\omega} = \nabla \times \mathbf{v}.$$

Если $\boldsymbol{\omega} = 0$, течение называется безвихревым.

Линия, касательная к которой в каждой точке совпадает с направлением скорости, называется **линией тока**. Для стационарного течения линии тока совпадают с траекториями частиц.

41.3 Законы сохранения массы, импульса и энергии

Закон сохранения массы

В интегральной форме для неподвижного контрольного объёма V :

$$\frac{d}{dt} \int_V \rho dV + \int_{\partial V} \rho \mathbf{v} \cdot \mathbf{n} dS = 0.$$

Локальная дифференциальная форма — **уравнение неразрывности**:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0.$$

С использованием материальной производной:

$$\frac{D\rho}{Dt} + \rho \nabla \cdot \mathbf{v} = 0.$$

Для несжимаемой жидкости с постоянной плотностью:

$$\nabla \cdot \mathbf{v} = 0.$$

Закон сохранения импульса

Баланс количества движения записывается как

$$\rho \frac{D\mathbf{v}}{Dt} = \nabla \cdot \boldsymbol{\sigma} + \rho \mathbf{f},$$

где $\boldsymbol{\sigma}$ — тензор напряжений, $\mathbf{f}$ — массовая сила на единицу массы, например ускорение свободного падения.

Тензор напряжений представляется в виде

$$\boldsymbol{\sigma} = -p\mathbf{I} + \boldsymbol{\tau},$$

где $-p\mathbf{I}$ — изотропная часть, соответствующая давлению, $\boldsymbol{\tau}$ — тензор вязких напряжений.

Следовательно,

$$\rho \frac{D\mathbf{v}}{Dt} = -\nabla p + \nabla \cdot \boldsymbol{\tau} + \rho \mathbf{f}.$$

Закон сохранения энергии

Полная удельная энергия

$$E = e + \frac{|\mathbf{v}|^2}{2},$$

где e — удельная внутренняя энергия.

Одна из форм закона сохранения полной энергии:

$$\frac{\partial(\rho E)}{\partial t} + \nabla \cdot [(\rho E + p)\mathbf{v}] = \nabla \cdot (\boldsymbol{\tau}\mathbf{v}) - \nabla \cdot \mathbf{q} + \rho \mathbf{f} \cdot \mathbf{v} + \rho r,$$

где $\mathbf{q}$ — плотность теплового потока, r — объёмный источник тепла на единицу массы.

Для внутренней энергии:

$$\rho \frac{De}{Dt} = \boldsymbol{\sigma} : \nabla \mathbf{v} - \nabla \cdot \mathbf{q} + \rho r.$$

Таким образом, фундаментальная модель течения строится как система уравнений баланса массы, импульса и энергии. Для замыкания этой системы необходимы определяющие соотношения.

41.4 Определяющие соотношения

Законы сохранения универсальны, но сами по себе не задают свойства конкретной жидкости или газа. Связь между напряжениями, деформациями, температурой, плотностью и другими величинами задаётся **определяющими соотношениями**.

Для ньютоновской жидкости вязкие напряжения линейно зависят от тензора скоростей деформации:

$$\boldsymbol{\tau} = 2\mu\mathbf{D} + \lambda(\nabla \cdot \mathbf{v})\mathbf{I},$$

где μ — динамическая вязкость, λ — второй коэффициент вязкости.

Для несжимаемой ньютоновской жидкости

$$\nabla \cdot \mathbf{v} = 0, \quad \boldsymbol{\tau} = 2\mu\mathbf{D}.$$

Передача тепла обычно описывается законом Фурье:

$$\mathbf{q} = -k\nabla T,$$

где k — коэффициент теплопроводности.

Для сжимаемого газа требуется уравнение состояния. Для идеального газа:

$$p = \rho RT,$$

где R — удельная газовая постоянная.

Для калорически совершенного идеального газа:

$$e = c_v T, \quad h = e + \frac{p}{\rho} = c_p T, \quad \gamma = \frac{c_p}{c_v}.$$

Таким образом, для модели сжимаемого вязкого теплопроводного газа используются: уравнения сохранения массы, импульса и энергии, ньютоновский закон вязкости, закон Фурье и уравнение состояния.

Если зависимость напряжений от скоростей деформации нелинейна, среда называется **неньютоновской**. Примеры: степенная жидкость, модель Бингама, вязкоупругие жидкости.

41.5 Уравнения Эйлера и Навье–Стокса

Уравнения Эйлера

Если вязкостью и теплопроводностью можно пренебречь, получаем модель идеальной жидкости или газа. Для сжимаемого течения система уравнений Эйлера имеет вид

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0,$$

$$\frac{\partial(\rho \mathbf{v})}{\partial t} + \nabla \cdot (\rho \mathbf{v} \otimes \mathbf{v} + p\mathbf{I}) = \rho \mathbf{f},$$

$$\frac{\partial(\rho E)}{\partial t} + \nabla \cdot [(\rho E + p)\mathbf{v}] = \rho \mathbf{f} \cdot \mathbf{v} + \rho r.$$

Для несжимаемой идеальной жидкости:

$$\nabla \cdot \mathbf{v} = 0,$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \rho \mathbf{f}.$$

Для стационарного безвихревого течения идеальной несжимаемой жидкости из уравнений Эйлера получается уравнение Бернулли:

$$\frac{p}{\rho} + \frac{|\mathbf{v}|^2}{2} + gz = \text{const}$$

вдоль линии тока.

Уравнения Навье–Стокса

Для ньютоновской жидкости с постоянными коэффициентами вязкости уравнение импульса имеет вид

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \mu \Delta \mathbf{v} + (\lambda + \mu) \nabla (\nabla \cdot \mathbf{v}) + \rho \mathbf{f}.$$

Для несжимаемой жидкости постоянной плотности:

$$\nabla \cdot \mathbf{v} = 0,$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \mu \Delta \mathbf{v} + \rho \mathbf{f}.$$

Вводя кинематическую вязкость

$$\nu = \frac{\mu}{\rho},$$

можно записать

$$\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} = -\frac{1}{\rho} \nabla p + \nu \Delta \mathbf{v} + \mathbf{f}.$$

Уравнения Навье–Стокса нелинейны из-за конвективного члена $(\mathbf{v} \cdot \nabla) \mathbf{v}$. Именно эта нелинейность в значительной мере определяет сложность задач гидрогазодинамики и возможность возникновения неустойчивостей и турбулентности.

41.6 Начальные и граничные условия

Для нестационарной задачи задаётся начальное состояние:

$$\mathbf{v}(\mathbf{x}, 0) = \mathbf{v}_0(\mathbf{x}), \quad \rho(\mathbf{x}, 0) = \rho_0(\mathbf{x}), \quad T(\mathbf{x}, 0) = T_0(\mathbf{x}),$$

в зависимости от рассматриваемой модели.

На твёрдой неподвижной стенке для вязкой жидкости обычно используется условие **прилипания**:

$$\mathbf{v} = \mathbf{0}.$$

Для движущейся стенки:

$$\mathbf{v} = \mathbf{v}_{\text{wall}}.$$

Условие непроницаемости имеет вид

$$\mathbf{v} \cdot \mathbf{n} = \mathbf{v}_{\text{wall}} \cdot \mathbf{n}.$$

Для идеальной жидкости, где касательные вязкие напряжения отсутствуют, часто задаётся только условие непротекания:

$$\mathbf{v} \cdot \mathbf{n} = 0.$$

На входе в расчётную область могут задаваться скорость, массовый расход, температура, полное давление и другие параметры. На выходе часто задают статическое давление либо используют условия, минимизирующие искусственное отражение возмущений.

Для температуры типичны:

$$T = T_0$$

— условие первого рода,

$$-k \frac{\partial T}{\partial n} = q_n$$

— заданный тепловой поток,

$$-k \frac{\partial T}{\partial n} = \alpha(T - T_\infty)$$

— конвективный теплообмен.

На свободной поверхности используются кинематическое условие движения границы и динамическое условие баланса напряжений. При наличии поверхностного натяжения скачок нормального напряжения связан с кривизной поверхности.

Для сжимаемых сверхзвуковых потоков число допустимых граничных условий зависит от направления характеристик: нельзя произвольно задавать все переменные на входе и выходе.

41.7 Критерии подобия и безразмерные параметры

Безразмеризация позволяет определить, какие физические механизмы доминируют в задаче. Пусть характерные масштабы равны

$$L, \quad U, \quad \rho_0, \quad T_0.$$

Введём

$$\mathbf{x} = L\mathbf{x}^*, \quad t = \frac{L}{U}t^*, \quad \mathbf{v} = U\mathbf{v}^*.$$

Для несжимаемых уравнений Навье–Стокса после безразмеризации возникает **число Рейнольдса**

$$\text{Re} = \frac{\rho UL}{\mu} = \frac{UL}{\nu}.$$

Оно характеризует отношение инерционных сил к вязким. При малых Re доминирует вязкость, при больших — инерционные эффекты и возрастает вероятность перехода к турбулентному режиму.

Для сжимаемых течений важным параметром является **число Маха**

$$\text{Ma} = \frac{U}{a},$$

где a — локальная скорость звука. При $\text{Ma} \ll 1$ эффекты сжимаемости обычно малы. При $\text{Ma} \gtrsim 1$ существенны волны сжатия, ударные волны и характеристики гиперболической части системы.

Число Фруда

$$\text{Fr} = \frac{U}{\sqrt{gL}}$$

характеризует отношение инерционных эффектов к гравитационным и особенно важно для течений со свободной поверхностью.

Число Прандтля

$$\text{Pr} = \frac{\nu}{\alpha},$$

где α — температуропроводность, сравнивает перенос импульса и тепла.

Число Пекле

$$\text{Pe} = \frac{UL}{\alpha} = \text{Re Pr}$$

характеризует отношение конвективного переноса тепла к теплопроводности.

Для нестационарных процессов используется **число Струхала**

$$\text{St} = \frac{fL}{U},$$

где f — характерная частота.

Подобие двух течений требует геометрического подобия и совпадения определяющих безразмерных параметров. Это лежит в основе физического моделирования и переноса результатов эксперимента с модели на натурный объект.

41.8 Модели турбулентности

При достаточно больших числах Рейнольдса течение может стать **турбулентным**: поля скорости и давления приобретают нерегулярные пульсации в широком диапазоне пространственных и временных масштабов.

Формально турбулентное течение по-прежнему описывается уравнениями Навье–Стокса. Проблема состоит в том, что прямое разрешение всех масштабов требует очень мелкой сетки и малого шага по времени.

Различают несколько основных подходов.

DNS (Direct Numerical Simulation) — прямое численное решение нестационарных уравнений Навье–Стокса с разрешением всех пространственных и временных масштабов турбулентности. Метод наиболее фундаментален, но крайне дорог вычислительно.

RANS (Reynolds-Averaged Navier–Stokes) основан на разложении

$$\mathbf{v} = \bar{\mathbf{v}} + \mathbf{v}', \quad p = \bar{p} + p',$$

где черта обозначает осреднение, штрих — пульсацию.

После осреднения в уравнениях появляются дополнительные рейнольдсовы напряжения

$$-\rho \overline{v'_i v'_j},$$

для которых необходимо задать модель замыкания. Часто используется гипотеза Буссинеска:

$$-\rho \overline{v'_i v'_j} = 2\mu_t \overline{D_{ij}} - \frac{2}{3}\rho k \delta_{ij},$$

где μ_t — турбулентная вязкость,

$$k = \frac{1}{2} \overline{v'_i v'_i}$$

— кинетическая энергия турбулентности.

Распространённые RANS-модели:

- модель k – ε ;
- модель k – ω ;
- модель SST k – ω ;
- модели рейнольдсовых напряжений.

LES (Large Eddy Simulation) разрешает крупные вихри непосредственно, а влияние мелкомасштабной турбулентности моделирует. По вычислительной стоимости и детальности LES занимает промежуточное положение между RANS и DNS.

Выбор модели определяется характером течения, требуемой точностью и доступными вычислительными ресурсами.

41.9 Основные численные методы

Аналитическое решение уравнений гидрогазодинамики возможно лишь для ограниченного класса задач, поэтому основным инструментом является численное моделирование.

Перед вычислением непрерывная область заменяется дискретной сеткой, а дифференциальные или интегральные уравнения — конечномерной системой алгебраических уравнений.

Основные методы пространственной дискретизации:

- **метод конечных разностей** — производные заменяются разностными аппроксимациями; удобен на структурированных сетках;
- **метод конечных объёмов** — интегральные законы сохранения записываются для каждого контрольного объёма; особенно распространён в вычислительной гидрогазодинамике, поскольку обеспечивает дискретное сохранение массы, импульса и энергии;
- **метод конечных элементов** — решение аппроксимируется в конечномерном пространстве базисных функций; удобен для сложной геометрии и вариационных постановок;
- **спектральные методы** — решение раскладывается по глобальным базисным функциям; обеспечивают высокую точность для достаточно гладких решений.

Для нестационарных задач дополнительно выполняется дискретизация по времени. Используются явные и неявные схемы, методы Рунге–Кутты, многошаговые методы.

Для явных схем важным ограничением является условие Куранта–Фридрихса–Леви:

$$\text{CFL} \sim \frac{U \Delta t}{\Delta x} \leq C,$$

где допустимая константа C зависит от конкретной схемы и уравнений.

В задачах несжимаемой жидкости давление не имеет собственного эволюционного уравнения и играет роль множителя, обеспечивающего выполнение условия

$$\nabla \cdot \mathbf{v} = 0.$$

Поэтому применяются специальные алгоритмы согласования скорости и давления: проекционные методы, SIMPLE, PISO и их модификации.

Для конвективных членов выбор аппроксимации особенно важен. Центральные схемы имеют малую численную диссипацию, но могут давать осцилляции при доминировании конвекции. Противопоточные схемы более устойчивы, но могут вносить искусственную диффузию. Для задач газовой динамики с разрывами применяются консервативные схемы повышенного разрешения: TVD, MUSCL, WENO и методы на основе решения задачи Римана.

К численному методу предъявляются требования:

- **аппроксимация** исходных уравнений;
- **устойчивость** относительно возмущений и ошибок округления;
- **сходимость** дискретного решения к точному при измельчении сетки;
- **консервативность**, особенно для законов сохранения;
- приемлемая вычислительная сложность и масштабируемость.

Практическая CFD-модель обычно включает следующие этапы:

1. выбор физической модели и набора уравнений;
2. построение геометрии и расчётной сетки;
3. задание свойств среды, начальных и граничных условий;
4. выбор пространственной и временной дискретизации;
5. решение полученной системы алгебраических уравнений;
6. контроль невязок и балансов;
7. исследование сеточной сходимости;
8. верификация и валидация результата.

Таким образом, математическая модель механики жидкости и газа представляет собой замкнутую систему законов сохранения, определяющих соотношений и начально-граничных условий. Выбор конкретной модели — Эйлера, Навье–Стокса, RANS, LES и т.д. — определяется физическими масштабами задачи, требуемой точностью и доступными вычислительными ресурсами.

42 Проверка адекватности математических моделей

42.1 Понятие адекватности модели

Математическая модель представляет собой формализованное описание существенных свойств, связей и закономерностей исследуемого объекта или процесса. При построении модели часть свойств реального объекта неизбежно отбрасывается, поэтому любая математическая модель является приближённым описанием действительности.

Адекватностью математической модели называют её способность с требуемой точностью воспроизводить свойства реального объекта, существенные для поставленной цели исследования, в заданной области применимости модели.

Таким образом, утверждение об адекватности всегда связано с:

- целью моделирования;
- набором наблюдаемых величин;
- допустимым уровнем погрешности;
- диапазоном параметров и режимов работы системы.

Модель может быть адекватной для одной задачи и неадекватной для другой. Например, упрощённая модель движения тела без сопротивления воздуха может быть достаточной для грубой оценки времени полёта на малых расстояниях, но непригодной для точного расчёта траектории.

Пусть реальный объект характеризуется наблюдаемой величиной

$$y^{\text{exp}}(x),$$

а математическая модель даёт прогноз

$$y^{\text{mod}}(x, \theta),$$

где θ — вектор параметров модели. Тогда расхождение

$$r(x, \theta) = y^{\text{exp}}(x) - y^{\text{mod}}(x, \theta)$$

называется **невязкой** или **остатком** модели.

В простейшем случае модель можно считать адекватной, если величина невязки в рассматриваемой области не превышает заданного допустимого уровня:

$$\|r\| \leq \varepsilon.$$

Однако одного совпадения с отдельным экспериментом недостаточно. Необходимо также проверять корректность реализации модели, устойчивость полученных результатов, воспроизводимость и способность модели описывать данные, которые не использовались при настройке её параметров.

Следует различать несколько близких понятий:

- **верификация** — правильно ли решены уравнения выбранной модели;
- **валидация** — правильно ли сама модель описывает реальный объект;
- **калибровка** — какие значения параметров модели лучше всего согласуют её с наблюдениями.

Условно эту разницу можно сформулировать следующим образом:

верификация: “правильно ли мы решаем модель?”

валидация: “правильную ли модель мы решаем?”

42.2 Верификация и валидация

Верификация заключается в проверке правильности математических преобразований, численных алгоритмов и программной реализации модели.

При верификации проверяется соответствие вычислительного результата исходной математической постановке.

Основные способы верификации:

1. сравнение с аналитическим решением для частного случая;

2. сравнение с эталонным численным решением;
3. сравнение результатов различных независимых алгоритмов;
4. исследование сходимости решения при измельчении сетки или уменьшении шага;
5. проверка законов сохранения и других инвариантов;
6. тестирование отдельных частей программного комплекса;
7. оценка влияния погрешностей округления и остановки итерационных процессов.

Например, если численная схема имеет порядок аппроксимации p , то при уменьшении характерного шага сетки h ожидается поведение ошибки вида

$$e_h \approx Ch^p.$$

Следовательно,

$$\frac{e_h}{e_{h/2}} \approx 2^p.$$

Наблюдение такого закона является одним из способов проверки реализации численного метода.

Валидация — это проверка соответствия математической модели реальному объекту или процессу. В отличие от верификации, валидация требует данных о реальной системе:

- экспериментальных измерений;
- натурных наблюдений;
- результатов эксплуатации;
- достоверных эмпирических зависимостей.

Типичная процедура проверки модели имеет вид

математическая модель $\longrightarrow$ численный алгоритм $\longrightarrow$ программа $\longrightarrow$ расчёт $\longrightarrow$ сравнение с экспериментом.

Если расчёт не согласуется с экспериментом, причиной могут быть:

1. неправильная математическая модель;
2. неверные значения её параметров;
3. недостаточно точный численный метод;
4. ошибка программной реализации;
5. погрешности экспериментальных данных.

Поэтому верификация обычно должна предшествовать валидации: бессмысленно оценивать адекватность физической модели, если неизвестно, правильно ли она решается вычислительным методом.

42.3 Сопоставление с экспериментальными данными

Пусть имеется набор экспериментальных наблюдений

$$\{(x_i, y_i^{\text{exp}})\}_{i=1}^N,$$

а расчёт по модели даёт значения

$$y_i^{\text{mod}} = y^{\text{mod}}(x_i, \theta).$$

Вводятся остатки

$$r_i = y_i^{\text{exp}} - y_i^{\text{mod}}.$$

Сравнение расчёта и эксперимента включает не только вычисление величины ошибок, но и анализ их структуры. Хорошая модель должна, как правило, характеризоваться:

- малыми остатками;

- отсутствием выраженного систематического смещения;
- отсутствием закономерной зависимости остатка от входных параметров;
- правильным воспроизведением качественного поведения системы;
- работоспособностью на данных, не использовавшихся при настройке.

Важно различать **обучающую** и **проверочную** выборки. Если параметры модели настраиваются по данным

$$D_{\text{cal}},$$

то оценивать прогнозирующую способность модели желательно на независимом наборе

$$D_{\text{val}}.$$

Иначе сложная модель может хорошо описывать исходный эксперимент, но плохо предсказывать новые наблюдения. Такая ситуация соответствует **переобучению**.

Для динамических моделей сравниваются не только отдельные значения, но и такие характеристики, как:

- положение и высота максимумов;
- периоды колебаний;
- скорости роста и затухания;
- переходные процессы;
- стационарные состояния.

Сопоставление желательно проводить не в одной точке, а во всей предполагаемой области применимости модели.

42.4 Критерии качества модели

Для количественной оценки качества модели применяются различные показатели.

Средняя абсолютная ошибка:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i^{\text{exp}} - y_i^{\text{mod}}|.$$

Среднеквадратичная ошибка:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i^{\text{exp}} - y_i^{\text{mod}})^2.$$

Среднеквадратичное отклонение:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i^{\text{exp}} - y_i^{\text{mod}})^2}.$$

В отличие от MAE, критерии MSE и RMSE сильнее штрафуют большие ошибки.

Если важна относительная ошибка, используют, например,

$$\delta_i = \frac{|y_i^{\text{exp}} - y_i^{\text{mod}}|}{|y_i^{\text{exp}}|}.$$

Для регрессионных моделей часто применяется коэффициент детерминации

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i^{\text{exp}} - y_i^{\text{mod}})^2}{\sum_{i=1}^N (y_i^{\text{exp}} - \bar{y}^{\text{exp}})^2},$$

где

$$\bar{y}^{\text{exp}} = \frac{1}{N} \sum_{i=1}^N y_i^{\text{exp}}.$$

Чем ближе R^2 к единице, тем большую часть вариации данных объясняет модель. Однако большое R^2 само по себе не доказывает адекватность модели.

При известных дисперсиях экспериментальных ошибок σ_i^2 естественно использовать взвешенный критерий

$$\chi^2 = \sum_{i=1}^N \frac{(y_i^{\text{exp}} - y_i^{\text{mod}})^2}{\sigma_i^2}.$$

Кроме численной величины ошибки необходимо учитывать:

- устойчивость результата;
- физическую интерпретируемость;
- вычислительную стоимость;
- сложность модели;
- способность модели к прогнозированию.

Чрезмерное усложнение модели не всегда улучшает её качество. Если две модели дают сопоставимую точность, обычно предпочтительна более простая модель с меньшим числом параметров.

42.5 Калибровка параметров

Во многих моделях имеются параметры, значения которых заранее известны неточно:

$$\theta = (\theta_1, \dots, \theta_m).$$

Калибровкой называют определение параметров модели на основании экспериментальных данных.

Пусть

$$y_i^{\text{mod}} = y(x_i, \theta).$$

Тогда параметры можно определять из задачи минимизации

$$\theta^* = \arg \min_{\theta} J(\theta),$$

где функционал качества, например, имеет вид

$$J(\theta) = \sum_{i=1}^N (y_i^{\text{exp}} - y(x_i, \theta))^2.$$

Если измерения имеют различную точность, применяется взвешенный метод наименьших квадратов:

$$J(\theta) = \sum_{i=1}^N w_i (y_i^{\text{exp}} - y(x_i, \theta))^2,$$

где обычно

$$w_i \sim \frac{1}{\sigma_i^2}.$$

В более общем случае задача калибровки может формулироваться как задача максимального правдоподобия:

$$\theta^* = \arg \max_{\theta} L(\theta \mid D),$$

где D — экспериментальные данные.

При калибровке важно не смешивать два процесса:

- параметры определяются по одной части данных;
- качество модели проверяется по другой части данных.

Иначе можно получить параметры, идеально описывающие конкретный набор измерений, но не имеющие прогнозирующей способности.

42.6 Идентифицируемость модели

Даже если задача калибровки сформулирована, из экспериментальных данных не всегда можно однозначно определить параметры модели.

Идентифицируемость характеризует возможность однозначного восстановления параметров модели по доступным наблюдениям.

Пусть

$$y = F(\theta).$$

Если

$$F(\theta^{(1)}) = F(\theta^{(2)})$$

влечёт

$$\theta^{(1)} = \theta^{(2)},$$

то отображение параметров в наблюдаемые величины является инъективным, и параметры могут быть однозначно идентифицированы.

Различают:

- **структурную идентифицируемость** — возможность однозначно определить параметры при идеальных, точных данных;
- **практическую идентифицируемость** — возможность достаточно точно определить параметры по реальным зашумлённым данным.

Модель может быть структурно идентифицируема, но практически плохо идентифицируема, если разные значения параметров приводят к почти одинаковым результатам.

Для локального анализа вводится матрица чувствительности

$$S_{ij} = \frac{\partial y_i}{\partial \theta_j}.$$

Если её столбцы линейно зависимы или почти линейно зависимы, то воздействие некоторых параметров на результат трудно различить. В таком случае задача определения параметров становится плохо обусловленной.

Неидентифицируемость может быть устранена:

- проведением дополнительных экспериментов;
- измерением дополнительных величин;
- фиксацией части параметров;
- изменением постановки эксперимента;
- упрощением модели;
- применением регуляризации.

42.7 Анализ чувствительности

Анализ чувствительности исследует изменение результата модели при изменении её входных данных и параметров.

Если выход модели

$$y = F(\theta_1, \dots, \theta_m),$$

то локальная чувствительность к параметру θ_j определяется производной

$$S_j = \frac{\partial F}{\partial \theta_j}.$$

Для сравнения параметров различных размерностей удобно использовать безразмерный коэффициент чувствительности:

$$S_j^{\text{rel}} = \frac{\theta_j}{F} \frac{\partial F}{\partial \theta_j}.$$

Он показывает относительное изменение результата при малом относительном изменении параметра. На практике производная может оцениваться конечной разностью:

$$\frac{\partial F}{\partial \theta_j} \approx \frac{F(\theta_j + \Delta\theta_j) - F(\theta_j)}{\Delta\theta_j}.$$

Различают:

- **локальный анализ чувствительности**, исследующий малые изменения около некоторого номинального набора параметров;
- **глобальный анализ чувствительности**, исследующий влияние параметров во всём допустимом диапазоне.

При глобальном анализе параметры рассматриваются как случайные величины. Например, дисперсия результата $\text{Var}(Y)$

может быть разложена на вклады отдельных параметров и их взаимодействий. На этом основаны, в частности, индексы чувствительности Соболя.

Анализ чувствительности позволяет:

1. определить наиболее важные параметры модели;
2. выявить практически несущественные параметры;
3. понять причины большой неопределённости результата;
4. спланировать необходимые измерения;
5. обнаружить проблемы идентифицируемости;
6. упростить модель.

42.8 Учёт неопределённости

Результат математического моделирования обычно содержит неопределённость, связанную как с самой моделью, так и с входными данными.

Основные источники неопределённости:

1. неопределённость параметров;
2. погрешности измерений;
3. неопределённость начальных и граничных условий;
4. упрощения математической модели;
5. численная погрешность;
6. случайный характер моделируемого процесса.

Часто различают два основных типа неопределённости.

Алеаторная неопределённость обусловлена объективной случайностью процесса. Например, случайные внешние воздействия. Она не устраняется простым увеличением числа измерений.

Эпистемическая неопределённость обусловлена недостатком знаний: неточностью параметров, неполнотой модели или недостаточным объёмом экспериментальных данных. Она потенциально может быть уменьшена дополнительными исследованиями.

Если входной параметр θ рассматривается как случайная величина с плотностью $p_\theta(\theta)$, а

$$Y = F(\theta),$$

то неопределённость входных данных распространяется на результат Y .

Одним из наиболее универсальных методов является метод Монте-Карло:

1. генерируются реализации входных параметров

$$\theta^{(1)}, \dots, \theta^{(M)};$$

2. для каждой реализации выполняется расчёт

$$Y^{(k)} = F(\theta^{(k)});$$

3. по полученной выборке оцениваются статистические характеристики результата.

Например,

$$\bar{Y} = \frac{1}{M} \sum_{k=1}^M Y^{(k)},$$

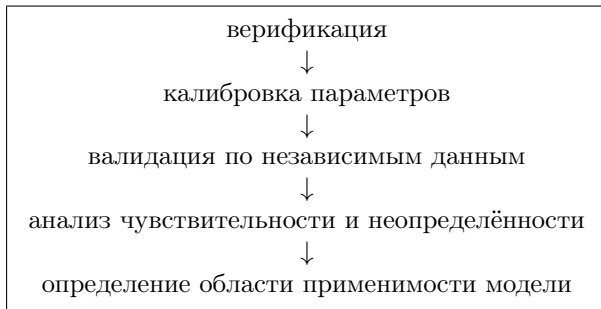
$$s_Y^2 = \frac{1}{M-1} \sum_{k=1}^M \left(Y^{(k)} - \bar{Y} \right)^2.$$

Вместо единственного значения результата корректнее во многих случаях приводить интервал

$$Y \in [Y_{\min}, Y_{\max}]$$

или доверительный интервал заданного уровня.

Таким образом, проверка адекватности математической модели представляет собой не один критерий, а совокупность процедур:



Модель следует считать адекватной не вообще, а только относительно заданной цели, набора наблюдаемых характеристик, требуемой точности и установленной области применимости.

43 Универсальность математических моделей

абстрагирование, одинаковые структуры, аналогии, безразмерные критерии, перенос модели

43.1 Абстрагирование при построении моделей

Математическая модель представляет собой формализованное описание существенных свойств исследуемого объекта или процесса с помощью математических объектов: уравнений, функций, графов, вероятностных распределений, алгоритмов и т.д.

При построении модели реальный объект заменяется его идеализированным представлением. Этот процесс называется **абстрагированием**.

Основная идея абстрагирования состоит в том, что из большого количества свойств реальной системы выделяются только те, которые существенны для решаемой задачи. Остальные свойства считаются второстепенными и в модель не включаются.

Например, при описании движения материальной точки не учитываются её форма и внутреннее строение. Реальное тело заменяется точкой с массой m , положение которой описывается радиус-вектором

$$\mathbf{r} = \mathbf{r}(t).$$

В результате закон движения может быть записан в форме

$$m \frac{d^2 \mathbf{r}}{dt^2} = \mathbf{F}.$$

Такое уравнение применимо к большому числу физических объектов, несмотря на существенные различия между ними.

Абстрагирование обычно включает следующие этапы:

- выделение объекта моделирования;
- определение цели моделирования;
- выбор существенных переменных и параметров;
- формулировку основных предположений;
- установление связей между переменными;
- математическую формализацию этих связей.

Чем выше уровень абстракции, тем шире класс объектов, которые может описывать модель. Однако чрезмерное абстрагирование может привести к потере существенных свойств системы.

Именно абстрагирование является одной из причин универсальности математических моделей: различные реальные системы после исключения несущественных деталей могут приводить к одинаковым математическим задачам.

43.2 Принцип аналогий

Принцип аналогий заключается в том, что различные по физической, биологической, экономической или иной природе системы могут описываться одинаковыми математическими соотношениями.

Если две системы после введения соответствующих переменных описываются одной и той же системой уравнений, то с математической точки зрения они обладают сходной структурой.

Простейший пример — линейное дифференциальное уравнение

$$\frac{dx}{dt} = -kx.$$

Его решение имеет вид

$$x(t) = x_0 e^{-kt}.$$

Одно и то же уравнение может описывать:

- радиоактивный распад;
- разряд конденсатора;
- охлаждение тела в простейшей модели;

- уменьшение концентрации вещества;
- выведение вещества из организма;
- изменение численности популяции при постоянной удельной смертности.

Физический смысл величин x и k в этих задачах различен, но математическая структура одинакова. Другой пример — уравнение диффузионного типа

$$\frac{\partial u}{\partial t} = D\Delta u.$$

В зависимости от интерпретации величины u оно может описывать:

- распределение концентрации вещества;
- распространение тепла;
- перенос примеси;
- некоторые вероятностные процессы;
- упрощённые модели распространения биологических популяций.

Таким образом, однажды исследовав математическую задачу, можно использовать полученные результаты для целого класса объектов.

Принцип аналогий позволяет:

- переносить математические методы между предметными областями;
- использовать уже исследованные модели для описания новых объектов;
- обнаруживать общие закономерности в различных системах;
- проводить вычислительные эксперименты на универсальных моделях.

При этом математическая аналогия не означает полной физической идентичности систем. Совпадение уравнений ещё не гарантирует совпадения всех свойств реальных объектов.

43.3 Подобие и безразмеризация

Одним из важных способов выявления универсальных свойств модели является переход к **безразмерным переменным**.

Пусть некоторое уравнение содержит характерные величины: характерную длину L , время T , скорость U и т.д. Вводятся безразмерные переменные, например

$$x = L\tilde{x}, \quad t = T\tilde{t}, \quad u = U\tilde{u}.$$

После подстановки в исходные уравнения размерные параметры объединяются в безразмерные комбинации. Например, для уравнения переноса и диффузии

$$\frac{\partial c}{\partial t} + \mathbf{u} \cdot \nabla c = D\Delta c$$

введём масштабы

$$x = L\tilde{x}, \quad t = \frac{L}{U}\tilde{t}.$$

После преобразования получаем

$$\frac{\partial c}{\partial \tilde{t}} + \tilde{\mathbf{u}} \cdot \tilde{\nabla} c = \frac{1}{Pe} \tilde{\Delta} c,$$

где

$$Pe = \frac{UL}{D}$$

— **число Пекле**.

Таким образом, поведение системы определяется не отдельно величинами U , L и D , а их безразмерной комбинацией.

В механике жидкости аналогично возникает число Рейнольдса

$$Re = \frac{\rho UL}{\mu} = \frac{UL}{\nu},$$

которое характеризует отношение инерционных и вязких эффектов.

Две физически различные системы, имеющие одинаковые значения определяющих безразмерных параметров, могут иметь сходное поведение. Это называется **подобием**.

Безразмеризация позволяет:

- уменьшить число параметров задачи;
- выделить определяющие физические механизмы;
- сравнивать системы различных размеров;
- планировать модельные эксперименты;
- выявлять предельные режимы;
- улучшать численную обусловленность задачи.

Общий подход к построению безразмерных комбинаций формализуется Π -теоремой Бэкингема: если задача содержит n размерных параметров, выражаемых через k независимых основных единиц измерения, то её можно описать с помощью $n - k$ независимых безразмерных комбинаций.

43.4 Масштабные законы

Масштабными законами называют зависимости, описывающие изменение характеристик системы при изменении её пространственного, временного или иного масштаба.

Часто они имеют степенной вид

$$Y \sim X^\alpha,$$

где α — показатель масштабирования.

Простейший пример связан с геометрически подобными телами. При изменении характерного линейного размера L

$$S \sim L^2, \quad V \sim L^3,$$

где S — площадь поверхности, а V — объём.

Следовательно,

$$\frac{S}{V} \sim \frac{1}{L}.$$

Это означает, что при увеличении размеров тела его объём растёт быстрее, чем площадь поверхности. Такое простое соотношение имеет последствия для теплообмена, биологических систем, прочности конструкций и других задач.

Другой пример получается для диффузионного процесса. Характерное время диффузии на расстояние L имеет порядок

$$T \sim \frac{L^2}{D}.$$

Следовательно, увеличение пространственного масштаба в два раза приводит примерно к четырёхкратному увеличению характерного времени диффузии.

Масштабный анализ позволяет:

- получать оценки без полного решения уравнений;
- определять доминирующие члены математической модели;
- строить упрощённые модели;
- сравнивать процессы разных масштабов;
- определять характерные времена и длины процесса.

Важным применением является выделение малого параметра. Например, если после безразмеризации получается

$$\varepsilon A(u) + B(u) = 0, \quad \varepsilon \ll 1,$$

то в первом приближении можно рассматривать более простую задачу

$$B(u) = 0.$$

При этом необходимо отдельно проверять, является ли такой предельный переход регулярным: в сингулярно возмущённых задачах малый коэффициент может приводить, например, к появлению пограничных слоёв.

43.5 Универсальные математические структуры

Во многих предметных областях повторяются одни и те же математические структуры.

Одной из наиболее распространённых является **закон баланса**. Для некоторой величины Q он символически имеет вид

$$\frac{dQ}{dt} = \text{приток} - \text{отток} + \text{источники}.$$

В пространственно распределённой системе закон локального баланса может быть записан как

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{J} = f,$$

где $\mathbf{J}$ — поток, а f — плотность источников.

Из этой общей формы выводятся:

- уравнение неразрывности;
- уравнение теплопроводности;
- уравнение диффузии;
- уравнения переноса примеси;
- различные популяционные и экономические балансовые модели.

Другой универсальной структурой является **динамическая система**

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t).$$

Вектор состояния $\mathbf{x}$ может обозначать совершенно разные величины:

- координаты и скорости механической системы;
- концентрации химических веществ;
- численности биологических популяций;
- макроэкономические показатели;
- электрические токи и напряжения.

Также широко встречаются:

- задачи оптимизации

$$J(x) \rightarrow \min;$$

- вариационные задачи

$$J[u] \rightarrow \min;$$

- сетевые модели и графы;
- марковские модели;
- задачи на собственные значения;
- интегральные и дифференциальные уравнения;
- стохастические дифференциальные уравнения.

Универсальность математических структур означает, что методы исследования обычно относятся не к конкретному физическому объекту, а к определённому классу математических задач.

Например, метод конечных элементов можно применять к задачам теории упругости, теплопроводности, электродинамики и многим другим задачам, поскольку после вариационной постановки они имеют сходную математическую структуру.

43.6 Перенос моделей между предметными областями

Если в новой предметной области обнаружена математическая структура, аналогичная уже исследованной, можно использовать ранее разработанную модель или её элементы.

Такой перенос обычно включает несколько этапов:

1. выделение состояния системы и основных процессов;
2. поиск известной математической структуры;
3. установление соответствия между переменными;
4. интерпретацию параметров в новой предметной области;
5. определение начальных и граничных условий;
6. проверку адекватности новой модели.

Например, логистическое уравнение

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right)$$

первоначально естественно интерпретируется как модель роста ограниченной популяции.

Однако аналогичная нелинейная структура может использоваться при описании распространения технологии, роста некоторого рынка или других процессов с насыщением.

Здесь:

r

характеризует начальную скорость роста, а

K

— предельный уровень насыщения.

При переносе модели меняется содержательная интерпретация параметров, но сохраняется математический аппарат анализа:

- поиск стационарных состояний;
- исследование устойчивости;
- анализ чувствительности;
- численное интегрирование;
- оценка параметров по данным.

Таким образом, универсальность модели существенно снижает стоимость исследования новых систем: не требуется каждый раз создавать математический аппарат с нуля.

43.7 Границы применимости универсальных моделей

Универсальность математической модели не означает её неограниченной применимости.

Каждая модель строится на основе определённых предположений, поэтому должна быть указана её **область применимости**.

Например, линейная модель

$$y = kx$$

может достаточно точно описывать систему при малых отклонениях от некоторого рабочего состояния, но становится непригодной при больших значениях x .

Аналогично уравнения механики сплошной среды предполагают, что вещество можно рассматривать как непрерывную среду. На масштабах, сопоставимых с молекулярными, это приближение перестаёт быть корректным.

Основными причинами ограничения применимости модели являются:

- нарушение исходных предположений;
- изменение характерных масштабов процесса;

- появление ранее неучтённых механизмов;
- изменение диапазона параметров;
- существенное влияние случайных факторов;
- недостаточная точность исходных данных;
- структурная погрешность самой модели.

Особенно осторожно необходимо относиться к переносу модели между различными предметными областями. Одинаковая математическая форма уравнений не гарантирует одинаковых:

- физических ограничений;
- допустимых значений параметров;
- начальных и граничных условий;
- критериев устойчивости;
- механизмов возникновения нелинейностей.

Поэтому универсальная модель должна проходить **валидацию** — сопоставление результатов моделирования с экспериментальными или наблюдаемыми данными.

Следует различать две идеи:

- **универсальность математической структуры** — одинаковые математические закономерности встречаются во многих системах;
- **адекватность конкретной модели** — модель достаточно точно описывает конкретный объект в заданных условиях.

Таким образом, универсальность математических моделей основана на абстрагировании, принципе аналогий, подобии, безразмеризации и существовании общих математических структур. Она позволяет переносить методы и результаты между различными предметными областями, однако такой перенос всегда должен сопровождаться анализом предположений и проверкой границ применимости модели.

44 Базовые концепции объектно-ориентированного программирования

Объектно-ориентированное программирование (ООП) — парадигма программирования, в которой программа представляется как совокупность взаимодействующих объектов. Каждый объект обладает состоянием, поведением и идентичностью.

Основными понятиями ООП являются классы и объекты, инкапсуляция, наследование, полиморфизм и абстракция. На практике важную роль также играют композиция объектов, интерфейсы и принципы проектирования, позволяющие строить расширяемые и сопровождаемые программные системы.

44.1 Объекты и классы

Объект — сущность программы, обладающая:

- **состоянием** — совокупностью хранимых данных;
- **поведением** — набором операций, которые объект способен выполнять;
- **идентичностью** — возможностью отличить данный объект от других, даже если их состояния совпадают.

Например, объект, представляющий матрицу, может хранить её размеры и элементы, а также предоставлять операции сложения, умножения, вычисления нормы и т.д.

Класс — описание множества объектов с общей структурой и поведением. Класс определяет:

- набор данных или полей;
- набор методов;
- правила создания и уничтожения объектов;
- ограничения доступа к отдельным элементам объекта.

Объект называют **экземпляром класса**.

Условно класс можно представить следующим образом:

```
class Vector {  
    data  
    coordinates  
  
    methods  
    norm()  
    add(Vector other)  
}
```

Конкретные объекты

```
v1 = Vector(...)  
v2 = Vector(...)
```

являются различными экземплярами одного класса **Vector**.

Следует различать **класс** и **тип**. Во многих объектно-ориентированных языках класс задаёт новый тип, но в общем случае эти понятия не тождественны. Например, интерфейс также может задавать тип, не задавая конкретного способа хранения состояния объекта.

44.2 Инкапсуляция

Инкапсуляция — объединение данных и операций над ними внутри некоторой программной сущности с контролем доступа к её внутреннему состоянию.

Основная идея состоит в том, что пользователь объекта взаимодействует не с его внутренним представлением, а с некоторым публичным интерфейсом.

Например, если класс описывает банковский счёт, непосредственное произвольное изменение величины баланса нежелательно:

```
account.balance = -1000000
```

Вместо этого состояние должно изменяться посредством допустимых операций:

```
account.deposit(amount)
account.withdraw(amount)
```

Метод `withdraw` может проверить достаточность средств и тем самым поддерживать корректность состояния объекта.

Таким образом, инкапсуляция позволяет поддерживать **инварианты объекта** — свойства, которые должны оставаться истинными при всех допустимых операциях.

Во многих языках для управления доступом используются модификаторы:

- `public` — элемент доступен пользователю класса;
- `private` — элемент доступен только внутри самого класса;
- `protected` — элемент доступен внутри класса и его наследников.

Инкапсуляция уменьшает зависимость внешнего кода от внутреннего устройства класса. Например, внутреннее представление контейнера можно заменить с массива на связный список без изменения пользовательского кода, если публичный интерфейс остаётся тем же.

Следует отличать инкапсуляцию от простого **сокрытия данных**. Сокрытие является одним из механизмов реализации инкапсуляции, однако сама инкапсуляция шире: она включает объединение состояния и поведения и определение правильной границы ответственности объекта.

44.3 Наследование

Наследование — механизм создания нового класса на основе существующего, при котором новый класс получает часть свойств и поведения базового класса и может расширять или изменять их.

Исходный класс называют **базовым** или **родительским**, а новый — **производным** или **дочерним**.

Например:

```
class Shape {
    area()
}

class Circle extends Shape {
    radius
    area()
}

class Rectangle extends Shape {
    width
    height
    area()
}
```

Классы `Circle` и `Rectangle` имеют общее свойство: оба являются геометрическими фигурами и поддерживают операцию вычисления площади.

Наследование решает две основные задачи:

1. повторное использование некоторой реализации;
2. построение иерархии типов.

Особенно важно различать эти задачи.

Наследование корректно использовать для отношения “**является**” (*is-a*):

Circle is a Shape.

Если же объект является лишь частью другого объекта, обычно следует использовать композицию:

Car has an Engine.

Наследование может быть:

- **одиночным** — класс имеет одного непосредственного предка;
- **множественным** — класс имеет нескольких непосредственных предков;
- **многоуровневым** — образуется цепочка наследования.

Множественное наследование может приводить к неоднозначностям. Классический пример — **ромбовидное наследование**, когда два класса наследуют общий базовый класс, а затем некоторый четвёртый класс наследует оба из них. Различные языки решают эту проблему по-разному или вообще запрещают множественное наследование классов.

44.4 Полиморфизм

Полиморфизм — возможность использовать один интерфейс для работы с объектами различных типов.

Идея полиморфизма состоит в том, что клиентский код задаёт объекту требуемую операцию, не обязательно зная его конкретный тип.

Например:

```
for shape in shapes
print(shape.area())
\end
```

В коллекции `shapes` могут одновременно находиться окружности, прямоугольники и другие фигуры. Для каждой из них вызывается одна операция `area`, однако выполняется соответствующая конкретному типу реализация.

Выделяют несколько видов полиморфизма.

1. Полиморфизм подтипов.

Объект производного типа может использоваться там, где ожидается объект базового типа:

```
Circle <: Shape.
```

Именно этот вид полиморфизма наиболее характерен для классического ООП.

2. Параметрический полиморфизм.

Алгоритм описывается независимо от конкретного типа данных. Примером являются шаблоны или обобщённые типы:

```
List<T>
Vector<T>
max<T>(a, b)
```

3. Ad-hoc-полиморфизм.

Одна операция имеет несколько реализаций для различных типов. Типичный механизм — перегрузка функций или операторов.

Необходимо также различать:

- **перегрузку** (*overloading*);
- **переопределение** (*overriding*).

При перегрузке существуют разные функции с одинаковым именем, но различными наборами параметров.

При переопределении производный класс предоставляет новую реализацию метода, унаследованного от базового класса.

Полиморфизм подтипов часто реализуется посредством **динамического диспетчеризования**: конкретная версия метода выбирается во время выполнения программы в зависимости от фактического типа объекта.

При **статическом диспетчеризовании** вызываемая операция определяется на этапе компиляции.

44.5 Абстракция и интерфейсы

Абстракция — выделение существенных свойств объекта при игнорировании несущественных для решаемой задачи деталей.

Например, при использовании численного решателя пользователю может быть существенно, что он способен решать систему

$$Ax = b,$$

но не обязательно знать, реализован ли внутри метод Гаусса, *LU*-разложение или итерационный метод.

Абстракция позволяет работать на более высоком уровне:

```
solver.solve(A, b)
```

вместо непосредственного управления всеми деталями алгоритма.

Одним из основных средств задания абстракции является **интерфейс**.

Интерфейс определяет набор операций, которые объект обязан поддерживать, но не обязательно задаёт их конкретную реализацию.

Например:

```
interface LinearSolver {  
    solve(Matrix A, Vector b)  
}
```

После этого могут существовать реализации:

```
GaussianSolver  
LUSolver  
ConjugateGradientSolver
```

Клиентскому коду достаточно работать с интерфейсом `LinearSolver`.

Это уменьшает связность компонентов программы:

клиент $\longrightarrow$ абстракция,

а не

клиент $\longrightarrow$ конкретная реализация.

В некоторых языках существуют также **абстрактные классы**. Они занимают промежуточное положение между обычным классом и интерфейсом: могут содержать состояние и готовую реализацию части методов, но при этом не предназначены для непосредственного создания объектов.

Абстракция и инкапсуляция связаны, но обозначают разные идеи:

- абстракция отвечает на вопрос *“что умеет объект?”*;
- инкапсуляция — *“что пользователь объекта не должен знать о его внутреннем устройстве?”*.

44.6 Композиция и агрегация

При построении сложных объектов широко используется отношение **“имеет”** (*has-a*).

Композиция — отношение между объектами, при котором один объект содержит другие объекты как свои составные части и управляет их жизненным циклом.

Например:

```
class Car {  
    Engine engine  
}
```

Двигатель является составной частью автомобиля.

При строгой композиции время жизни составной части связано с временем жизни целого: уничтожение целого приводит к уничтожению его частей.

Агрегация — более слабая форма отношения “часть–целое”. Объект использует другой объект, но не обязательно владеет им и не управляет его жизненным циклом.

Например, кафедра содержит преподавателей, однако преподаватель как объект может существовать независимо от конкретной кафедры.

Условно:

композиция $\Rightarrow$ сильное владение,

агрегация $\Rightarrow$ слабая связь.

Композиция часто является более гибкой альтернативой наследованию.

Пусть требуется изменить поведение объекта. При наследовании это поведение жёстко фиксируется иерархией типов:

$$A \leftarrow B \leftarrow C.$$

При композиции объект может делегировать работу другому объекту:

$$A \longrightarrow B.$$

Такой компонент можно заменить независимо от класса владельца.

Отсюда происходит известный принцип:

предпочитать композицию наследованию, если наследование не выражает естественного отношения подтипов.

44.7 Принципы проектирования объектно-ориентированных систем

Само использование классов и наследования не гарантирует хорошую архитектуру. При проектировании ООП-систем важно стремиться к:

- высокой связности элементов внутри одного класса;
- слабой зависимости между различными классами;
- ясному распределению ответственности;
- минимизации знания одного компонента о внутреннем устройстве другого.

Одним из наиболее известных наборов рекомендаций являются **принципы SOLID**.

S — Single Responsibility Principle.

Класс должен иметь одну основную ответственность и одну основную причину для изменения.

Например, класс численного решателя не должен одновременно решать систему, сохранять результаты в базу данных и строить графики.

O — Open/Closed Principle.

Программные сущности должны быть открыты для расширения, но закрыты для модификации.

Желательно иметь возможность добавить новую реализацию алгоритма, не переписывая существующий клиентский код.

L — Liskov Substitution Principle.

Объекты производного типа должны быть корректно взаимозаменяемы с объектами базового типа.

Если

$$B <: A,$$

то программа, корректно работающая с объектом типа A , должна сохранять корректность при подстановке объекта типа B .

Нарушение этого принципа означает, что иерархия наследования выбрана неправильно.

I — Interface Segregation Principle.

Лучше использовать несколько небольших специализированных интерфейсов, чем один большой интерфейс, заставляющий класс реализовывать ненужные методы.

Например, вместо:

```
interface NumericalObject {
    solve()
    draw()
    save()
    print()
}
```

целесообразнее определить независимые интерфейсы:

Solvable
Drawable
Serializable

D — Dependency Inversion Principle.

Высокоуровневые компоненты не должны непосредственно зависеть от конкретных низкоуровневых реализаций. И те, и другие должны зависеть от абстракций.

Например, алгоритм вычислительного эксперимента должен зависеть от интерфейса

LinearSolver

а не непосредственно от класса

GaussianEliminationSolver.

Тогда конкретный решатель можно заменить без изменения логики всего приложения.

С SOLID тесно связан принцип **разделения ответственности** (*separation of concerns*): различные аспекты программы должны по возможности быть изолированы друг от друга.

Также полезен принцип минимальной информированности: объект должен знать как можно меньше о внутренней структуре других объектов. Это уменьшает связность программных компонентов.

44.8 Преимущества и ограничения объектно-ориентированного подхода

К основным преимуществам ООП относятся:

- **модульность** — программа разделяется на относительно самостоятельные компоненты;
- **инкапсуляция состояния** — локализуются изменения данных;
- **расширяемость** — новые реализации могут добавляться через существующие абстракции;
- **повторное использование кода**;
- **полиморфизм** — единый код способен работать с объектами разных типов;
- **соответствие предметной области** — многие системы удобно представлять как взаимодействие сущностей;
- **удобство сопровождения** при корректно спроектированной архитектуре.

Однако ООП не является универсальным решением.

К ограничениям относятся:

- усложнение архитектуры при чрезмерном количестве классов и уровней абстракции;
- жёсткие иерархии наследования могут затруднять дальнейшее развитие системы;
- динамический полиморфизм может создавать дополнительные накладные расходы;
- объектное представление иногда хуже соответствует задачам обработки больших однородных массивов данных;
- многочисленные мелкие объекты могут ухудшать локальность данных в памяти;
- чрезмерное применение шаблонов проектирования может приводить к неоправданному усложнению программы.

Особенно это заметно в высокопроизводительных и численных вычислениях. Например, представление миллиона частиц как миллиона сложных объектов может оказаться менее эффективным, чем хранение координат, скоростей и масс в отдельных непрерывных массивах.

Поэтому в современных программах ООП часто сочетается с другими парадигмами: процедурным, функциональным, обобщённым и data-oriented программированием.

Таким образом, центральная идея ООП состоит не в самом наличии классов, а в организации программы как системы взаимодействующих компонентов с чётко заданными ответственностями и интерфейсами.

45 Компьютерная графика. Работа с графическими библиотеками

Компьютерная графика — область информатики, изучающая методы создания, хранения, обработки и визуального представления изображений с помощью вычислительной техники.

В общем случае графическая система преобразует некоторое математическое или информационное описание сцены в изображение:

модель сцены $\longrightarrow$ геометрические преобразования $\longrightarrow$ проекция $\longrightarrow$ растеризация $\longrightarrow$ изображение.

Компьютерная графика используется не только для получения реалистичных изображений, но и для визуального анализа результатов вычислений, математического моделирования, САПР, научной визуализации и создания пользовательских интерфейсов.

45.1 Основные задачи компьютерной графики

К основным задачам компьютерной графики относятся:

- **синтез изображений** — построение изображения по математическому описанию сцены;
- **геометрическое моделирование** — описание формы и положения объектов в двумерном или трёхмерном пространстве;
- **визуализация данных** — представление численных данных в форме, удобной для анализа человеком;
- **обработка изображений** — изменение уже существующих изображений: фильтрация, масштабирование, коррекция, выделение границ и т.д.;
- **анимация** — формирование последовательности изображений, описывающей изменение сцены во времени;
- **интерактивная графика** — изменение изображения в ответ на действия пользователя;
- **фотореалистичный рендеринг** — моделирование распространения света для получения изображения, близкого к реальному.

Следует различать компьютерную графику и компьютерное зрение. В компьютерной графике обычно решается задача

описание сцены $\rightarrow$ изображение,

а в компьютерном зрении — в некотором смысле обратная задача:

изображение $\rightarrow$ информация о сцене.

Для математического моделирования особенно важна **научная визуализация**: построение графиков функций, полей, поверхностей, сеток, линий уровня, траекторий, изоповерхностей и других представлений результатов численного расчёта.

45.2 Растровая и векторная графика

Существуют два основных способа представления графической информации: растровый и векторный.

Растровая графика

Растровое изображение представляет собой прямоугольную матрицу элементов — **пикселей**:

$$I = \{c_{ij}\},$$

где c_{ij} задаёт цвет пикселя с координатами (i, j) .

Если используется цветовая модель RGB, то пиксель можно представить тройкой

$$c_{ij} = (R_{ij}, G_{ij}, B_{ij}).$$

Часто добавляется четвёртая компонента α , задающая прозрачность:

$$c_{ij} = (R, G, B, \alpha).$$

Количество битов, используемых для хранения цвета одного пикселя, называется **глубиной цвета**. Например, при использовании 8 бит на каждый канал RGB получаем

$$2^{24}$$

возможных цветов.

Основные характеристики растрового изображения:

- ширина и высота в пикселях;
- разрешение;
- глубина цвета;
- цветовая модель;
- наличие альфа-канала.

Преимуществами растровой графики являются простота представления и возможность описывать сложные изображения с большим количеством мелких деталей.

Недостатком является зависимость от разрешения: при значительном увеличении изображения становятся заметны отдельные пиксели.

Векторная графика

Векторное изображение задаётся набором геометрических примитивов:

- точек;
- отрезков;
- кривых;
- многоугольников;
- окружностей;
- текста.

Например, отрезок можно задать двумя точками

$$A = (x_1, y_1), \quad B = (x_2, y_2),$$

а окружность — центром (x_0, y_0) и радиусом r :

$$(x - x_0)^2 + (y - y_0)^2 = r^2.$$

Векторная графика не зависит непосредственно от разрешения устройства вывода. При изменении масштаба геометрические примитивы строятся заново.

Преимущества векторной графики:

- качественное масштабирование;
- компактное представление простых геометрических объектов;
- удобство редактирования отдельных элементов.

Недостатком является сложность описания фотографий и других изображений с большим количеством нерегулярных деталей.

При отображении на экран векторное изображение в конечном итоге также должно быть преобразовано в набор пикселей. Этот процесс называется **растеризацией**.

45.3 Геометрические преобразования и системы координат

Основными геометрическими преобразованиями являются:

- перенос;
- масштабирование;
- вращение;
- отражение;
- проецирование.

Для удобного представления преобразований используются **однородные координаты**. Точка трёхмерного пространства

$$(x, y, z)$$

представляется четырёхмерным вектором

$$\mathbf{p} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}.$$

Тогда любое аффинное преобразование можно записать как умножение на матрицу:

$$\mathbf{p}' = M\mathbf{p}.$$

Матрица переноса имеет вид

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Матрица масштабирования:

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Например, вращение вокруг оси z на угол φ задаётся матрицей

$$R_z = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 & 0 \\ \sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Несколько преобразований можно объединить:

$$M = M_3 M_2 M_1.$$

При этом порядок умножения матриц существенен, поскольку в общем случае

$$M_1 M_2 \neq M_2 M_1.$$

В трёхмерной графике обычно последовательно используются несколько систем координат:

1. **локальные координаты объекта** — координаты относительно самого объекта;
2. **мировые координаты** — положение объекта в общей сцене;
3. **координаты камеры** — система координат наблюдателя;
4. **координаты отсечения** (*clip coordinates*);

5. **нормализованные координаты устройства** (*NDC*);

6. **экранные координаты**.

Типичная последовательность преобразований вершины имеет вид

$$p_{\text{clip}} = PVMp_{\text{local}},$$

где

- M — матрица модели;
- V — матрица вида;
- P — матрица проекции.

Для трёхмерных сцен применяются два основных вида проекции.

При **ортографической проекции** размер объекта практически не зависит от расстояния до камеры.

При **перспективной проекции** удалённые объекты выглядят меньше. После перспективного преобразования выполняется деление на однородную координату:

$$x_{\text{NDC}} = \frac{x_c}{w_c}, \quad y_{\text{NDC}} = \frac{y_c}{w_c}, \quad z_{\text{NDC}} = \frac{z_c}{w_c}.$$

45.4 Графический конвейер

Графический конвейер (*graphics pipeline*) — последовательность этапов преобразования описания сцены в растровое изображение.

Для современного растеризационного графического API конвейер можно схематически представить следующим образом:

вершины → обработка вершин → сборка примитивов →
→ отсечение → растеризация → обработка фрагментов → кадр.

Исходная геометрия обычно представляется набором вершин. Вершина может содержать:

- координаты;
- нормаль;
- цвет;
- текстурные координаты;
- дополнительные пользовательские атрибуты.

На стадии обработки вершин координаты преобразуются из локальной системы координат в координаты отсечения. Затем вершины собираются в графические примитивы:

- точки;
- линии;
- треугольники.

Треугольник является основным примитивом трёхмерной растровой графики, поскольку:

- три точки всегда определяют плоскость;
- треугольник всегда выпуклый;
- его легко растеризовать;
- произвольную полигональную поверхность можно разбить на треугольники.

Примитивы, полностью находящиеся вне области видимости, отбрасываются. Пересекающие границу области видимости примитивы подвергаются **отсечению**.

После этого производится переход к экранным координатам и растеризация.

45.5 Растеризация и визуализация

Растеризация — преобразование геометрического примитива в множество фрагментов, соответствующих потенциальным пикселям экрана.

Например, при растеризации треугольника требуется определить множество пикселей, центры которых принадлежат его проекции.

Атрибуты вершин внутри треугольника интерполируются. Для этого удобно использовать **барицентрические координаты**:

$$p = \lambda_1 p_1 + \lambda_2 p_2 + \lambda_3 p_3,$$

где

$$\lambda_1 + \lambda_2 + \lambda_3 = 1.$$

Тогда некоторый атрибут a , заданный в вершинах, можно интерполировать:

$$a(p) = \lambda_1 a_1 + \lambda_2 a_2 + \lambda_3 a_3.$$

На стадии обработки фрагментов обычно определяются:

- цвет;
- прозрачность;
- значение глубины;
- параметры освещения;
- значения текстур.

Для удаления невидимых поверхностей используется **буфер глубины** (*Z-buffer*).

Для каждого пикселя хранится глубина ближайшего уже нарисованного фрагмента. Новый фрагмент выводится только в случае успешного теста глубины, например

$$z_{\text{new}} < z_{\text{buffer}}.$$

После этого значение глубины обновляется.

Для вычисления цвета поверхности могут использоваться модели освещения. Например, простая диффузная составляющая модели Ламберта имеет вид

$$I_d = k_d I_L \max(0, \mathbf{n} \cdot \mathbf{l}),$$

где

- $\mathbf{n}$ — единичная нормаль к поверхности;
- $\mathbf{l}$ — направление на источник света;
- I_L — интенсивность источника;
- k_d — коэффициент диффузного отражения.

В реальных системах также применяются текстурирование, смешивание цветов (*blending*), сглаживание границ (*antialiasing*), карты нормалей, теневые карты и другие методы.

Следует отличать **растеризацию** от более общего понятия **рендеринга**. Рендеринг — процесс получения изображения сцены в целом, а растеризация — лишь один из возможных способов определить вклад геометрических примитивов в пиксели.

Другим важным классом методов является **трассировка лучей**. В простейшем варианте для каждого пикселя строится луч

$$r(t) = o + td,$$

где o — положение камеры, а d — направление луча. Затем определяется ближайшее пересечение луча с объектами сцены.

Трассировка лучей естественным образом позволяет моделировать отражения, преломления и тени, но обычно требует большего объёма вычислений, чем растеризация.

45.6 Графические программные интерфейсы

Для работы с графическим оборудованием используются **графические программные интерфейсы** — API (*Application Programming Interface*).

К распространённым графическим API относятся:

- OpenGL;
- Vulkan;
- Direct3D;
- Metal;
- WebGL;
- WebGPU.

Графический API предоставляет программе средства для:

- создания графического контекста или устройства;
- передачи данных в память GPU;
- создания буферов вершин и индексов;
- загрузки текстур;
- создания и запуска шейдеров;
- настройки графического конвейера;
- формирования команд рисования;
- синхронизации CPU и GPU;
- вывода сформированного кадра на экран.

Например, геометрия может храниться в **вершинном буфере** (*vertex buffer*), а порядок соединения вершин в примитивы — в **индексном буфере**.

Если имеются вершины

$$v_0, v_1, \dots, v_{N-1},$$

то треугольники можно задавать индексами:

$$(0, 1, 2), \quad (2, 3, 0), \quad \dots$$

Это позволяет не хранить одну и ту же вершину несколько раз.

Графические API можно условно разделить на высокоуровневые и низкоуровневые.

OpenGL традиционно скрывает значительную часть управления ресурсами и синхронизацией за внутренним состоянием драйвера.

Vulkan, Direct3D 12 и Metal предоставляют программисту значительно более явный контроль над:

- памятью;
- очередями команд;
- синхронизацией;
- состоянием графического конвейера.

Это позволяет уменьшить накладные расходы и лучше использовать современное многоядерное оборудование, но усложняет программу.

Над низкоуровневыми API часто используются более высокоуровневые библиотеки и движки. Например, для научной визуализации применяются библиотеки, которые скрывают значительную часть работы с GPU и позволяют строить графики, поверхности и поля непосредственно из численных данных.

45.7 Шейдеры и программируемый конвейер

Современные GPU используют **программируемый графический конвейер**. Часть стадий обработки графики задаётся небольшими программами — **шейдерами**.

Основными типами шейдеров являются:

- вершинный шейдер;
- фрагментный (пиксельный) шейдер;
- геометрический шейдер;
- тесселяционные шейдеры;
- вычислительный шейдер.

Вершинный шейдер выполняется для каждой вершины. Одна из его основных задач — вычислить положение вершины в координатах отсечения:

$$p_{\text{clip}} = PVMp.$$

Кроме положения вершины он может преобразовывать нормали, вычислять текстурные координаты и передавать параметры следующим стадиям конвейера.

Фрагментный шейдер выполняется для фрагментов, полученных после растеризации. Обычно он вычисляет итоговый цвет:

$$C = F(\text{normal}, \text{light}, \text{texture}, \dots).$$

Например, значение цвета текстуры определяется по текстурным координатам

$$(u, v).$$

Изображение-текстура рассматривается как функция

$$T(u, v),$$

возвращающая цвет.

Шейдеры выполняются параллельно для большого количества вершин или фрагментов. Именно массовый параллелизм является одной из причин высокой эффективности GPU в задачах компьютерной графики.

Вычислительные шейдеры отличаются от обычных графических шейдеров тем, что не обязаны непосредственно участвовать в рисовании примитивов. Они позволяют использовать GPU для вычислений общего назначения, например:

- обработки изображений;
- моделирования частиц;
- вычисления физических полей;
- подготовки данных для визуализации;
- некоторых численных алгоритмов.

Поэтому современная граница между графическими вычислениями и вычислениями общего назначения на GPU является достаточно условной.

45.8 Научная визуализация и интерактивная графика

В математическом моделировании компьютерная графика является инструментом анализа результатов вычислительного эксперимента.

В зависимости от размерности и природы данных применяются различные способы визуализации.

Для скалярной функции одной переменной

$$y = f(x)$$

обычно строится график функции.

Для функции двух переменных

$$z = f(x, y)$$

можно использовать:

- поверхность;
- карту цветов;
- линии уровня.

Линией уровня является множество точек

$$f(x, y) = C.$$

Для скалярного трёхмерного поля

$$f(x, y, z)$$

используются сечения, объёмная визуализация и **изоповерхности**

$$f(x, y, z) = C.$$

Для векторного поля

$$\mathbf{v}(x, y, z)$$

могут применяться:

- стрелочные диаграммы;
- линии тока;
- траектории;
- цветовое кодирование модуля вектора.

В задачах вычислительной механики часто требуется визуализировать:

- расчётную сетку;
- поле температуры;
- поле давления;
- скорость потока;
- напряжения и деформации;
- распределение ошибок;
- движение частиц;
- изменение решения во времени.

Большое значение имеет **интерактивная визуализация**. Пользователь может изменять:

- положение камеры;
- масштаб;
- параметры отображения;
- момент времени;
- отображаемую переменную;
- порог изоповерхности;
- диапазон цветовой шкалы.

Типичный интерактивный графический цикл имеет вид:

1. обработать события ввода;
2. изменить состояние сцены;

3. выполнить необходимые вычисления;
4. сформировать новый кадр;
5. вывести его на экран.

При интерактивной визуализации важна частота обновления изображения (*frame rate*). Если за секунду выводится N кадров, то

$$FPS = N.$$

При частоте 60 FPS на построение одного кадра приходится примерно

$$\frac{1}{60} \approx 16.7 \text{ мс.}$$

Поэтому при работе с большими расчётными данными применяются:

- уменьшение объёма отображаемых данных;
- уровни детализации;
- пространственные структуры данных;
- параллельная обработка;
- вычисления на GPU;
- предварительная обработка данных.

Таким образом, графические библиотеки отделяют прикладную программу от деталей конкретного графического оборудования, а современный графический конвейер обеспечивает массово-параллельное преобразование геометрии и данных в изображение. Для задач математического моделирования компьютерная графика играет двойную роль: она используется как средство построения пользовательского интерфейса и как инструмент исследования результатов численного эксперимента.

46 Пакеты прикладных программ для математического моделирования

Пакеты прикладных программ являются одним из основных инструментов вычислительного математического моделирования. Они позволяют перейти от математической постановки задачи к её численному решению, анализу результатов и визуализации без необходимости реализовывать весь вычислительный алгоритм с нуля.

При этом использование готового пакета не устраняет необходимость понимать математическую модель и численный метод. Пользователь должен уметь оценивать применимость выбранной модели, корректность постановки задачи, свойства численного алгоритма и достоверность полученного решения.

46.1 Понятие пакета прикладных программ

Пакет прикладных программ (ППП) — совокупность программных средств, предназначенных для решения определённого класса прикладных задач и объединённых общими средствами задания исходных данных, вычислений и обработки результатов.

В математическом моделировании ППП обычно обеспечивает некоторую или всю цепочку вычислительного эксперимента:

математическая модель $\longrightarrow$ дискретная модель $\longrightarrow$ численный алгоритм $\longrightarrow$ программная реализация $\longrightarrow$ результат.

В отличие от отдельной программы, пакет обычно содержит:

- набор готовых вычислительных алгоритмов;
- средства задания параметров модели;
- средства работы с геометрией и расчётными сетками;
- библиотеки математических функций;
- средства визуализации;
- средства обмена данными;
- интерфейсы для расширения и автоматизации расчётов.

Примерами различных классов таких систем являются математические среды MATLAB, Mathematica, Maple, системы инженерного моделирования ANSYS и COMSOL, а также специализированные открытые комплексы, например OpenFOAM.

Следует различать **математическую модель**, **численный метод** и **программный пакет**. Пакет является инструментом реализации модели, но сам по себе не гарантирует правильность постановки или физическую адекватность полученного результата.

46.2 Классификация пакетов математического моделирования

Пакеты можно классифицировать по нескольким признакам.

По назначению:

1. Универсальные математические среды.

Содержат средства линейной алгебры, решения ОДУ, оптимизации, обработки данных, статистики и визуализации.

2. Системы компьютерной алгебры.

Ориентированы прежде всего на символьные преобразования: дифференцирование, интегрирование, решение уравнений, преобразование выражений, работу с рядами и специальными функциями.

3. САЕ-системы (*Computer-Aided Engineering*).

Предназначены для численного моделирования инженерных объектов: механики твёрдого тела, теплопроводности, гидродинамики, электромагнитных задач и т.д.

4. Специализированные вычислительные пакеты.

Предназначены для конкретного класса задач, например вычислительной гидродинамики, молекулярной динамики, оптимизации или обработки экспериментальных данных.

5. Библиотеки численных методов.

Не обязательно имеют графический интерфейс, а предоставляют программные реализации алгоритмов для использования в пользовательских программах.

По способу взаимодействия можно выделить:

- пакеты с графическим пользовательским интерфейсом;
- пакеты с командным интерфейсом;
- программные библиотеки;
- гибридные системы, совмещающие GUI, язык сценариев и API.

По степени открытости различают коммерческие и свободно распространяемые пакеты, а также системы с открытым и закрытым исходным кодом.

46.3 Архитектура и основные компоненты

Современный программный комплекс математического моделирования обычно имеет модульную архитектуру.

Основные компоненты можно представить в виде цепочки

препроцессор $\rightarrow$ решатель $\rightarrow$ постпроцессор.

Препроцессор используется для подготовки задачи. В нём выполняются:

- построение или импорт геометрии;
- задание физических свойств;
- выбор математической модели;
- задание начальных и граничных условий;
- построение вычислительной сетки;
- настройка параметров численного метода.

Решатель является вычислительным ядром системы. Он формирует и решает дискретную задачу.

Например, после дискретизации дифференциального уравнения может возникнуть система линейных алгебраических уравнений

$$Ax = b,$$

либо нелинейная система

$$F(x) = 0.$$

Для нестационарной задачи решатель последовательно вычисляет приближения

$$u^0, u^1, \dots, u^N,$$

соответствующие различным моментам времени.

Постпроцессор предназначен для анализа результатов:

- построения графиков;
- визуализации полей;
- вычисления интегральных характеристик;
- анализа экстремальных значений;
- сравнения нескольких расчётов;
- экспорта результатов.

Кроме этих трёх компонентов в пакет могут входить:

- база материалов и физических моделей;
- генератор расчётных сеток;
- модуль оптимизации;
- средства параллельных вычислений;
- язык сценариев;
- программный интерфейс API;
- система управления расчётными проектами.

46.4 Численные решатели

Центральной частью пакета математического моделирования являются **численные решатели**.

В зависимости от задачи могут использоваться различные классы алгоритмов.

Решение систем линейных уравнений:

- метод Гаусса;
- LU -разложение;
- метод Холецкого;
- итерационные методы;
- методы подпространств Крылова.

Для больших разреженных систем особенно важны итерационные методы и предобуславливатели.

Решение нелинейных систем

$$F(x) = 0$$

может выполняться методом Ньютона и его модификациями:

$$J(x^{(k)})\Delta x^{(k)} = -F(x^{(k)}),$$

$$x^{(k+1)} = x^{(k)} + \Delta x^{(k)},$$

где J — матрица Якоби.

Для обыкновенных дифференциальных уравнений

$$\frac{dy}{dt} = f(t, y)$$

используются, например,

- методы Эйлера;
- методы Рунге–Кутты;
- методы Адамса;
- BDF-методы.

Для уравнений в частных производных применяются:

- метод конечных разностей;
- метод конечных элементов;
- метод конечных объёмов;
- метод граничных элементов;
- спектральные методы.

Важными характеристиками решателя являются:

- порядок аппроксимации;
- устойчивость;
- скорость сходимости;
- вычислительная сложность;
- требования к памяти;
- возможность параллельного выполнения.

Пользователь должен контролировать критерии остановки итерационного процесса. Например, решение может считаться найденным при выполнении условия

$$\|r^{(k)}\| < \varepsilon,$$

где

$$r^{(k)} = b - Ax^{(k)}$$

— невязка, а ε — заданная точность.

Таким образом, получение программой некоторого числа ещё не означает получения корректного решения: необходимо контролировать сходимость, погрешность и устойчивость вычислительного процесса.

46.5 Подготовка исходных данных и постобработка

Качество результата в значительной степени определяется качеством исходных данных.

При подготовке задачи задаются:

- геометрические параметры;
- физические свойства;
- коэффициенты математической модели;
- начальные условия;
- граничные условия;
- параметры дискретизации;
- параметры численного решателя.

Для пространственных задач важным этапом является построение **вычислительной сетки**.

Сетка должна быть достаточно подробной в областях, где решение быстро изменяется, например:

- вблизи границ;
- около концентраторов напряжений;
- в пограничных слоях;
- в областях сильных градиентов температуры или скорости.

Однако чрезмерное сгущение сетки увеличивает вычислительные затраты.

Поэтому выполняют исследование **сеточной сходимости**: решают одну и ту же задачу на последовательности сеток

$$h_1 > h_2 > h_3 > \dots$$

и проверяют стабилизацию интересующей величины:

$$Q_{h_1}, Q_{h_2}, Q_{h_3}, \dots \rightarrow Q.$$

После расчёта проводится постобработка.

Для скалярного поля

$$u = u(x, y, z)$$

можно строить линии или поверхности уровня

$$u(x, y, z) = C.$$

Для векторных полей, например скорости

$$\mathbf{v} = (v_x, v_y, v_z),$$

используются стрелочные диаграммы, линии тока и распределение модуля

$$|\mathbf{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}.$$

При постобработке важно анализировать не только красивую визуализацию, но и количественные характеристики: интегралы, нормы ошибок, балансы, экстремумы и зависимости от параметров.

46.6 Форматы данных и интеграция

В реальном вычислительном проекте один программный пакет редко используется изолированно.

Необходим обмен:

- геометрическими моделями;
- расчётными сетками;
- табличными данными;
- матрицами и векторами;
- полями физических величин;
- результатами вычислительного эксперимента.

Форматы можно разделить на:

- **текстовые** — CSV, JSON, XML и другие;
- **бинарные** — эффективны для больших массивов данных;
- **специализированные научные форматы**;
- **форматы геометрии и расчётных сеток**.

Текстовые форматы просты для анализа и переноса, но при больших объёмах данных занимают много места и медленнее обрабатываются.

Бинарные форматы обеспечивают более компактное хранение и быстрый ввод-вывод, но требуют согласованного программного интерфейса.

Современные пакеты часто предоставляют **API** (*Application Programming Interface*), позволяющий управлять расчётом из внешней программы.

Например, можно автоматизировать серию вычислительных экспериментов:

$$p_1, p_2, \dots, p_N \longrightarrow Q(p_1), Q(p_2), \dots, Q(p_N),$$

где p_i — различные значения параметра модели.

Это особенно важно при:

- оптимизации;
- идентификации параметров;
- анализе чувствительности;
- обработке большого числа вариантов;
- построении суррогатных моделей.

46.7 Верификация программного комплекса

Наличие готового программного пакета не освобождает пользователя от проверки правильности вычислений.

Следует различать два понятия:

- **верификация** — проверка того, что математическая и численная задача реализована программой правильно;
- **валидация** — проверка того, что сама математическая модель достаточно хорошо описывает реальный объект.

Условно:

верификация: «правильно ли решаем уравнения?» ,

валидация: «правильные ли уравнения решаем?» .

Для верификации программного комплекса применяются следующие подходы.

1. Сравнение с аналитическим решением.

Если для тестовой задачи известно точное решение $u(x)$, вычисляют погрешность

$$e_h = u_h - u$$

и её норму

$$\|e_h\|.$$

2. Исследование сходимости.

При уменьшении шага дискретизации должно наблюдаться

$$\|u - u_h\| \rightarrow 0, \quad h \rightarrow 0.$$

Если метод имеет порядок p , ожидается поведение

$$\|u - u_h\| \approx Ch^p.$$

3. Метод изготовленных решений.

Заранее выбирается функция $u(x)$, после чего по ней вычисляется такая правая часть уравнения, чтобы выбранная функция являлась его точным решением. Затем проверяется способность программы воспроизвести это решение с ожидаемой погрешностью.

4. Сравнение с независимой программной реализацией.

Одна задача решается различными программами или различными алгоритмами.

5. Проверка законов сохранения.

Например, для замкнутой физической системы контролируются баланс массы, импульса или энергии.

6. Регрессионное тестирование.

После изменения программного кода повторяются заранее выбранные тестовые задачи и проверяется, что результаты не изменились неожиданным образом.

Кроме того, следует контролировать:

- независимость результата от сетки;
- влияние временного шага;
- величину невязки;
- чувствительность к параметрам решателя;
- корректность единиц измерения;
- выполнение начальных и граничных условий.

46.8 Критерии выбора пакета

Выбор программного пакета должен определяться прежде всего задачей, а не популярностью конкретного продукта. Основными критериями являются:

1. Соответствие математической модели.

Пакет должен поддерживать необходимые уравнения, физические модели и типы граничных условий.

2. Наличие подходящих численных методов.

Важно учитывать устойчивость, точность и применимость решателя к рассматриваемой задаче.

3. Размер задачи.

Для больших задач существенны работа с разреженными матрицами, параллельные вычисления и экономное использование памяти.

4. Возможность работы на многопроцессорных системах.

Для ресурсоёмких моделей важна масштабируемость на CPU и GPU.

5. Средства построения сеток и постобработки.

Для инженерных задач эти возможности могут быть не менее важны, чем сам численный решатель.

6. Расширяемость.

Желательна возможность добавления собственных моделей, алгоритмов и пользовательских функций.

7. Интеграция с другими программами.

Важны поддерживаемые форматы данных, API и средства автоматизации.

8. Воспроизводимость расчётов.

Необходимо иметь возможность сохранить параметры модели, сетку, настройки решателя и версию программного комплекса.

9. Лицензирование и стоимость.

Для научных и промышленных проектов условия лицензии могут существенно влиять на выбор системы.

10. Документация и поддержка.

Наличие документации, тестовых задач и активного сообщества существенно облегчает проверку и сопровождение модели.

Таким образом, пакет прикладных программ следует рассматривать как **инструмент вычислительного эксперимента**. Его основная ценность состоит не просто в возможности получить численный результат, а в обеспечении полного цикла:

постановка задачи → дискретизация → решение → анализ → проверка результата
--

При этом ответственность за выбор модели, численного метода и интерпретацию результатов остаётся за исследователем.

47 Представление о языках программирования высокого уровня

Язык программирования — формальная знаковая система, предназначенная для описания алгоритмов и данных в форме, пригодной для исполнения вычислительной системой.

Языки программирования высокого уровня предоставляют программисту абстракции, которые позволяют описывать решение задачи преимущественно в терминах алгоритмов, структур данных и предметной области, а не конкретных команд процессора.

К языкам высокого уровня относятся, например, C, C++, Java, Python, C#, Fortran и многие другие. Граница между языками высокого и низкого уровня условна: разные языки предоставляют различную степень абстракции от аппаратного обеспечения.

47.1 Уровни языков программирования

Наиболее низким уровнем представления программы является **машинный код**. Он состоит из инструкций, непосредственно исполняемых процессором. Набор инструкций определяется архитектурой процессора (ISA — Instruction Set Architecture).

Следующим уровнем является **язык ассемблера**. Его команды обычно практически однозначно соответствуют машинным инструкциям, но вместо числовых кодов используются мнемонические обозначения:

```
mov eax, 5
add eax, 3
```

Ассемблер остаётся тесно связанным с конкретной архитектурой процессора.

Языки высокого уровня используют более сложные конструкции: переменные, выражения, функции, классы, контейнеры, исключения, модули и т.д. Например,

```
for (int i = 0; i < n; ++i)
    sum += a[i];
```

одна конструкция высокого уровня может транслироваться в большое число машинных инструкций. Основные преимущества языков высокого уровня:

- независимость программы от деталей конкретного процессора;
- более высокая выразительность;
- сокращение объёма исходного кода;
- повышение читаемости и сопровождаемости программ;
- наличие развитых систем типов, библиотек и средств абстракции;
- возможность автоматического управления частью ресурсов программы.

Однако повышение уровня абстракции может сопровождаться уменьшением прямого контроля над аппаратными ресурсами и дополнительными накладными расходами.

Следует отметить, что деление языков на “низкоуровневые” и “высокоуровневые” не является строгой классификацией. Например, язык C обычно относят к языкам высокого уровня, однако он предоставляет прямую работу с адресами памяти и позволяет достаточно точно контролировать представление данных.

47.2 Синтаксис и семантика

Любой язык программирования характеризуется прежде всего его **синтаксисом** и **семантикой**.

Синтаксис — совокупность правил, определяющих допустимую структуру программ на данном языке.

Например, конструкция

```
x = a + b;
```

является синтаксически корректным оператором во многих C-подобных языках.

Синтаксис языка можно формально задавать с помощью грамматик. Часто применяют контекстно-свободные грамматики и формулу Бэкуса–Наура.

Упрощённое грамматическое правило для арифметического выражения может иметь вид

$$\langle expr \rangle ::= \langle expr \rangle + \langle term \rangle \mid \langle term \rangle.$$

При обработке программы исходный текст сначала разбивается на **лексемы**, после чего на основе грамматики строится его синтаксическая структура, например абстрактное синтаксическое дерево (AST — Abstract Syntax Tree).

Семантика определяет смысл синтаксически корректных конструкций: какие действия должна выполнять программа и как изменяется её состояние.

Выделяют, например:

- операционную семантику — смысл конструкции задаётся через последовательность вычислительных действий;
- денотационную семантику — конструкции сопоставляются некоторым математическим объектам;
- аксиоматическую семантику — поведение программы описывается логическими утверждениями о её состоянии.

Таким образом, синтаксически корректная программа не обязательно является семантически корректной. Например, обращение к несуществующей переменной может соответствовать некоторой грамматической конструкции, но нарушать правила семантики языка.

47.3 Типы данных и системы типов

Тип данных определяет множество допустимых значений и операций, которые можно выполнять над этими значениями.

К базовым типам относятся:

- целые числа;
- вещественные числа;
- логические значения;
- символы;
- строки.

На их основе строятся составные типы:

- массивы;
- структуры и записи;
- кортежи;
- классы;
- перечисления;
- указатели и ссылки;
- контейнеры и другие параметризованные типы.

Система типов — совокупность правил, определяющих типы выражений, переменных и операций и допустимость их совместного использования.

По моменту проверки типов различают:

1. **статическую типизацию** — основные проверки выполняются до запуска программы;
2. **динамическую типизацию** — тип значения определяется и проверяется во время исполнения программы.

Например, C++, Java и Fortran в основном используют статическую типизацию, а Python — динамическую.

При статической типизации компилятор может обнаружить многие ошибки заранее:

```
int x = "hello";    // ошибка типов
```

В языках с динамической типизацией переменная обычно связывается не с одним фиксированным типом, а со значениями, тип которых определяется во время исполнения.

Отдельно говорят о **сильной** и **слабой** типизации. Эти термины не имеют абсолютно строгого универсального определения, но обычно отражают степень допустимости неявных преобразований между различными типами.

Многие современные языки поддерживают **параметрический полиморфизм** — возможность описывать алгоритмы независимо от конкретного типа данных. Например, шаблоны в C++ или обобщённые типы (generics) в Java.

Система типов выполняет несколько функций:

- обнаружение ошибок;
- документирование интерфейсов;
- ограничение множества допустимых операций;
- возможность оптимизации программы;
- повышение надёжности программного обеспечения.

47.4 Основные парадигмы программирования

Парадигма программирования — совокупность принципов и способов организации вычислений и структуры программы.

Один язык может поддерживать несколько парадигм.

Императивное программирование

Программа рассматривается как последовательность команд, изменяющих состояние вычислительной системы.

Основными средствами являются:

- присваивание;
- условные операторы;
- циклы;
- последовательное выполнение команд.

Большинство традиционных языков поддерживает императивный стиль.

Процедурное программирование

Является развитием императивного подхода. Программа разбивается на процедуры и функции, каждая из которых решает некоторую подзадачу.

Основная идея — структурная декомпозиция алгоритма.

Классическими примерами являются C, Pascal и Fortran.

Объектно-ориентированное программирование

Основной единицей программы является объект, объединяющий данные и операции над ними.

К основным понятиям ООП относятся:

- класс;
- объект;
- инкапсуляция;
- наследование;
- полиморфизм.

ООП особенно удобно для построения больших программных систем с большим числом взаимодействующих сущностей.

Функциональное программирование

В функциональном подходе вычисление рассматривается прежде всего как вычисление значений функций.

Характерными особенностями являются:

- функции как объекты первого класса;
- функции высших порядков;
- предпочтение неизменяемых данных;
- ограничение побочных эффектов;
- широкое использование рекурсии и композиции функций.

Функциональные возможности присутствуют не только в специализированных языках, но и в C++, Python, Java и других универсальных языках.

Декларативное программирование

Программист описывает прежде всего *что* требуется получить, а не точную последовательность действий, необходимых для получения результата.

Примером декларативного языка является SQL:

```
SELECT name FROM students WHERE grade > 4;
```

Пользователь задаёт требуемое множество данных, а способ выполнения запроса выбирается системой управления базами данных.

47.5 Компиляция и интерпретация

Программа, написанная на языке высокого уровня, в конечном счёте должна быть преобразована в форму, которую способна выполнить вычислительная система.

Компиляция — преобразование исходной программы в другую эквивалентную программу, обычно в машинный код или некоторое промежуточное представление.

Упрощённо процесс компиляции можно представить как

исходный код → лексический анализ → синтаксический анализ → семантический анализ → промежуточное представление

После компиляции отдельных модулей обычно выполняется **компоновка** (linking), при которой объектные файлы и библиотеки объединяются в исполняемый файл.

Интерпретация — выполнение программы другим программным средством — интерпретатором, который анализирует программу и непосредственно осуществляет заданные ею действия.

Однако противопоставление “компилируемый язык — интерпретируемый язык” является упрощением. Компиляция и интерпретация являются свойствами *реализации языка*, а не обязательно самого языка.

Например, распространена схема

исходный код → байт-код → виртуальная машина.

Кроме того, используется **JIT-компиляция** (Just-In-Time compilation), при которой промежуточный код преобразуется в машинный непосредственно во время работы программы.

Таким образом, современные реализации могут сочетать компиляцию, интерпретацию и динамическую компиляцию.

47.6 Организация памяти и выполнение программы

При выполнении программы её адресное пространство условно можно разделить на несколько областей:

- область машинного кода;
- область глобальных и статических данных;
- стек;
- динамическая память, или куча.

Стек вызовов используется для хранения информации о вызванных функциях: параметров, локальных переменных, адресов возврата и служебной информации.

При вызове функции обычно создаётся новый **кадр стека**. После завершения функции соответствующий кадр удаляется.

Преимуществом стека является быстрое выделение и освобождение памяти. Недостатком — ограниченный размер и время жизни объектов, обычно связанное со временем выполнения функции.

Куча используется для динамического выделения памяти, когда объём или время жизни объекта заранее неизвестны.

В языках с ручным управлением памятью программист явно управляет её выделением и освобождением. Ошибки могут приводить к:

- утечкам памяти;
- обращению к уже освобождённой памяти;

- двойному освобождению;
- повреждению памяти.

Для уменьшения числа подобных ошибок используются различные механизмы.

В C++ широко применяется принцип **RAII** (Resource Acquisition Is Initialization): время жизни ресурса связывается со временем жизни объекта.

Другой подход — **автоматическая сборка мусора** (garbage collection). Среда выполнения периодически определяет объекты, которые больше недостижимы из программы, и освобождает занимаемую ими память.

Сборка мусора применяется, например, во многих реализациях Java, C# и Python.

Выполнение программы также включает работу с:

- стеком вызовов;
- обработкой исключений;
- вводом-выводом;
- потоками выполнения;
- системными вызовами;
- динамическими библиотеками.

Таким образом, язык высокого уровня скрывает значительную часть деталей исполнения, однако понимание организации памяти остаётся важным для оценки производительности и корректности программы.

47.7 Библиотеки и экосистема языка

Практическая ценность языка определяется не только его синтаксисом, но и доступной программной инфраструктурой.

Стандартная библиотека предоставляет базовые средства, которые считаются частью программной платформы языка:

- контейнеры и структуры данных;
- математические функции;
- работу со строками;
- файловый ввод-вывод;
- многопоточность;
- сетевые средства;
- алгоритмы.

Дополнительные библиотеки позволяют использовать уже реализованные алгоритмы и программные компоненты, не реализуя их заново.

Для математического моделирования особенно важны библиотеки:

- линейной алгебры;
- численного интегрирования;
- решения ОДУ и УЧП;
- оптимизации;
- статистической обработки;
- визуализации;
- параллельных вычислений.

Совокупность библиотек, систем сборки, менеджеров пакетов, средств тестирования, профилирования, отладки, документации и пользовательского сообщества обычно называют **экосистемой языка**.

Для реальной разработки зрелость экосистемы нередко оказывается не менее важной, чем свойства самого языка.

47.8 Критерии выбора языка программирования

Не существует языка, оптимального для всех задач. Выбор определяется требованиями конкретного проекта. Основными критериями являются:

1. Соответствие предметной области.

Например, при научных расчётах существенно наличие эффективных библиотек линейной алгебры, решения дифференциальных уравнений и визуализации.

2. Производительность.

Для вычислительно трудоёмких задач важны скорость исполнения, возможность векторизации, параллелизма и эффективной работы с памятью.

3. Уровень контроля над ресурсами.

Системное и высокопроизводительное программирование зачастую требует непосредственного управления памятью и представлением данных.

4. Скорость разработки.

Высокоуровневые динамические языки часто позволяют быстрее создавать и изменять программы за счёт более компактного кода и развитых библиотек.

5. Надёжность и безопасность.

Существенны система типов, средства контроля памяти, обработка ошибок и возможности статического анализа.

6. Переносимость.

Важно, на каких операционных системах и архитектурах может выполняться программа.

7. Наличие библиотек и инструментов.

Иногда именно наличие требуемого численного или прикладного пакета фактически определяет выбор языка.

8. Совместимость с существующим программным обеспечением.

Для крупных научных комплексов важна возможность использовать уже существующий код, написанный на других языках.

9. Поддержка параллельных вычислений.

Для математического моделирования существенна возможность использовать многопоточность, распределённые вычисления, MPI, OpenMP и ускорители.

10. Квалификация команды и сопровождаемость кода.

Теоретически наиболее подходящий язык не всегда является практически оптимальным, если коллектив не располагает необходимой экспертизой.

На практике часто применяется не один язык, а их комбинация. Например, высокоуровневый язык может использоваться для подготовки данных, управления вычислительным экспериментом и визуализации, а наиболее вычислительно затратные части программы реализуются на компилируемом языке.

Таким образом, язык программирования высокого уровня представляет собой средство абстрагирования от аппаратной реализации вычислений. Его основные характеристики включают систему типов, модель выполнения, поддерживаемые парадигмы, механизмы управления памятью и программную экосистему. Выбор языка должен определяться требованиями задачи, а не только удобством его синтаксиса.

48 Суперкомпьютерные технологии

Суперкомпьютерными технологиями называют совокупность аппаратных и программных средств, предназначенных для решения вычислительно сложных задач с использованием массового параллелизма.

Типичные области применения суперкомпьютеров: численное моделирование физических процессов, механика сплошных сред, климатическое моделирование, молекулярная динамика, обработка больших объёмов данных, машинное обучение и решение больших задач линейной алгебры.

Главной особенностью высокопроизводительных вычислений является то, что производительность определяется не только быстродействием отдельного процессора, но и возможностью эффективно использовать большое число вычислительных устройств, организовать обмен данными между ними и обеспечить достаточную пропускную способность памяти.

48.1 Архитектура высокопроизводительных вычислительных систем

Современный суперкомпьютер, как правило, представляет собой совокупность вычислительных узлов, соединённых высокоскоростной сетью.

Типичный вычислительный узел содержит:

- один или несколько многоядерных центральных процессоров (CPU);
- оперативную память;
- локальные уровни кэш-памяти;
- сетевой адаптер;
- иногда один или несколько графических ускорителей (GPU).

Таким образом, современная суперкомпьютерная система обычно является **гетерогенной**: разные типы вычислительных устройств предназначены для различных видов нагрузки.

По характеру организации памяти выделяют две принципиальные архитектуры.

Системы с общей памятью имеют единое адресное пространство: все процессоры или ядра могут обращаться к одним и тем же данным.

Системы с распределённой памятью состоят из отдельных узлов, каждый из которых имеет собственную оперативную память. Обмен между узлами осуществляется посредством передачи сообщений.

На практике распространены гибридные системы: внутри одного узла имеется общая память, а между узлами память распределена.

Для классификации архитектур также часто используется классификация Флинна:

- SISD — один поток команд, один поток данных;
- SIMD — один поток команд, множество потоков данных;
- MISD — множество потоков команд, один поток данных;
- MIMD — множество потоков команд, множество потоков данных.

Современные суперкомпьютеры в целом относятся к классу MIMD, однако внутри процессоров и ускорителей широко применяется SIMD- или SIMT-параллелизм.

Важнейшей частью суперкомпьютера является сеть межсоединений. Для неё существенны:

- латентность передачи сообщения;
- пропускная способность;
- топология сети;
- способность одновременно обслуживать большое количество обменов.

При масштабировании задачи на тысячи вычислительных узлов время коммуникаций может стать сопоставимым или даже превышать время собственно вычислений.

48.2 Параллелизм и уровни параллельной обработки

Параллелизм означает одновременное выполнение нескольких операций.

Можно выделить несколько уровней параллелизма.

Параллелизм на уровне инструкций реализуется непосредственно процессором: конвейеризация, суперскалярное исполнение, внеочередное выполнение.

Векторный параллелизм состоит в выполнении одной операции сразу над несколькими элементами данных. Для этого используются SIMD-инструкции.

Например, операция

$$c_i = a_i + b_i$$

может выполняться одновременно для нескольких значений i .

Параллелизм на уровне потоков реализуется на многоядерных CPU. Различные потоки программы выполняются различными ядрами.

Параллелизм на уровне процессов используется прежде всего в системах с распределённой памятью. Каждый процесс имеет собственное адресное пространство и взаимодействует с другими процессами посредством сообщений.

Массовый параллелизм характерен для GPU, где одновременно могут выполняться тысячи лёгких вычислительных потоков.

Таким образом, одна программа может одновременно использовать несколько уровней: например,

$$\text{MPI} + \text{OpenMP} + \text{SIMD},$$

или

$$\text{MPI} + \text{GPU}.$$

Такой подход называют **гибридным параллельным программированием**.

48.3 Иерархия памяти

Скорость современных процессоров существенно выше скорости доступа к основной памяти. Поэтому память организуется иерархически.

Типичная иерархия имеет вид:

регистры $\rightarrow L1 \rightarrow L2 \rightarrow L3 \rightarrow$ оперативная память $\rightarrow$ локальное хранилище $\rightarrow$ параллельная файловая система.

Чем ближе уровень памяти к процессору, тем меньше его объём, но тем ниже латентность и выше скорость доступа.

Кэш-память автоматически сохраняет недавно использованные участки основной памяти. Поэтому пространственная и временная локальность обращений к данным существенно влияет на производительность программы.

Например, последовательная обработка элементов массива обычно значительно эффективнее случайного доступа к памяти.

В многопроцессорных узлах распространена архитектура NUMA (*Non-Uniform Memory Access*). Физически память распределена между процессорными сокетами, хотя логически образует единое адресное пространство. Доступ ядра к локальной памяти обычно быстрее доступа к памяти другого сокета.

Поэтому для НРС важны:

- локальность данных;
- привязка потоков к ядрам;
- размещение данных в памяти нужного NUMA-узла;
- уменьшение количества обращений к памяти;
- эффективное использование кэш-памяти.

Для оценки вычислительного алгоритма часто используют **арифметическую интенсивность**

$$I = \frac{\text{число арифметических операций}}{\text{объём переданных данных}}.$$

При малой арифметической интенсивности производительность ограничивается пропускной способностью памяти, а при большой — производительностью вычислительных устройств.

В модели Roofline это записывается как

$$P = \min(P_{\max}, B_{\max}I),$$

где $P_{\max}$ — пиковая вычислительная производительность, $B_{\max}$ — максимальная пропускная способность памяти.

48.4 Распределённая память и технология MPI

В системе с распределённой памятью каждый вычислительный процесс имеет собственное адресное пространство. Непосредственно обратиться к данным другого процесса нельзя.

Для взаимодействия используется передача сообщений.

Наиболее распространённым стандартом является **MPI** (*Message Passing Interface*).

При запуске MPI-программы создаётся множество процессов. Каждому процессу присваивается уникальный номер — **rank**.

Если используется p процессов, то

$$\text{rank} \in \{0, 1, \dots, p-1\}.$$

Процессы объединяются в группы, называемые **коммуникаторами**. Основным коммуникатором является `MPI_COMM_WORLD`. MPI предоставляет два основных типа обменов.

Двухточечные операции передают данные непосредственно между двумя процессами:

- `MPI_Send`;
- `MPI_Recv`;
- неблокирующие `MPI_Isend` и `MPI_Irecv`.

Неблокирующие операции позволяют совмещать вычисления и коммуникации.

Коллективные операции выполняются группой процессов:

- `MPI_Bcast` — широковещательная рассылка;
- `MPI_Scatter` — распределение данных;
- `MPI_Gather` — сбор данных;
- `MPI_Reduce` — редукция;
- `MPI_Allreduce` — редукция с передачей результата всем процессам;
- `MPI_Barrier` — синхронизация.

Например, при вычислении суммы

$$S = \sum_{i=1}^N a_i$$

массив можно разделить между p процессами. Каждый процесс вычисляет локальную сумму

$$S_k = \sum_{i \in \Omega_k} a_i,$$

после чего глобальный результат вычисляется коллективной операцией редукции:

$$S = \sum_{k=0}^{p-1} S_k.$$

Основные накладные расходы MPI связаны с передачей сообщений. Упрощённо время передачи сообщения размера m можно представить как

$$T_{\text{comm}} = \alpha + \beta m,$$

где α характеризует латентность сети, а β — время передачи единицы данных.

Следовательно, большое число маленьких сообщений часто менее эффективно, чем меньшее число сообщений большего размера.

48.5 Общая память и технология OpenMP

При общей памяти все потоки работают с одним адресным пространством.

Одним из основных средств программирования таких систем является **OpenMP**.

OpenMP реализует модель **fork-join**.

Последовательная программа начинает выполняться одним главным потоком. При входе в параллельную область создаётся группа потоков:

$$1 \longrightarrow p.$$

После завершения параллельной области потоки синхронизируются и выполнение снова продолжается одним потоком:

$$p \longrightarrow 1.$$

В языках C и C++ параллельные области задаются директивами вида `#pragma omp ....`

Основными конструкциями являются:

- **parallel** — создание параллельной области;
- **for** — распределение итераций цикла между потоками;
- **sections** — выполнение разных участков кода разными потоками;
- **reduction** — параллельное выполнение редукции;
- **critical** — критическая секция;
- **atomic** — атомарная операция;
- **barrier** — барьерная синхронизация.

Переменные могут быть:

- **shared** — общими для потоков;
- **private** — локальными для каждого потока.

Основная проблема программирования с общей памятью — **гонка данных** (*race condition*).

Она возникает, когда несколько потоков одновременно обращаются к одной области памяти, причём хотя бы один поток изменяет её, а порядок операций не определён.

Использование синхронизации устраняет гонки, но чрезмерная синхронизация снижает параллельную производительность.

Другой важный эффект — **ложное совместное использование** (*false sharing*). Он возникает, когда разные потоки изменяют разные переменные, расположенные в одной линии кэша. Несмотря на логическую независимость данных, между кэшами возникает интенсивный обмен.

48.6 Вычисления на графических процессорах

GPU первоначально создавались для компьютерной графики, однако их архитектура хорошо подходит для массово-параллельных вычислений.

CPU содержит сравнительно небольшое число сложных вычислительных ядер, оптимизированных для последовательного исполнения и сложного управления.

GPU содержит большое число более простых вычислительных устройств и ориентирован на выполнение одной и той же последовательности операций над большим количеством данных.

Для GPU характерна модель SIMT (*Single Instruction, Multiple Threads*).

Потоки группируются в аппаратные группы — например, *warp*. Потоки одной такой группы исполняют одну инструкцию одновременно.

Поэтому условные переходы вида

```
if (condition)
```

могут снижать эффективность, если разные потоки одной группы выбирают разные ветви. Этот эффект называется **дивергенцией потоков**.

Память GPU также имеет иерархическую структуру:

- регистры;

- локальная и разделяемая память вычислительного блока;
- кэш;
- глобальная память GPU.

Глобальная память имеет большую пропускную способность, но сравнительно высокую латентность. Поэтому важным условием эффективных GPU-вычислений является объединение обращений соседних потоков к соседним участкам памяти.

Распространёнными технологиями программирования GPU являются CUDA, OpenCL и модели директивного программирования.

Для гибридного суперкомпьютера типична схема

MPI между узлами + GPU внутри узла.

Особое значение имеет стоимость передачи данных между CPU и GPU. Если вычислений над данными мало, время копирования может превысить выигрыш от использования ускорителя.

GPU особенно эффективны для задач с:

- большим числом однотипных операций;
- высокой степенью параллелизма;
- высокой арифметической интенсивностью;
- регулярным доступом к памяти.

48.7 Декомпозиция вычислительной задачи

Для выполнения задачи на параллельной системе её необходимо разбить на части — выполнить **декомпозицию**.

Выделяют несколько основных подходов.

Декомпозиция по данным означает разделение исходных данных между вычислителями.

Например, область

Ω

при решении дифференциального уравнения можно представить как

$$\Omega = \bigcup_{k=0}^{p-1} \Omega_k.$$

Каждый процесс обрабатывает собственную подобласть Ω_k . Значения на границах подобластей периодически передаются соседним процессам.

Такой подход широко применяется в методах конечных разностей, конечных объёмов и конечных элементов.

Функциональная декомпозиция состоит в том, что различным вычислителям назначаются разные операции или этапы алгоритма.

Задачный параллелизм используется, если вычисление можно представить как набор независимых или частично зависимых задач.

К качеству декомпозиции предъявляются несколько требований.

Балансировка нагрузки: вычислительная работа должна распределяться примерно равномерно:

$$W_1 \approx W_2 \approx \dots \approx W_p.$$

Если один процесс работает значительно дольше остальных, остальные процессы будут простаивать в ожидании.

Минимизация коммуникаций: следует уменьшать объём и число обменов.

Достаточная гранулярность: вычислительная работа между обменами должна быть достаточно большой по сравнению со стоимостью коммуникаций.

Условно эффективный параллельный алгоритм должен удовлетворять

$$T_{\text{calc}} \gg T_{\text{comm}}.$$

В реальных программах также стремятся совмещать коммуникации и вычисления: пока часть данных передаётся по сети, процессор выполняет операции над уже доступными данными.

48.8 Производительность, ускорение и масштабируемость

Производительность суперкомпьютеров часто измеряется в FLOPS — числе операций с плавающей точкой в секунду. Используются единицы:

$$1 \text{ GFLOPS} = 10^9 \text{ FLOPS},$$

$$1 \text{ TFLOPS} = 10^{12} \text{ FLOPS},$$

$$1 \text{ PFLOPS} = 10^{15} \text{ FLOPS},$$

$$1 \text{ EFLOPS} = 10^{18} \text{ FLOPS}.$$

При этом пиковая производительность оборудования и производительность реальной прикладной программы могут различаться в несколько раз.

Пусть T_1 — время выполнения задачи одним вычислителем, а T_p — время выполнения на p вычислителях.

Ускорением называется

$$S_p = \frac{T_1}{T_p}.$$

В идеальном случае

$$S_p = p.$$

Параллельная эффективность определяется как

$$E_p = \frac{S_p}{p} = \frac{T_1}{pT_p}.$$

Идеальное значение:

$$E_p = 1.$$

На практике

$$E_p < 1$$

из-за коммуникаций, синхронизаций, дисбаланса нагрузки и последовательных участков программы.

Пусть доля программы, допускающая параллельное исполнение, равна α , а последовательная доля равна $1 - \alpha$.

Тогда согласно **закону Амдала**

$$S_p = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}.$$

При $p \rightarrow \infty$

$$S_{\max} = \frac{1}{1 - \alpha}.$$

Например, если 5% программы остаётся последовательным, то максимальное ускорение независимо от числа процессоров не превышает

$$S_{\max} = \frac{1}{0.05} = 20.$$

Закон Амдала относится прежде всего к задаче фиксированного размера.

Однако при увеличении числа процессоров часто увеличивают и размер задачи. Этот случай описывает **закон Густафсона**.

Если s — доля последовательного времени при выполнении задачи на p процессорах, то масштабированное ускорение оценивается как

$$S_p^{(G)} = p - s(p - 1).$$

Это показывает, что большое число процессоров эффективно использовать для решения всё более крупных задач. Различают два основных типа масштабируемости.

Сильная масштабируемость исследуется при фиксированном размере задачи:

$$N = \text{const}, \quad p \uparrow.$$

Хорошая сильная масштабируемость означает уменьшение T_p при увеличении p .

Слабая масштабируемость исследуется при примерно постоянной нагрузке на один процесс:

$$\frac{N}{p} \approx \text{const.}$$

Идеальная слабая масштабируемость означает

$$T_p \approx \text{const}$$

при одновременном увеличении N и p .

С ростом числа процессов масштабируемость обычно ухудшается из-за:

- увеличения числа коммуникаций;
- роста стоимости коллективных операций;
- синхронизаций;
- дисбаланса нагрузки;
- недостаточного объёма работы на один процесс.

Поэтому использование большего числа процессоров не гарантирует уменьшения времени расчёта.

48.9 Отказоустойчивость и воспроизводимость

При использовании нескольких вычислительных узлов вероятность аппаратного или программного отказа возрастает с увеличением масштаба системы.

Если расчёт продолжается часы или дни, потеря всей выполненной работы при отказе одного узла становится неприемлемой.

Основным методом защиты длительных вычислений является **checkpoint/restart**.

Через определённые интервалы времени программа сохраняет своё состояние:

$$X(t_k) \longrightarrow \text{checkpoint.}$$

После отказа вычисление продолжается не с начального состояния, а с последней контрольной точки.

Другими методами обеспечения отказоустойчивости являются:

- резервирование данных;
- репликация вычислений;
- перераспределение работы после отказа процесса;
- применение отказоустойчивых файловых систем;
- алгоритмические методы восстановления данных.

Отдельной проблемой является **воспроизводимость** параллельных вычислений.

Операции с плавающей точкой в общем случае не ассоциативны:

$$(a + b) + c \neq a + (b + c).$$

В последовательной программе порядок суммирования обычно фиксирован. В параллельной редукции порядок операций может зависеть от числа процессов, реализации библиотеки или порядка прихода сообщений.

Поэтому два математически эквивалентных запуска могут дать немного различные результаты.

Для повышения воспроизводимости следует:

- фиксировать версии компилятора и библиотек;
- сохранять параметры компиляции;
- фиксировать число процессов и потоков;
- фиксировать начальные значения генераторов случайных чисел;
- сохранять входные данные и параметры расчёта;
- контролировать порядок редукций, если требуется побитовая воспроизводимость;

- использовать системы контроля версий;
- сохранять метаданные вычислительного эксперимента.

Таким образом, эффективность суперкомпьютерного расчёта определяется не столько самим наличием большого числа вычислителей, сколько согласованностью всех компонентов:

алгоритм + декомпозиция + вычислительная архитектура + память + коммуникации
--

Хороший параллельный алгоритм должен обеспечивать достаточный параллелизм, равномерную загрузку вычислителей, локальность данных и относительно малые накладные расходы на обмены и синхронизацию.

49 Основные понятия и принципы математического моделирования. Прямые и обратные задачи математического моделирования. Универсальность математических моделей. Принцип аналогий. Иерархия моделей

49.1 Основные понятия математического моделирования

Модель — это объект, система или формальное описание, которое замещает исследуемый объект и воспроизводит существенные для поставленной задачи свойства этого объекта.

Математическая модель — описание объекта или процесса с помощью математических понятий и соотношений: уравнений, неравенств, функционалов, алгоритмов, вероятностных законов и т. д.

В общем виде математическую модель можно представить как систему

$$\mathcal{M} = (\mathbf{x}, \mathbf{p}, \mathbf{u}, \mathcal{F}, \mathcal{B}),$$

где

- $\mathbf{x}$ — независимые переменные, например координаты и время;
- $\mathbf{u}$ — искомые переменные, описывающие состояние системы;
- $\mathbf{p}$ — параметры модели;
- $\mathcal{F}$ — математические соотношения, описывающие поведение системы;
- $\mathcal{B}$ — начальные, граничные и другие дополнительные условия.

Например, процесс теплопроводности может описываться уравнением

$$\rho c \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + q,$$

где T — температура, ρ — плотность, c — теплоёмкость, k — коэффициент теплопроводности, q — объёмный источник тепла.

Математическое моделирование представляет собой исследование объекта не непосредственно, а посредством построения и анализа его математической модели.

Основными целями моделирования являются:

- объяснение поведения системы;
- прогнозирование её состояния;
- определение неизвестных параметров;
- управление системой;
- оптимизация её характеристик;
- проведение вычислительного эксперимента вместо дорогостоящего, опасного или невозможного натурального эксперимента.

При построении модели обычно используются следующие основные принципы.

1. **Абстрагирование.** В модель включаются только свойства объекта, существенные с точки зрения поставленной задачи.
2. **Адекватность.** Модель должна с требуемой точностью воспроизводить свойства реального объекта в рассматриваемом диапазоне условий.
3. **Простота.** Не следует вводить в модель сложность, которая не улучшает решение поставленной задачи.
4. **Системность.** Объект рассматривается как совокупность взаимосвязанных элементов.
5. **Иерархичность.** Для одного объекта могут строиться модели различной сложности и детализации.
6. **Проверяемость.** Результаты модели должны допускать сравнение с экспериментом, аналитическим решением или результатами других моделей.

49.2 Объект, модель и цель моделирования

Объект моделирования — реальная или абстрактная система, процесс или явление, свойства которого требуется исследовать.

Перед построением модели необходимо определить **цель моделирования**. От цели зависит, какие свойства объекта считаются существенными.

Например, автомобиль может моделироваться:

- как материальная точка — при исследовании движения по трассе;
- как твёрдое тело — при исследовании динамики;
- как система упругих тел — при расчёте прочности;
- как трёхмерное тело в потоке газа — при исследовании аэродинамики.

Таким образом, одному объекту соответствует не одна “истинная” модель, а множество моделей. Удобно выделять три группы величин:

- **входные воздействия f** ;
- **параметры модели p** ;
- **состояние системы u** .

Тогда модель можно символически записать как

$$A(u, p) = f.$$

Наблюдаемые величины могут быть связаны с состоянием системой наблюдения

$$y = H(u).$$

Переход от реального объекта к математической модели всегда связан с идеализацией. Например, могут использоваться предположения:

- абсолютно твёрдое тело;
- несжимаемая жидкость;
- сплошная среда;
- идеальный газ;
- точечная масса;
- отсутствие трения;
- линейность зависимости между величинами.

Каждое такое предположение ограничивает область применимости модели.

49.3 Этапы построения математической модели

Типичная схема математического моделирования включает следующие этапы.

1. Постановка задачи.

Определяются объект, цель исследования, искомые величины и требуемая точность.

2. Выделение существенных свойств объекта.

Формулируются основные предположения и упрощения.

3. Построение концептуальной модели.

Определяются элементы системы и связи между ними.

4. Математическая формализация.

Законы и связи записываются в виде математических соотношений:

$$\mathcal{A}(\mathbf{u}, \mathbf{p}) = \mathbf{f}.$$

В физических задачах основой модели часто являются фундаментальные законы сохранения:

$$\text{изменение запаса} = \text{приход} - \text{расход} + \text{источники}.$$

5. Задание начальных и граничных условий.

Одних дифференциальных уравнений обычно недостаточно для однозначного определения решения.

6. Определение параметров модели.

Они могут быть известны из эксперимента, справочных данных либо определяться в результате решения обратной задачи.

7. Исследование математической постановки.

Проверяются существование, единственность и устойчивость решения.

8. Выбор метода решения.

Если аналитическое решение невозможно, строится численный метод.

9. Реализация алгоритма в виде программы.

Таким образом возникает классическая триада

$$\boxed{\text{модель} \longrightarrow \text{алгоритм} \longrightarrow \text{программа}}.$$

10. Верификация и валидация.

Проверяются правильность вычислительной реализации и соответствие модели реальному объекту.

11. Вычислительный эксперимент.

Выполняется серия расчётов при различных значениях параметров и условий.

12. Анализ результатов и уточнение модели.

Процесс моделирования обычно является циклическим:

$$\text{объект} \rightarrow \text{модель} \rightarrow \text{расчёт} \rightarrow \text{сравнение с данными} \rightarrow \text{уточнение модели}.$$

49.4 Прямые задачи

Прямой задачей математического моделирования называется задача, в которой известны математическая модель, её параметры, начальные и граничные условия, а требуется определить состояние системы.

В операторной форме:

$$\mathcal{A}(\mathbf{u}, \mathbf{p}) = \mathbf{f},$$

где известны $\mathcal{A}$, $\mathbf{p}$ и $\mathbf{f}$, а требуется найти $\mathbf{u}$.

Например:

- известны свойства материала и нагрузка — требуется определить напряжения и деформации;
- известны коэффициенты теплопроводности и источники тепла — требуется найти распределение температуры;
- известны начальное состояние и параметры динамической системы — требуется определить её дальнейшую эволюцию.

Для задачи теплопроводности прямой задачей является, например,

$$\begin{cases} \frac{\partial T}{\partial t} = a^2 \frac{\partial^2 T}{\partial x^2}, & 0 < x < L, \quad t > 0, \\ T(x, 0) = T_0(x), \\ T(0, t) = T_1(t), \\ T(L, t) = T_2(t), \end{cases}$$

где все коэффициенты и условия известны, а требуется определить $T(x, t)$.

Классическим требованием к математической постановке является **корректность по Адамару**. Задача называется корректной, если:

1. решение существует;
2. решение единственно;
3. решение устойчиво относительно малых изменений исходных данных.

49.5 Обратные задачи

В **обратной задаче** часть параметров, воздействий или характеристик модели неизвестна и должна быть восстановлена по наблюдаемым результатам.

Пусть

$$\mathcal{A}(\mathbf{u}, \mathbf{p}) = \mathbf{f},$$

а измеряемая величина определяется оператором

$$\mathbf{y} = \mathcal{H}(\mathbf{u}).$$

В прямой задаче по заданному $\mathbf{p}$ находится $\mathbf{y}$:

$$\mathbf{p} \longrightarrow \mathbf{u} \longrightarrow \mathbf{y}.$$

В обратной задаче по измеренным данным $\mathbf{y}^\delta$ требуется восстановить

$$\mathbf{p}.$$

Например:

- по измеренному температурному полю определить коэффициент теплопроводности;
- по перемещениям конструкции определить приложенные силы;
- по значениям поля на границе определить внутренний источник;
- по экспериментальным данным определить параметры модели;
- по томографическим измерениям восстановить внутреннюю структуру объекта.

Можно выделить несколько типов обратных задач:

- восстановление параметров;
- восстановление начальных условий;
- восстановление граничных условий;
- восстановление источников;
- восстановление формы области;
- идентификация самой структуры модели.

Обратные задачи часто являются **некорректными**: малые ошибки измерений могут вызывать большие изменения восстановленного решения.

Если необходимо решить

$$Ap = y^\delta,$$

где y^δ содержит погрешность измерения, непосредственное обращение оператора A может быть неустойчивым.

Поэтому применяют методы регуляризации. Например, в методе Тихонова решается задача

$$p_\alpha = \arg \min_p (\|Ap - y^\delta\|^2 + \alpha \|Lp\|^2),$$

где $\alpha > 0$ — параметр регуляризации, а оператор L задаёт дополнительные требования к искомому решению.

Первое слагаемое обеспечивает согласование с экспериментальными данными, а второе препятствует появлению неустойчивых решений.

49.6 Универсальность моделей

Одной из важнейших особенностей математического моделирования является **универсальность математических моделей**.

Она состоит в том, что математически одинаковые или близкие модели могут описывать совершенно различные физические, технические, биологические или экономические процессы.

Причина заключается в том, что при абстрагировании от конкретной природы объекта остаётся структура связей между величинами.

Например, уравнение

$$\frac{\partial u}{\partial t} = D\Delta u$$

может описывать:

- теплопроводность;
- диффузию вещества;
- распространение примеси;
- некоторые процессы переноса в биологических системах.

При этом физический смысл u и коэффициента D различается, но математическая структура задачи остаётся одинаковой.

Другой пример — уравнение гармонического осциллятора

$$\ddot{x} + \omega^2 x = 0.$$

Оно возникает при описании:

- механических колебаний;
- электрических колебаний;
- малых колебаний сложных динамических систем.

Универсальность позволяет:

- переносить математические результаты между предметными областями;
- применять одни и те же численные методы к различным задачам;
- создавать универсальные программные комплексы;
- классифицировать модели не только по предметной области, но и по математической структуре.

Например, один и тот же метод конечных элементов может использоваться для расчёта задач упругости, теплопроводности, электромагнетизма и других краевых задач.

49.7 Принцип аналогий

Принцип аналогий состоит в том, что при построении модели нового объекта используется сходство его структуры или поведения с уже изученной системой.

Различают:

- физическую аналогию;
- структурную аналогию;
- математическую аналогию.

При математической аналогии процессы различной физической природы описываются одинаковыми математическими уравнениями.

Например, стационарное распределение температуры без источников:

$$\Delta T = 0$$

и электростатический потенциал в области без зарядов:

$$\Delta \varphi = 0$$

описываются одним и тем же уравнением Лапласа.

Поэтому свойства решения, методы анализа и численные алгоритмы, разработанные для одной задачи, могут быть использованы для другой.

Другой пример — механическая система

$$m\ddot{x} + c\dot{x} + kx = F(t)$$

и электрическая RLC -цепь имеют математически аналогичные уравнения.

Принцип аналогий играет важную роль на начальном этапе моделирования: вместо построения модели с нуля можно определить, к какому уже известному классу математических моделей относится рассматриваемый объект.

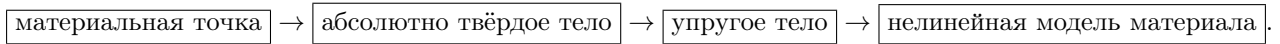
Однако наличие математической аналогии не означает полной физической эквивалентности систем. Область применимости модели должна проверяться отдельно.

49.8 Иерархия моделей

Для одного объекта обычно существует не одна, а целая **иерархия математических моделей**.

Под иерархией понимается совокупность моделей различной степени сложности, детализации и вычислительной стоимости.

Например, движение тела можно описывать последовательно моделями:



Каждый следующий уровень учитывает дополнительные эффекты.

Другой пример — моделирование движения жидкости:

потенциальное течение → уравнения Эйлера → уравнения Навье–Стокса → модели турбулентности.

Модели могут различаться:

- по масштабу:

микро → мезо → макро;

- по пространственной детализации:

сосредоточенная → одномерная → двумерная → трёхмерная;

- по типу описания:

детерминированная ↔ стохастическая;

- по характеру зависимостей:

линейная → нелинейная;

- по учёту времени:

стационарная → нестационарная.

Выбор уровня иерархии определяется целью исследования.

Более сложная модель не обязательно лучше. Она требует:

- большего количества исходных данных;
- большего количества параметров;
- более сложных численных методов;
- больших вычислительных ресурсов.

Поэтому модель должна иметь минимальную сложность, достаточную для достижения требуемой точности.

На практике часто используется подход

простая модель → сравнение с данными → добавление необходимых эффектов.

Иерархия моделей также позволяет проводить взаимную проверку: результаты сложной модели можно сравнивать с более простой моделью в тех режимах, где применимы обе.

49.9 Проверка и уточнение модели

После построения математической модели необходимо убедиться, что она пригодна для решения поставленной задачи.

Следует различать два понятия.

Верификация отвечает на вопрос:

“Правильно ли мы решаем построенную математическую модель?”

Проверяется:

- правильность математических преобразований;
- правильность программной реализации;
- сходимость численного метода;
- зависимость результата от шага сетки;
- выполнение законов сохранения;
- совпадение с аналитическими решениями для тестовых задач.

Например, при последовательном измельчении сетки

$$h_1 > h_2 > h_3 > \dots$$

численное решение должно стабилизироваться:

$$u_h \rightarrow u, \quad h \rightarrow 0.$$

Валидация отвечает на другой вопрос:

“Правильную ли модель реального объекта мы решаем?”

Для этого результаты расчёта сравниваются с экспериментальными или натурными данными.

Если наблюдения имеют вид y_i^{exp} , а модель даёт y_i^{mod} , то, например, можно использовать среднеквадратичную ошибку

$$E = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i^{\text{mod}} - y_i^{\text{exp}})^2}.$$

Дополнительно проводится **анализ чувствительности**: исследуется влияние параметров модели на результат,

$$S_i = \frac{\partial y}{\partial p_i}.$$

Если небольшое изменение некоторого параметра вызывает большое изменение результата, точность определения этого параметра становится особенно важной.

Если результаты модели не соответствуют наблюдениям, возможны:

- уточнение параметров;
- изменение начальных или граничных условий;
- отказ от слишком грубых предположений;
- добавление ранее не учитывавшихся физических эффектов;
- переход к более высокому уровню иерархии моделей.

Таким образом, математическое моделирование представляет собой итерационный процесс:

постановка задачи → модель → алгоритм → расчёт → проверка → уточнение модели.

Основная идея состоит не в построении максимально сложной модели, а в построении модели, которая при разумной вычислительной стоимости адекватно описывает существенные для поставленной задачи свойства объекта.

- 50 Интерполирование функций, полиномы Чебышева, интерполяция с кратными узлами, кубические сплайны
 - 50.1 Постановка задачи интерполяции
 - 50.2 Интерполяционные многочлены Лагранжа и Ньютона
 - 50.3 Погрешность полиномиальной интерполяции
 - 50.4 Полиномы и узлы Чебышева
 - 50.5 Интерполяция с кратными узлами
 - 50.6 Интерполяционный многочлен Эрмита
 - 50.7 Кубические интерполяционные сплайны
 - 50.8 Граничные условия для кубического сплайна
 - 50.9 Сравнение глобальной и сплайн-интерполяции

51 Исследование операций и задачи искусственного интеллекта. Экспертизы и неформальные процедуры. Автоматизация проектирования

51.1 Предмет и задачи исследования операций

Исследование операций — прикладная математическая дисциплина, изучающая методы обоснования и принятия оптимальных решений при управлении сложными системами.

Объектом исследования обычно является некоторая управляемая система, функционирование которой зависит от выбранного решения. Требуется определить такое решение, которое в заданных условиях обеспечивает наилучшее значение выбранного критерия.

В общем виде задачу можно представить как

$$x^* = \arg \min_{x \in X} F(x),$$

или

$$x^* = \arg \max_{x \in X} F(x),$$

где x — вектор управляемых параметров, X — множество допустимых решений, $F(x)$ — критерий эффективности.

Типичная схема исследования операций включает:

1. анализ реальной системы и постановку задачи;
2. выделение управляемых и неуправляемых факторов;
3. построение математической модели;
4. выбор критерия эффективности;
5. нахождение оптимального или рационального решения;
6. анализ чувствительности решения;
7. интерпретацию результатов и принятие решения.

К основным классам задач исследования операций относятся:

- распределение ограниченных ресурсов;
- транспортные задачи;
- задачи назначения;
- сетевые задачи и задачи о потоках;
- календарное и оперативное планирование;
- управление запасами;
- теория массового обслуживания;
- теория игр;
- управление проектами;
- имитационное моделирование.

Характерной особенностью исследования операций является системный подход: оптимизируется не отдельный элемент системы, а эффективность функционирования системы в целом.

51.2 Модели принятия решений

Принятие решения состоит в выборе одного варианта из множества допустимых альтернатив.

Формальная модель принятия решений содержит:

- множество альтернатив X ;
- множество возможных состояний внешней среды;
- критерии оценки решений;
- ограничения;
- информацию о последствиях выбора;
- правило выбора решения.

В простейшем случае имеется один критерий:

$$F(x) \rightarrow \min, \quad x \in X.$$

При наличии нескольких критериев задача принимает вид

$$F(x) = (F_1(x), F_2(x), \dots, F_m(x)), \quad x \in X.$$

Одновременная оптимизация всех критериев обычно невозможна, поэтому применяют:

- свёртку критериев:

$$F(x) = \sum_{i=1}^m w_i F_i(x), \quad w_i \geq 0, \quad \sum_{i=1}^m w_i = 1;$$

- выделение главного критерия и перевод остальных в ограничения;
- поиск множества Парето;
- методы экспертного ранжирования альтернатив.

Решение x_1 называется **Парето-доминирующим** решение x_2 , если оно не хуже x_2 по всем критериям и хотя бы по одному критерию строго лучше.

В зависимости от имеющейся информации различают принятие решений:

- **в условиях определённости** — последствия каждого решения известны;
- **в условиях риска** — возможные состояния и их вероятности известны;
- **в условиях неопределённости** — вероятности состояний неизвестны;
- **в условиях конфликта** — результат зависит также от решений других участников.

При наличии случайных факторов может оптимизироваться математическое ожидание:

$$x^* = \arg \max_{x \in X} \mathbf{E} F(x, \xi),$$

где ξ — случайный фактор.

51.3 Оптимизационные задачи исследования операций

Основной математический аппарат исследования операций связан с оптимизацией.

Общая задача математического программирования имеет вид

$$F(x) \rightarrow \min,$$

при ограничениях

$$g_i(x) \leq 0, \quad i = 1, \dots, m,$$

$$h_j(x) = 0, \quad j = 1, \dots, k.$$

В зависимости от свойств модели возникают различные классы задач.

Линейное программирование:

$$c^T x \rightarrow \min, \quad Ax \leq b.$$

Целочисленное программирование:

$$x_i \in \mathbb{Z}.$$

Такие задачи возникают, например, когда переменная означает количество объектов или факт принятия решения:

$$x_i \in \{0, 1\}.$$

Нелинейное программирование возникает, если целевая функция или ограничения нелинейны.

Динамическое программирование используется для многоэтапных процессов. Основная идея заключается в том, что оптимальное решение задачи строится из оптимальных решений её подзадач. Это выражается **принципом оптимальности Беллмана**.

Сетевые модели используются для задач поиска кратчайшего пути, максимального потока, минимального остовного дерева и других задач на графах.

Кроме аналитических и численных методов оптимизации широко применяется **имитационное моделирование**. Оно особенно полезно, когда явное аналитическое решение задачи получить невозможно.

51.4 Основные задачи искусственного интеллекта

Искусственный интеллект — направление информатики, связанное с построением систем, способных решать задачи, для которых традиционно требуется интеллектуальная деятельность человека.

К основным задачам искусственного интеллекта относятся:

- поиск решений в пространстве состояний;
- планирование действий;
- представление знаний;
- логический вывод;
- распознавание и классификация объектов;
- машинное обучение;
- обработка естественного языка;
- прогнозирование;
- поддержка принятия решений;
- управление интеллектуальными агентами и робототехническими системами.

Одним из классических подходов является представление задачи в виде пространства состояний.

Пусть S — множество возможных состояний системы, s_0 — начальное состояние, S_f — множество целевых состояний. Необходимо найти последовательность допустимых переходов

$$s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_k, \quad s_k \in S_f.$$

Для поиска применяются:

- поиск в ширину;
- поиск в глубину;
- поиск с ограничением глубины;
- эвристический поиск;
- алгоритм A^* .

В алгоритме A^* вершина выбирается по оценке

$$f(n) = g(n) + h(n),$$

где $g(n)$ — стоимость пути от начального состояния до n , а $h(n)$ — эвристическая оценка стоимости пути от n до цели.

Современные системы искусственного интеллекта часто используют машинное обучение. В этом случае зависимость

$$y = f(x)$$

не задаётся непосредственно программистом, а восстанавливается по обучающей выборке

$$D = \{(x_i, y_i)\}_{i=1}^N.$$

Исследование операций и искусственный интеллект имеют общую цель — получение рационального решения. Однако в исследовании операций обычно предполагается наличие достаточно формализованной математической модели, тогда как методы ИИ применимы и к задачам, для которых знания представлены частично, эвристически или извлекаются из данных.

51.5 Экспертные системы

Экспертная система — программная система, использующая знания специалистов некоторой предметной области для решения задач, требующих экспертных рассуждений.

Классическая экспертная система содержит:

- базу знаний;
- рабочую память;
- механизм логического вывода;
- подсистему приобретения и редактирования знаний;
- пользовательский интерфейс;
- подсистему объяснения полученного решения.

Знания могут представляться в виде:

- продукционных правил;
- логических формул;
- фреймов;
- семантических сетей;
- онтологий.

Наиболее распространённая форма продукционного правила:

ЕСЛИ A , ТО B .

Например:

ЕСЛИ температура высокая И давление растёт, ТО состояние системы опасное.

Различают два основных способа применения правил.

Прямой вывод начинается с известных фактов. К ним последовательно применяются правила до получения новых фактов и заключения:

факты $\rightarrow$ правила $\rightarrow$ заключение.

Обратный вывод начинается с некоторой гипотезы. Система определяет, какие условия должны выполняться для её подтверждения, после чего пытается доказать эти условия.

Экспертные системы применяются для диагностики, классификации, интерпретации данных, выбора оборудования, планирования, проектирования и поддержки принятия решений.

Их важное достоинство — возможность явно представить экспертные знания и объяснить логику получения результата.

Основной недостаток — сложность извлечения знаний у экспертов и последующего сопровождения базы знаний. Это называют **проблемой приобретения знаний**.

51.6 Экспертизы и неформальные процедуры

Не все задачи принятия решений допускают полное математическое описание. Причинами могут быть:

- недостаток количественных данных;
- наличие качественных критериев;
- уникальность рассматриваемой ситуации;
- высокая неопределённость;
- невозможность формализовать опыт специалиста.

В таких случаях используются **экспертные оценки** и **неформальные процедуры**.

Экспертная оценка заключается в получении и обработке суждений специалистов предметной области. Основными методами являются:

- ранжирование альтернатив;
- непосредственная балльная оценка;
- попарное сравнение;
- анкетирование;
- интервьюирование;
- метод Делфи;
- коллективное обсуждение;
- построение сценариев.

При ранжировании эксперт располагает объектами в порядке предпочтения:

$$x_{i_1} \succ x_{i_2} \succ \dots \succ x_{i_n}.$$

При использовании нескольких экспертов их оценки могут агрегироваться, например,

$$R_i = \sum_{j=1}^m w_j r_{ij},$$

где r_{ij} — оценка объекта i экспертом j , w_j — вес компетентности эксперта.

Для попарных сравнений может использоваться матрица

$$A = (a_{ij}),$$

где a_{ij} характеризует предпочтительность альтернативы i относительно альтернативы j .

Метод Делфи основан на последовательном анонимном опросе группы экспертов. После каждого раунда экспертам предоставляется обобщённая информация о результатах предыдущего раунда, что позволяет постепенно сближать оценки.

Неформальные процедуры не означают произвольное принятие решений. Их задача — формализовать настолько, насколько возможно, знания и опыт специалистов там, где построение полной математической модели затруднено.

На практике эффективным является сочетание:

математическая модель + данные + экспертные знания.

51.7 Системы автоматизированного проектирования

Система автоматизированного проектирования (САПР) — организационно-техническая система, предназначенная для автоматизации процессов проектирования объектов и систем.

В англоязычной терминологии часто используется обозначение **CAD** (*Computer-Aided Design*).

Процесс проектирования можно представить как поиск параметров объекта

$$x = (x_1, \dots, x_n)$$

при выполнении совокупности требований

$$g_i(x) \leq 0$$

и достижении требуемых характеристик

$$F(x) \rightarrow \min \quad \text{или} \quad F(x) \rightarrow \max.$$

САПР могут обеспечивать:

- геометрическое моделирование;
- параметрическое проектирование;
- ведение проектной документации;
- инженерные расчёты;
- моделирование поведения проектируемого объекта;
- проверку ограничений;
- оптимизацию параметров конструкции;
- хранение и повторное использование проектных данных.

В инженерной практике обычно различают:

- **CAD** — построение и редактирование модели объекта;
- **CAE** (*Computer-Aided Engineering*) — инженерный анализ и вычислительное моделирование;
- **CAM** (*Computer-Aided Manufacturing*) — автоматизированная подготовка производства.

При параметрическом проектировании модель задаётся набором параметров. Изменение параметров автоматически приводит к перестроению геометрии и связанных характеристик объекта.

Это позволяет автоматически исследовать множество вариантов проекта и решать задачи оптимального проектирования.

51.8 Интеграция оптимизации, искусственного интеллекта и проектирования

Современная автоматизация проектирования объединяет методы математического моделирования, оптимизации, искусственного интеллекта и экспертного анализа.

Типичный автоматизированный цикл имеет вид:

параметры проекта $\rightarrow$ математическая модель $\rightarrow$ расчёт $\rightarrow$ оценка критериев $\rightarrow$ оптимизация $\rightarrow$ новые параметры

Цикл повторяется до достижения заданного критерия остановки.

Например, при проектировании конструкции требуется подобрать вектор параметров x так, чтобы

$$m(x) \rightarrow \min,$$

при ограничениях

$$\sigma_{\max}(x) \leq [\sigma], \quad u_{\max}(x) \leq [u],$$

где m — масса конструкции, $\sigma_{\max}$ — максимальное напряжение, $u_{\max}$ — максимальное перемещение.

Математическая оптимизация позволяет автоматически искать лучшие значения параметров.

Методы искусственного интеллекта могут использоваться для:

- выбора первоначальной конфигурации;
- прогнозирования характеристик проектируемого объекта;
- поиска закономерностей в результатах расчётов;
- построения приближённых моделей;
- генерации вариантов проекта;

- обнаружения ошибочных или недопустимых решений;
- использования накопленного опыта предыдущих проектов.

Экспертные знания позволяют учитывать требования, которые трудно представить в виде точных математических ограничений.

Таким образом, современная интеллектуальная САПР может рассматриваться как система поддержки принятия проектных решений, в которой объединяются

модель + оптимизация + данные + экспертные знания + ИИ.

При этом окончательное решение во многих задачах остаётся за человеком. Автоматизированная система перебирает и анализирует большое количество вариантов, проверяет ограничения и предлагает рациональные решения, а специалист оценивает факторы, которые невозможно полностью формализовать.

- 52 Вариационные методы решения краевых задач и определения собственных значений
 - 52.1 Слабая постановка краевой задачи
 - 52.2 Энергетический функционал
 - 52.3 Метод Ритца
 - 52.4 Метод Галёркина
 - 52.5 Вариационная постановка задачи на собственные значения
 - 52.6 Отношение Рэля
 - 52.7 Минимаксные принципы
 - 52.8 Сходимость и оценка погрешности

53 Вычислительный эксперимент. Принципы проведения вычислительного эксперимента. Модель, алгоритм, программа

53.1 Понятие вычислительного эксперимента

Вычислительный эксперимент — исследование математической модели объекта или процесса с помощью вычислительных методов и компьютерных программ.

В отличие от натурного эксперимента, где исследуется непосредственно реальный объект, в вычислительном эксперименте исследуется его математическая модель.

В общем виде математическую модель можно записать как

$$\mathcal{M}(u, p, x, t) = 0,$$

где

- u — искомые характеристики системы;
- p — параметры модели;
- x, t — пространственные и временные переменные;
- $\mathcal{M}$ — совокупность уравнений, ограничений, начальных и граничных условий.

После выбора численного метода непрерывная математическая задача заменяется конечномерной вычислительной задачей

$$\mathcal{M}_h(u_h, p) = 0,$$

где h обозначает параметры дискретизации: шаг сетки, шаг по времени, число элементов и т.п.

Таким образом, вычислительный эксперимент представляет собой последовательность вычислений, выполняемых для исследования поведения модели при различных исходных данных, параметрах и режимах.

Он особенно важен в тех случаях, когда натурный эксперимент:

- слишком дорог;
- технически сложен;
- опасен;
- занимает слишком много времени;
- принципиально невозможен;
- не позволяет непосредственно наблюдать интересующие характеристики.

53.2 Цели и задачи вычислительного эксперимента

Основной целью вычислительного эксперимента является получение информации о свойствах исследуемой системы на основе её математической модели.

К основным задачам относятся:

1. Исследование поведения системы.

Определяется изменение характеристик объекта во времени и пространстве, устанавливаются характерные режимы его работы.

2. Исследование влияния параметров.

Определяется зависимость результата

$$y = F(p_1, p_2, \dots, p_m)$$

от параметров модели.

3. Поиск критических режимов.

Исследуются условия потери устойчивости, возникновения резонанса, разрушения, перехода к другому режиму и т.п.

4. Оптимизация.

Требуется найти параметры p , минимизирующие или максимизирующие некоторый критерий

$$J(p) \rightarrow \min_{p \in P}$$

при выполнении ограничений модели.

5. Проверка гипотез.

С помощью серии расчётов сравниваются различные предположения о механизмах исследуемого процесса.

6. Прогнозирование.

Модель используется для предсказания поведения системы в условиях, которые ещё не наблюдались экспериментально.

7. Снижение объёма натурных экспериментов.

Вычисления позволяют предварительно определить наиболее интересные области параметров и сократить число дорогостоящих физических экспериментов.

Важно, что вычислительный эксперимент не устраняет необходимость натурального эксперимента. Он дополняет его и позволяет исследовать модель в значительно более широком диапазоне условий.

53.3 Триада «модель–алгоритм–программа»

Основой вычислительного эксперимента является триада

математическая модель $\longrightarrow$ вычислительный алгоритм $\longrightarrow$ программа

Каждый из этих уровней отвечает на свой вопрос.

Математическая модель

Модель описывает существенные свойства исследуемого объекта средствами математики.

Она может включать:

- алгебраические уравнения;
- обыкновенные дифференциальные уравнения;
- уравнения в частных производных;
- интегральные уравнения;
- вероятностные зависимости;
- ограничения;
- начальные и граничные условия.

Например, процесс теплопроводности может описываться уравнением

$$\frac{\partial u}{\partial t} = a^2 \frac{\partial^2 u}{\partial x^2}.$$

Модель представляет физическую, экономическую, биологическую или иную предметную задачу в математической форме.

Алгоритм

Как правило, математическая модель не может быть непосредственно решена компьютером.

Поэтому строится **вычислительный алгоритм** — конечная последовательность операций, позволяющая получить приближённое решение задачи.

Например, производную можно заменить конечной разностью

$$\frac{\partial^2 u}{\partial x^2}(x_i, t_k) \approx \frac{u_{i-1}^k - 2u_i^k + u_{i+1}^k}{h^2}.$$

После дискретизации исходная задача сводится к конечномерной системе алгебраических уравнений. Для вычислительного алгоритма существенны:

- корректность;
- точность;
- устойчивость;
- сходимость;
- вычислительная сложность;
- требования к памяти;
- возможность распараллеливания.

Программа

Программа представляет собой реализацию алгоритма на конкретной вычислительной системе.

Она включает:

- представление исходных данных;
- реализацию вычислительных методов;
- организацию вычислений;
- хранение промежуточных результатов;
- ввод и вывод данных;
- средства анализа и визуализации результатов.

Важно различать ошибки трёх уровней.

Ошибочная модель не станет правильной при использовании точного алгоритма.

Правильная модель не даст правильного результата при некорректном численном методе.

Наконец, правильная модель и алгоритм могут быть испорчены ошибкой программной реализации.

Поэтому каждый уровень триады должен проверяться отдельно.

53.4 Этапы проведения вычислительного эксперимента

Типичный вычислительный эксперимент состоит из следующих этапов.

1. Постановка задачи.

Определяются объект исследования, цель моделирования, интересующие характеристики и диапазоны изменения параметров.

2. Построение математической модели.

Выбираются существенные свойства объекта, вводятся предположения и упрощения, формулируются уравнения, начальные и граничные условия.

3. Исследование математической задачи.

Желательно установить существование и единственность решения, свойства решения, характерные масштабы и возможные особые режимы.

4. Выбор численного метода.

Непрерывная задача заменяется дискретной. Выбираются сетка, шаги, аппроксимации и методы решения возникающих алгебраических систем.

5. Разработка вычислительного алгоритма.

Определяется последовательность операций, критерии завершения итераций, контроль ошибок и структура данных.

6. Программная реализация.

Алгоритм реализуется в виде программы или с использованием существующего программного комплекса.

7. Верификация.

Проверяется правильность вычислительного алгоритма и его программной реализации.

8. Проведение серии расчётов.

Выполняются вычисления для различных параметров, начальных данных и вариантов модели.

9. Валидация.

Результаты модели сопоставляются с экспериментальными данными или другими достоверными источниками.

10. Анализ результатов.

Определяются закономерности, чувствительность решения к параметрам, области устойчивости и границы применимости модели.

11. Уточнение модели.

При необходимости изменяются модель, численный метод или параметры, после чего вычислительный эксперимент повторяется.

Таким образом, вычислительный эксперимент имеет итерационный характер:

модель $\rightarrow$ расчёт $\rightarrow$ сравнение $\rightarrow$ уточнение модели.

53.5 Планирование серии расчётов

Отдельный расчёт обычно даёт мало информации. Для исследования модели проводится **серия вычислительных экспериментов**.

Пусть модель зависит от параметров

$$p = (p_1, \dots, p_m).$$

Необходимо выбрать набор точек

$$p^{(1)}, p^{(2)}, \dots, p^{(N)},$$

в которых будут выполнены расчёты.

При планировании определяются:

- диапазоны изменения параметров;
- значения управляющих факторов;
- шаг изменения параметров;
- исследуемые выходные характеристики;
- критерии сравнения различных режимов;
- требуемая точность вычислений.

Наиболее простой подход — изменение одного параметра при фиксированных остальных:

$$y = y(p_i).$$

Однако при большом количестве параметров такой подход становится неэффективным, поскольку число возможных комбинаций быстро возрастает.

Поэтому могут применяться методы **планирования эксперимента**, позволяющие получить максимальное количество информации при ограниченном числе расчётов.

Важной задачей является также **анализ чувствительности**. Для параметра p_i можно рассматривать величину

$$S_i = \frac{\partial y}{\partial p_i},$$

характеризующую чувствительность результата y к изменению данного параметра.

Если вычисление производной затруднено, используется конечная разность

$$S_i \approx \frac{y(p_i + \Delta p_i) - y(p_i)}{\Delta p_i}.$$

Планирование расчётов должно учитывать не только свойства модели, но и вычислительную стоимость одного запуска программы.

53.6 Верификация алгоритма и программы

Верификация отвечает на вопрос:

«Правильно ли мы решаем математическую задачу?»

При верификации проверяется соответствие вычислительного результата исходной математической модели. Следует различать несколько источников ошибок:

$$\varepsilon_{\text{общ}} \approx \varepsilon_{\text{дискр}} + \varepsilon_{\text{итер}} + \varepsilon_{\text{окр}},$$

где

- $\varepsilon_{\text{дискр}}$ — ошибка дискретизации;
- $\varepsilon_{\text{итер}}$ — ошибка незавершённого итерационного процесса;
- $\varepsilon_{\text{окр}}$ — ошибки округления.

Основные способы верификации:

1. Сравнение с аналитическим решением.

Если для специального случая известно точное решение u , вычисляется ошибка

$$e_h = \|u - u_h\|.$$

2. Исследование сходимости по сетке.

Расчёт повторяется на последовательности сеток

$$h, \quad \frac{h}{2}, \quad \frac{h}{4}.$$

Если метод имеет порядок p , то ожидается

$$\|u - u_h\| = O(h^p).$$

При отсутствии точного решения сравнивают результаты на различных сетках:

$$\|u_h - u_{h/2}\|.$$

3. Проверка законов сохранения.

Например, если исходная модель сохраняет массу или энергию, необходимо контролировать соответствующий баланс.

4. Сравнение с независимой программой или методом.

Одна задача решается различными численными методами или программными пакетами.

5. Тестирование программы.

Проверяются отдельные программные модули, граничные случаи, обработка входных данных и устойчивость программы к некорректным данным.

Таким образом, верификация относится прежде всего к связке

математическая модель $\longrightarrow$ алгоритм $\longrightarrow$ программа.

53.7 Валидация математической модели

Валидация отвечает на другой вопрос:

«Правильно ли математическая модель описывает реальный объект?»

Для этого результаты вычислений сопоставляются с экспериментальными данными

$$y_i^{\text{calc}} \quad \text{и} \quad y_i^{\text{exp}}.$$

Например, можно использовать среднеквадратичное отклонение

$$E = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i^{\text{calc}} - y_i^{\text{exp}})^2}.$$

Валидация может включать:

- качественное сравнение характера процессов;
- количественное сравнение с экспериментом;
- сравнение характерных величин;
- проверку предсказаний модели;
- исследование диапазона её применимости.

Необходимо чётко различать верификацию и валидацию:

верификация:	правильно ли решены уравнения?
валидация:	правильные ли уравнения выбраны для объекта?

Даже полностью верифицированная программа может давать физически неправильные результаты, если сама математическая модель неадекватна объекту.

53.8 Анализ и представление результатов

После выполнения расчётов необходимо не просто получить набор чисел, а извлечь из него информацию о свойствах исследуемой системы.

Анализ может включать:

- построение зависимостей выходных величин от параметров;
- исследование пространственных и временных распределений;
- определение максимумов и минимумов;
- выявление критических значений параметров;
- исследование устойчивости;
- анализ чувствительности;
- статистическую обработку результатов;

- сравнение нескольких моделей или алгоритмов.

Результаты удобно представлять в виде:

- таблиц;
- графиков;
- полей и изолиний;
- трёхмерных визуализаций;
- диаграмм;
- анимаций динамических процессов.

При представлении результатов обязательно должны быть указаны:

- значения параметров модели;
- используемый численный метод;
- параметры дискретизации;
- критерии остановки итераций;
- единицы измерения;
- оценка точности результата.

В противном случае результат трудно интерпретировать и практически невозможно воспроизвести.

53.9 Воспроизводимость вычислительного эксперимента

Воспроизводимость означает возможность повторить вычислительный эксперимент и получить те же или статистически согласующиеся результаты.

Для воспроизводимости необходимо сохранять:

- точную формулировку математической модели;
- исходные данные;
- значения параметров;
- начальные и граничные условия;
- используемый численный метод;
- параметры дискретизации;
- версию программы;
- версии используемых библиотек;
- параметры вычислительной среды;
- случайные начальные значения при использовании стохастических методов.

Для стохастического алгоритма, например,

$$X^{(k+1)} = F(X^{(k)}, \xi_k),$$

результат зависит от последовательности случайных чисел ξ_k . Поэтому для точного повторения расчёта необходимо фиксировать начальное состояние генератора случайных чисел.

В современных вычислительных исследованиях желательно также использовать:

- системы контроля версий;
- автоматизированные сценарии запуска расчётов;
- конфигурационные файлы вместо ручного задания параметров;

- автоматическую обработку результатов;
- сохранение метаданных вычислительного эксперимента.

Воспроизводимость позволяет отличить научный вычислительный эксперимент от разового запуска программы, результат которого невозможно независимо проверить.

В итоге вычислительный эксперимент представляет собой не просто численное решение одной задачи, а полный цикл исследования:

постановка задачи → модель → алгоритм → программа → расчёт → верификация и валидация → анализ результатов

- 54 Моделирование методом динамики частиц. Молекулярная динамика
 - 54.1 Частичная модель вещества
 - 54.2 Уравнения движения частиц
 - 54.3 Потенциалы межчастичного взаимодействия
 - 54.4 Начальные и граничные условия
 - 54.5 Численное интегрирование уравнений движения
 - 54.6 Списки соседей и вычислительная эффективность
 - 54.7 Термостаты и статистические ансамбли
 - 54.8 Вычисление макроскопических характеристик
 - 54.9 Преимущества и ограничения молекулярной динамики

55 Алгоритмические языки. Представление о языках программирования высокого уровня. Пакеты прикладных программ

55.1 Алгоритм и алгоритмический язык

Алгоритм — конечная последовательность однозначно определённых действий, приводящая от исходных данных к результату за конечное число шагов.

Обычно выделяют следующие основные свойства алгоритма:

- **дискретность** — алгоритм состоит из отдельных шагов;
- **определённость** — каждое действие должно быть однозначно задано;
- **конечность** — для допустимых исходных данных выполнение должно завершаться за конечное число шагов;
- **результативность** — после завершения должен быть получен некоторый результат;
- **массовость** — алгоритм предназначен не только для одного набора данных, а для некоторого класса однотипных задач.

Алгоритм можно задавать различными способами:

- словесным описанием;
- математическими формулами;
- блок-схемой;
- псевдокодом;
- программой на формальном языке.

Алгоритмический язык — формальный язык, предназначенный для записи алгоритмов в форме, допускающей их однозначное понимание и, как правило, автоматическое выполнение или преобразование.

Алгоритмический язык задаёт:

- алфавит допустимых символов;
- правила образования конструкций языка;
- способы представления данных;
- управляющие конструкции;
- правила выполнения записанных конструкций.

Исторически понятие алгоритмического языка является несколько более широким, чем понятие языка программирования. Алгоритм можно записывать, например, на формализованном псевдокоде, который не предназначен непосредственно для выполнения компьютером. **Язык программирования** представляет собой формальный алгоритмический язык, для которого определены средства трансляции и выполнения программ на вычислительной системе.

Типичная программа содержит три основных составляющих:

1. описание данных;
2. описание операций над ними;
3. описание порядка выполнения этих операций.

Для структурного программирования базовыми управляющими конструкциями являются:

- **следование** — последовательное выполнение операторов;
- **ветвление** — выбор одного из вариантов вычисления;
- **цикл** — многократное выполнение некоторого набора действий.

55.2 Синтаксис и семантика языка

Описание любого формального языка включает две принципиально различные части: **синтаксис** и **семантику**.

Синтаксис определяет, какие последовательности символов являются правильными конструкциями языка.

К синтаксису относятся:

- лексемы языка: идентификаторы, ключевые слова, литералы, знаки операций;
- правила записи выражений;
- правила объявления переменных и функций;
- правила построения операторов;
- структура программных модулей.

Например, выражение

$$a = b + c;$$

может быть синтаксически корректным в одном языке, тогда как отсутствие точки с запятой в том же месте может приводить к синтаксической ошибке.

Формальное описание синтаксиса часто строится при помощи грамматик, например формы Бэкуса–Наура.

Семантика определяет смысл синтаксически корректной конструкции, то есть результат её выполнения.

Например, выражение

$$x = x + 1$$

с точки зрения математики является противоречивым равенством, но в императивном языке программирования оно означает присваивание переменной x значения, на единицу большего предыдущего.

Различают:

- **статическую семантику** — свойства программы, проверяемые до её выполнения;
- **динамическую семантику** — поведение программы непосредственно во время выполнения.

К статическим проверкам могут относиться:

- соответствие типов;
- область видимости идентификаторов;
- корректность количества и типов аргументов функции.

Важным элементом языка является **система типов**. Тип определяет множество допустимых значений и операций над ними.

Распространённые типы данных:

- целые числа;
- вещественные числа;
- логические значения;
- символы и строки;
- массивы;
- записи и структуры;
- указатели и ссылки;
- пользовательские типы и классы.

Системы типов могут быть:

- **статическими**, когда большинство типов известно до выполнения программы;
- **динамическими**, когда тип значения определяется непосредственно во время выполнения.

55.3 Языки низкого и высокого уровня

Уровень языка характеризует степень абстрагирования от конкретной архитектуры вычислительной машины.

Языки низкого уровня тесно связаны с системой команд процессора.

К ним относят:

- машинный код;
- языки ассемблера.

Машинная программа непосредственно представляется последовательностью команд процессора.

Ассемблер использует символические обозначения машинных команд, например:

```
MOV AX, BX  
ADD AX, 1
```

Преимущества низкоуровневого программирования:

- непосредственный контроль над аппаратурой;
- возможность точного управления памятью;
- потенциально высокая производительность;
- возможность использования специфических команд процессора.

Недостатки:

- сильная зависимость программы от архитектуры;
- высокая трудоёмкость разработки;
- сложность сопровождения;
- повышенная вероятность ошибок.

Языки высокого уровня предоставляют программисту абстракции, не связанные непосредственно с машинными командами.

К ним относятся, например: **Fortran**, **C++**, **Java**, **Python**, **C#**.

Основные особенности высокоуровневых языков:

- наличие развитых типов данных;
- возможность использования функций и модулей;
- наличие структур управления;
- автоматизация части работы с памятью;
- библиотеки стандартных алгоритмов и структур данных;
- относительная переносимость исходного кода между различными платформами.

Высокоуровневый язык позволяет описывать задачу в терминах, более близких к предметной области, а не к системе команд процессора.

Следует отметить, что деление на уровни условно. Например, **C** и **C++** являются языками высокого уровня, но предоставляют достаточно низкоуровневые средства работы с памятью и аппаратурой.

55.4 Парадигмы программирования

Парадигма программирования — совокупность принципов и способов организации вычислений и структуры программы.

Современные языки часто являются **мультипарадигменными**, то есть поддерживают одновременно несколько подходов.

Императивное программирование Программа рассматривается как последовательность команд, изменяющих состояние вычислительной системы.

Основной механизм — присваивание:

$$x \leftarrow x + 1.$$

К императивным языкам относятся, например, **C**, **Fortran**, **Pascal**.

Процедурное программирование Является развитием императивного подхода. Программа разбивается на процедуры и функции.

Основные преимущества:

- декомпозиция задачи;
- повторное использование кода;
- локализация данных и алгоритмов.

Объектно-ориентированное программирование Программа строится как система взаимодействующих объектов.

Основными понятиями являются:

- класс;
- объект;
- инкапсуляция;
- наследование;
- полиморфизм.

Объект объединяет данные и операции над ними.

Например, объект, моделирующий матрицу, может хранить её элементы и предоставлять методы умножения, вычисления нормы или решения системы линейных уравнений.

Функциональное программирование Вычисление рассматривается как вычисление значений функций.

Характерные особенности:

- широкое использование функций высших порядков;
- стремление уменьшить количество изменяемых состояний;
- использование рекурсии;
- удобство композиции функций.

Логическое программирование Программа описывает набор фактов и логических правил, а вычисление заключается в логическом выводе.

Классическим примером является язык **Prolog**.

Декларативный подход В декларативных языках программист преимущественно указывает, *что* требуется получить, не задавая детально, *как* именно выполнить вычисление.

К декларативным языкам предметных областей можно отнести, например, язык запросов **SQL**.

55.5 Трансляторы: компиляторы и интерпретаторы

Для выполнения программы, записанной на языке высокого уровня, необходимо преобразовать её в форму, понятную вычислительной системе.

Транслятор — программа, преобразующая текст с одного формального языка на другой.

Основные виды трансляторов:

- ассемблер;
- компилятор;

- интерпретатор.

Ассемблер преобразует программу на языке ассемблера в машинный код.

Компилятор преобразует исходную программу целиком или крупными единицами в машинный код либо в некоторое промежуточное представление.

Типичная схема компиляции:

исходный текст → лексический анализ → синтаксический анализ → семантический анализ → оптимизация → генерация кода

На этапе **лексического анализа** последовательность символов разбивается на лексемы.

На этапе **синтаксического анализа** проверяется соответствие программы грамматике языка и строится её синтаксическое представление.

При **семантическом анализе** проверяются типы, объявления идентификаторов и другие смысловые ограничения.

После этого компилятор может выполнять различные оптимизации и генерировать целевой код.

Отдельным этапом является **компоновка** (*linking*), при которой объектные модули программы и необходимые библиотеки объединяются в исполняемый файл.

Интерпретатор непосредственно выполняет программу или её промежуточное представление, не создавая заранее самостоятельный машинный исполняемый файл для всей программы.

Схематично:

исходная программа → интерпретатор → выполнение.

На практике граница между компиляцией и интерпретацией не является строгой.

Например, распространена схема

исходный код → байт-код → виртуальная машина → машинный код.

Также используется **ЖИТ-компиляция** (*Just-In-Time*), при которой отдельные участки программы переводятся в машинный код непосредственно во время выполнения.

Поэтому язык сам по себе не является строго “компилируемым” или “интерпретируемым”: это прежде всего свойство конкретной реализации языка.

55.6 Среда выполнения и библиотеки

Работа программы определяется не только самим языком и его транслятором. Важную роль играет **среда выполнения** (*runtime environment*).

Среда выполнения может обеспечивать:

- загрузку программы;
- управление памятью;
- организацию вызовов функций;
- обработку исключений;
- ввод и вывод;
- взаимодействие с операционной системой;
- динамическую загрузку библиотек;
- выполнение промежуточного кода;
- многопоточность.

Память выполняющейся программы условно можно разделить на несколько областей:

- область машинного кода;
- область глобальных и статических данных;
- **стек** вызовов функций;
- **кучу** для динамически создаваемых объектов.

В одних языках управление динамической памятью выполняется программистом, в других значительная часть этой работы возлагается на среду выполнения и **сборщик мусора**.

Библиотека — набор готовых программных компонентов, предназначенных для повторного использования. Стандартные библиотеки языка обычно предоставляют:

- контейнеры и структуры данных;
- математические функции;
- работу со строками;
- файловый ввод-вывод;
- средства многопоточности;
- сетевые функции.

Кроме стандартной библиотеки существуют специализированные библиотеки.

В научных вычислениях распространены библиотеки для:

- линейной алгебры;
- решения ОДУ и УЧП;
- оптимизации;
- статистического анализа;
- параллельных вычислений;
- визуализации результатов.

Использование библиотек позволяет не реализовывать повторно стандартные алгоритмы и применять хорошо протестированные и оптимизированные программные компоненты.

55.7 Пакеты прикладных программ

Пакет прикладных программ (ППП) — комплекс взаимосвязанных программных средств, предназначенных для решения некоторого класса прикладных задач.

В отличие от отдельной библиотеки, ППП обычно предоставляет пользователю более высокоуровневую законченную среду решения задач.

Пакет может включать:

- пользовательский интерфейс;
- язык команд или встроенный язык программирования;
- набор алгоритмов;
- библиотеки;
- средства хранения данных;
- средства визуализации;
- средства импорта и экспорта;
- документацию.

ППП можно классифицировать по назначению.

Пакеты общего назначения Используются для типовых пользовательских задач:

- текстовые процессоры;
- электронные таблицы;
- системы управления базами данных.

Математические пакеты Предназначены для аналитических и численных вычислений.

Можно выделить:

- системы компьютерной алгебры;
- системы численных вычислений;
- статистические пакеты;
- средства оптимизации.

Например, математические системы могут выполнять:

- операции линейной алгебры;
- решение систем уравнений;
- численное интегрирование;
- решение дифференциальных уравнений;
- оптимизацию;
- обработку экспериментальных данных;
- построение графиков.

Пакеты математического моделирования Для сложного моделирования обычно используется типовая технологическая цепочка:

геометрия → математическая модель → дискретизация → решатель → постобработка.

Такие комплексы могут содержать:

- генераторы расчётных сеток;
- конечно-разностные или конечно-элементные решатели;
- решатели систем линейных и нелинейных уравнений;
- средства параллельных вычислений;
- средства визуализации полей и сеток.

Примерами классов специализированных ППП являются:

- системы конечно-элементного анализа;
- пакеты вычислительной гидродинамики;
- системы компьютерной алгебры;
- статистические системы;
- системы автоматизированного проектирования.

Важным преимуществом ППП является возможность решать сложные прикладные задачи без реализации всей вычислительной инфраструктуры с нуля.

Однако применение готового пакета не освобождает исследователя от необходимости понимать:

- используемую математическую модель;
- численный метод;
- ограничения решателя;
- погрешности вычислений;
- условия применимости полученного результата.

Иначе возникает ситуация, когда программный комплекс выдаёт численный результат, но пользователь не способен оценить его корректность.

55.8 Критерии выбора языка и программного пакета

Выбор языка программирования определяется не существованием некоторого “лучшего языка”, а требованиями конкретной задачи.

Основные критерии выбора языка:

1. соответствие предметной области;
2. производительность;
3. наличие необходимых библиотек;
4. простота разработки и сопровождения;
5. переносимость;
6. поддержка параллельных вычислений;
7. взаимодействие с другими языками и программами;
8. качество средств тестирования и отладки;
9. наличие специалистов и развитость экосистемы.

Например, при численном моделировании может использоваться комбинация нескольких языков:

- высокопроизводительное вычислительное ядро реализуется на компилируемом языке;
- подготовка данных и автоматизация расчётов выполняются на скриптовом языке;
- специализированные библиотеки используются для линейной алгебры, сеток и параллельных вычислений.

При выборе пакета прикладных программ следует учитывать:

1. наличие необходимых математических моделей;
2. реализованные численные методы;
3. возможность задания требуемых начальных и граничных условий;
4. устойчивость и точность решателей;
5. возможности построения и импорта сеток;
6. масштабируемость на многопроцессорных системах;
7. средства визуализации и постобработки;
8. возможность автоматизации серии расчётов;
9. поддерживаемые форматы данных;
10. возможность интеграции с собственным программным кодом;
11. наличие документации и технической поддержки;
12. стоимость и лицензионные ограничения;
13. воспроизводимость вычислительных результатов.

Таким образом, язык программирования и пакет прикладных программ решают разные уровни задачи. Язык представляет средства построения алгоритмов и программ, библиотеки предоставляют готовые программные компоненты, а пакет прикладных программ является более законченной системой решения определённого класса прикладных задач.

Для математического моделирования типична следующая иерархия:

математическая модель → алгоритм → программная реализация → вычислительный результат
--

При этом корректность конечного результата определяется не только качеством программного обеспечения, но и адекватностью модели, корректностью алгоритма и правильностью выбора численных методов.

- 56 Численные методы вычисления определителей и обратных матриц
 - 56.1 Вычисление определителя методом Гаусса
 - 56.2 Вычисление определителя по LU -разложению
 - 56.3 Вычисление обратной матрицы через решение систем
 - 56.4 Метод Гаусса–Жордана
 - 56.5 Обусловленность задачи обращения матрицы
 - 56.6 Численная устойчивость и выбор главного элемента
 - 56.7 Оценка вычислительной сложности
 - 56.8 Особенности разреженных и структурированных матриц

57 Метод граничных элементов

57.1 Основная идея метода граничных элементов

57.2 Фундаментальное решение и функция Грина

57.3 Переход к граничному интегральному уравнению

57.4 Дискретизация границы

57.5 Граничные элементы и аппроксимация неизвестных

57.6 Формирование системы линейных уравнений

57.7 Сингулярные интегралы

57.8 Преимущества и ограничения метода

57.9 Сравнение с методом конечных элементов

58 Алгоритмы сортировки

58.1 Постановка задачи сортировки

Сортировкой называется задача упорядочения элементов некоторой последовательности в соответствии с заданным отношением порядка.

Пусть дана последовательность

$$A = (a_1, a_2, \dots, a_n).$$

Требуется построить перестановку элементов

$$A' = (a'_1, a'_2, \dots, a'_n),$$

такую, что

$$a'_1 \leq a'_2 \leq \dots \leq a'_n.$$

При этом A' должна содержать те же элементы, что и A , то есть отличаться от исходной последовательности только перестановкой элементов.

На практике сортируются не просто числа, а **записи**, содержащие некоторый ключ:

$$a_i = (\text{key}_i, \text{data}_i).$$

Порядок элементов определяется значениями ключей.

Сортировка является одной из фундаментальных алгоритмических задач. Она используется для:

- ускорения поиска;
- группировки одинаковых элементов;
- построения индексов;
- обработки баз данных;
- подготовки данных к другим алгоритмам;
- нахождения медиан, квантилей и порядковых статистик.

Алгоритмы сортировки существенно различаются по вычислительной сложности, объёму дополнительной памяти и требованиям к исходным данным.

58.2 Устойчивость и другие свойства алгоритмов

При выборе алгоритма сортировки важна не только его асимптотическая сложность.

Устойчивая сортировка сохраняет взаимный порядок элементов с одинаковыми ключами.

Например, пусть записи имеют вид

$$(2, A), \quad (1, B), \quad (2, C).$$

После устойчивой сортировки по первому полю получим

$$(1, B), \quad (2, A), \quad (2, C).$$

Записи $(2, A)$ и $(2, C)$ сохранили исходный порядок.

Устойчивость особенно важна при последовательной сортировке по нескольким ключам. Например, если записи сначала отсортировать по имени, а затем устойчиво по фамилии, то внутри одинаковых фамилий сохранится сортировка по имени.

Основные характеристики алгоритмов сортировки:

- **временная сложность** — число операций в зависимости от n ;
- **пространственная сложность** — объём дополнительной памяти;
- **устойчивость**;
- **сортировка на месте** (*in-place*) — использование $O(1)$ дополнительной памяти, не считая стека рекурсии;

- **адаптивность** — способность работать быстрее на частично упорядоченных данных;
- **онлайновость** — возможность обрабатывать элементы по мере их поступления;
- использование или отсутствие операций сравнения.

Сортировки обычно разделяют на две большие группы:

1. **сортировки сравнениями**, в которых информация о порядке элементов получается с помощью операций вида

$$a_i < a_j;$$

2. **сортировки без сравнений**, использующие дополнительные свойства ключей.

58.3 Сортировки вставками, выбором и обменами

Сортировка вставками

Идея **сортировки вставками** (*Insertion Sort*) состоит в том, что на каждом шаге очередной элемент вставляется в правильное место уже отсортированной части массива.

Пусть элементы

$$a_1, \dots, a_{i-1}$$

уже отсортированы. Элемент a_i сравнивается с предыдущими элементами и сдвигается влево до тех пор, пока не займёт требуемое положение.

Для каждого элемента может потребоваться просмотр почти всей предшествующей части массива.

В худшем случае:

$$T(n) = 1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2} = \Theta(n^2).$$

В лучшем случае, когда массив уже отсортирован,

$$T(n) = \Theta(n).$$

Свойства:

- худшая сложность — $O(n^2)$;
- средняя сложность — $O(n^2)$;
- лучшая сложность — $O(n)$;
- дополнительная память — $O(1)$;
- сортировка устойчива;
- алгоритм адаптивен;
- эффективен для небольших или почти отсортированных массивов.

Сортировка выбором

В **сортировке выбором** (*Selection Sort*) на каждом шаге из ещё не отсортированной части массива выбирается минимальный элемент и переставляется в начало этой части.

На первом шаге ищется минимум среди n элементов, затем среди $n-1$ и так далее.

Число сравнений:

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}.$$

Следовательно,

$$T(n) = \Theta(n^2)$$

при любом расположении элементов.

При этом число перестановок равно лишь $O(n)$.

Свойства:

- сложность во всех случаях — $\Theta(n^2)$;
- дополнительная память — $O(1)$;
- обычная реализация неустойчива;
- число обменов мало.

Алгоритм может быть полезен, когда операция перемещения элемента значительно дороже его сравнения.

Сортировка обменами

Наиболее известным алгоритмом этого класса является **пузырьковая сортировка** (*Bubble Sort*).

Последовательно сравниваются соседние элементы:

$$a_i, a_{i+1}.$$

Если

$$a_i > a_{i+1},$$

то они меняются местами.

После одного полного прохода максимальный элемент оказывается в конце массива. После следующего прохода на своё место перемещается второй по величине элемент и т.д.

В худшем и среднем случаях:

$$T(n) = \Theta(n^2).$$

При использовании признака того, что за проход не произошло ни одного обмена, для уже отсортированного массива получается

$$T(n) = \Theta(n).$$

Свойства:

- простота реализации;
- устойчивость;
- $O(1)$ дополнительной памяти;
- низкая практическая эффективность при больших n .

Поэтому пузырьковая сортировка имеет преимущественно учебное значение.

58.4 Сортировка слиянием

Сортировка слиянием (*Merge Sort*) основана на принципе “разделяй и властвуй”.

Алгоритм состоит из трёх этапов:

1. разбить массив примерно пополам;
2. рекурсивно отсортировать обе половины;
3. слить две отсортированные последовательности.

Пусть имеются две отсортированные последовательности

$$A = (a_1, \dots, a_k), \quad B = (b_1, \dots, b_m).$$

При слиянии сравниваются первые ещё не использованные элементы обеих последовательностей. Меньший из них добавляется в результирующий массив.

Слияние выполняется за

$$\Theta(k + m).$$

Для всего алгоритма получается рекуррентное соотношение

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$

Отсюда

$$T(n) = \Theta(n \log n).$$

Это справедливо для лучшего, среднего и худшего случаев.

Свойства Merge Sort:

- время работы — $\Theta(n \log n)$;
- классическая реализация требует $O(n)$ дополнительной памяти;
- алгоритм устойчив;
- хорошо подходит для связанных списков;
- эффективен для внешней сортировки;
- хорошо допускает распараллеливание.

В отличие от быстрой сортировки, сортировка слиянием имеет гарантированную сложность $O(n \log n)$.

58.5 Быстрая сортировка

Быстрая сортировка (*Quick Sort*) также основана на принципе “разделяй и властвуй”.

Выбирается некоторый элемент массива, называемый **опорным элементом** (*pivot*).

Затем массив разделяется так, чтобы элементы с меньшими ключами находились с одной стороны от опорного элемента, а с большими — с другой.

После разделения обе части сортируются рекурсивно.

Если разделение массива происходит примерно пополам, то

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n),$$

откуда

$$T(n) = \Theta(n \log n).$$

Однако при неудачном выборе опорного элемента одна часть может содержать $n - 1$ элементов, а другая — ни одного. Тогда

$$T(n) = T(n - 1) + \Theta(n),$$

и

$$T(n) = \Theta(n^2).$$

Таким образом:

$$T_{\text{avg}}(n) = \Theta(n \log n), \quad T_{\text{worst}}(n) = \Theta(n^2).$$

Для уменьшения вероятности плохого разбиения применяют:

- случайный выбор опорного элемента;
- выбор среднего из первого, среднего и последнего элементов;
- другие эвристики выбора *pivot*.

Свойства Quick Sort:

- среднее время — $\Theta(n \log n)$;
- худшее время — $\Theta(n^2)$;
- обычно работает на месте;
- средний размер стека рекурсии — $O(\log n)$;
- классический вариант неустойчив;
- обладает хорошей локальностью обращений к памяти;
- на практике часто является одним из наиболее быстрых алгоритмов сортировки массивов в оперативной памяти.

Несмотря на квадратичный худший случай, Quick Sort часто превосходит Merge Sort и Heap Sort на массивах за счёт малой константы и эффективного использования кэш-памяти.

58.6 Пирамидалная сортировка

Пирамидалная сортировка (*Heap Sort*) использует структуру данных **двоичная куча**.

Максимальной кучей называется двоичное дерево, для каждой вершины которого выполнено

$$a_{\text{parent}} \geq a_{\text{child}}.$$

Поэтому максимальный элемент всегда расположен в корне.

Если куча хранится в массиве, то для вершины с индексом i индексы потомков можно определить арифметически.

Например, при нумерации с единицы:

$$\text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1.$$

Сортировка состоит из двух этапов:

1. из исходного массива строится максимальная куча;
2. корневой элемент последовательно меняется с последним элементом ещё не отсортированной части, после чего восстанавливается свойство кучи.

Кучу из n элементов можно построить за

$$O(n).$$

Извлечение максимального элемента и восстановление кучи требует

$$O(\log n).$$

Такая операция выполняется n раз, поэтому

$$T(n) = O(n \log n).$$

Причём эта оценка справедлива и в худшем случае.

Свойства:

- гарантированное время — $O(n \log n)$;
- дополнительная память — $O(1)$;
- алгоритм неустойчив;
- не требует дополнительного массива;
- обычно медленнее Quick Sort на практике из-за худшей локальности памяти.

58.7 Сортировки без сравнений

Нижняя оценка $O(n \log n)$ относится только к алгоритмам, основанным на сравнениях.

Если о ключах известна дополнительная информация, возможна сортировка за линейное время относительно числа элементов.

Сортировка подсчётом

Пусть ключи являются целыми числами из диапазона

$$0, \dots, k - 1.$$

Создаётся массив счётчиков

$$C[0], \dots, C[k - 1],$$

где $C[i]$ содержит число элементов с ключом i .

После подсчёта элементов массив можно восстановить в отсортированном виде.

Сложность:

$$T(n, k) = O(n + k),$$

память:

$$O(k).$$

При подходящей реализации алгоритм является устойчивым.

Если $k = O(n)$, время работы линейно:

$$T(n) = O(n).$$

Поразрядная сортировка

Radix Sort сортирует ключи последовательно по отдельным разрядам.

Например, для десятичных чисел можно сначала выполнить устойчивую сортировку по единицам, затем по десяткам, сотням и т.д.

Если число разрядов равно d , а сортировка одного разряда требует $O(n + k)$ операций, то

$$T(n) = O(d(n + k)).$$

В качестве внутреннего алгоритма часто используется устойчивая сортировка подсчётом.

Карманная сортировка

В **Bucket Sort** диапазон возможных значений делится на несколько интервалов — “карманов”. Элементы распределяются по карманам, каждый карман сортируется отдельно, после чего результаты объединяются.

При подходящем распределении данных среднее время может составлять

$$O(n).$$

Однако эффективность сильно зависит от распределения входных значений.

58.8 Оценка временной и пространственной сложности

Основные алгоритмы можно сравнить следующим образом:

Алгоритм	Лучший случай	Средний случай	Худший случай	Память
Вставки	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Выбор	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Пузырьковая	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Слияние	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Быстрая	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$
Пирамидальная	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Подсчётом	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$

Здесь k — размер диапазона ключей.

Для Quick Sort указана типичная глубина стека рекурсии. В худшем случае она может достигать $O(n)$, если специально не ограничивать рекурсию.

Асимптотическая сложность является важной, но не единственной характеристикой. Например, два алгоритма с одинаковой сложностью

$$O(n \log n)$$

могут существенно различаться на практике из-за:

- числа операций сравнения;
- числа перемещений элементов;
- характера обращений к памяти;
- использования кэш-памяти;
- стоимости выделения дополнительной памяти.

58.9 Нижняя оценка для сортировок сравнениями

Рассмотрим произвольный алгоритм сортировки, который получает информацию о взаимном порядке элементов только посредством сравнений.

Работу такого алгоритма можно представить в виде **дерева решений**.

Каждая внутренняя вершина соответствует сравнению двух элементов. В зависимости от результата сравнения алгоритм переходит в одну из ветвей.

Для n различных элементов существует

$$n!$$

возможных перестановок.

Чтобы алгоритм мог отличить каждую перестановку от всех остальных, дерево решений должно иметь не менее $n!$ листьев.

Если высота бинарного дерева равна h , то оно имеет не более

$$2^h$$

листьев.

Следовательно,

$$2^h \geq n!,$$

откуда

$$h \geq \log_2(n!).$$

По формуле Стирлинга

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n,$$

поэтому

$$\log_2(n!) = \Theta(n \log n).$$

Таким образом, для любого алгоритма сортировки сравнениями существует вход, на котором необходимо выполнить не менее

$$\Omega(n \log n)$$

сравнений.

Следовательно, асимптотически оптимальные сортировки сравнениями имеют сложность

$$\Theta(n \log n).$$

К ним относятся, например, Merge Sort и Heap Sort.

Это ограничение не противоречит существованию сортировки подсчётом или Radix Sort с линейной сложностью, поскольку они используют дополнительную информацию о структуре ключей и не являются чистыми сортировками сравнениями.

58.10 Внешняя сортировка и выбор алгоритма

Если все данные помещаются в оперативную память, говорят о **внутренней сортировке**.

Если объём данных превышает доступную оперативную память и существенная часть данных хранится на внешнем устройстве, применяется **внешняя сортировка**.

В этом случае основной стоимостью становятся не арифметические операции, а операции ввода-вывода.

Классическим методом является **внешняя сортировка слиянием**.

Она выполняется следующим образом:

1. из внешней памяти считывается блок данных, помещающийся в ОЗУ;
2. блок сортируется внутренним алгоритмом;
3. отсортированный блок записывается на диск; такой блок называется **серией** или **run**;
4. полученные серии объединяются многопутевым слиянием;
5. процесс повторяется до получения одной отсортированной последовательности.

Слияние особенно удобно для внешней памяти, поскольку данные читаются последовательно большими блоками.

При выборе алгоритма сортировки следует учитывать свойства задачи.

Для небольших массивов хорошо подходит сортировка вставками.

Для произвольных массивов в оперативной памяти на практике часто эффективна Quick Sort или гибридные алгоритмы на её основе.

Если требуется гарантированное

$$O(n \log n)$$

и минимальная дополнительная память, может использоваться Heap Sort.

Если важна устойчивость и допустимо выделение дополнительной памяти, подходит Merge Sort.

Если ключи — целые числа из небольшого диапазона, эффективна сортировка подсчётом.

Для составных целочисленных или строковых ключей может применяться Radix Sort.

Для данных, не помещающихся в оперативной памяти, естественным выбором является внешняя сортировка слиянием.

Таким образом, универсально “лучшего” алгоритма сортировки не существует: выбор определяется размером данных, типом ключей, требованием устойчивости, ограничениями памяти и архитектурой вычислительной системы.